REALIZATION CALCULUS

Komorebi Math

The Python Library

A practical manual

Komorebi P.

Contents

Preface iii

Paths through the manual iv

I Using the library 1

1 First calculations 2

2 The stable interface 12

3 The executable language 14

II Operators and coordinates 23

4 Coordinate systems and recurrence models 24

5 Analysis and synthesis 31

6 Redundant frames 35

7 Metric packets 38

8 Distortion cocycles 41

9 Convex packets 47

III Arithmetic, functions and signals 52

10 Sequences and local comparisons 53

11 Normalized generating functions 56

12 Special functions 61

13 Positive quadratic-form tails 64

14 Finite join quotients 68

15 Signal processing and recovery 70

IV Nonlinear calculus and integration 89

16 Nonlinear calculus 90

17 Scalable nonlinear execution 95

18 Streamed packet jets 100

19 Finite measures and integration 106

20 Typed finite expressions 112

21 Finite carrier completion 125

V Models, portable results and complete programs 126

22 Quantum and memory models 127

23 Portable persistence 131

24 Export and LaTeX 136

25 Behavioral fixtures and profiling 140

26 Extensions and companion C integration 142

27 Worked calculations and independent controls 145

A Public API reference 161

B Thematic API inventory 181

C Platforms, backends and PowerShell 188

D Upgrading applications 195

E Reproducibility and qualification 197

F Glossary 201

Preface

Komorebi Math turns finite constructions of Realization Calculus into Python objects that can be inspected, combined and tested. Operators act on specified spaces. Coordinates describe the same finite data in different forms. Packets organize projections, transfers and refinements. Certificates record what a computation established and the assumptions under which it did so.

This manual follows that working language from installation to complete mathematical programs. It begins with small calculations and the stable interface, then develops coordinates, arithmetic, signals, nonlinear calculus, integration and finite quantum models. Later chapters explain how to preserve and exchange results. The reference appendices collect the broader interfaces, operating requirements and maintainer procedures.

The distinction between exact arithmetic, numerical evidence and conditional conclusions is part of the mathematics of the library. A finite calculation establishes a statement about its supplied objects under its stated rules. A saved record preserves that evidence; a fresh validity claim requires the relevant operation to be run again. The chapters explain these boundaries where they matter to the calculation.

Reading and running the examples

Begin with the installation and first programs, then follow the subject that matches the problem at hand. The final worked-calculation chapter brings several domains together through examples with independent controls. The glossary explains the recurring language of realizations, packets, coordinates, residuals and certificates.

Code listings distinguish independently executable Python examples from interface sketches and contextual fragments. A sketch shows an API or a formula and may rely on objects defined in the surrounding discussion. Shell commands assume the working directory and environment stated in their section. Expected output is shown separately from the program that produces it.

The executable Python examples run with assertions enabled. Dependencies and finite input conventions are stated alongside the examples. Optional GPU workflows and external LaTeX compilation require their own environments; a successful CPU example does not qualify those runtimes.

Printed cross-references give section numbers and remain clickable in the PDF. File and script names in commands are literal. API identifiers retain their exact Python spelling.

Paths through the manual

Choose a subject from the table, then follow the numbered cross-references. The opening chapters establish installation and certificate semantics. The appendices collect reference material and operating requirements.

Goal	Guide
Construct operators, seeds and packets	Unified language (§ 3)
Change bases or transport finite data	Coordinate atlas (§ 4)
Analyze and synthesize coordinates	Analysis and synthesis (§ 5), redundant frames (§ 6)
Process finite or streamed signals	Signal processing (§ 15), Fourier recovery (§ 15.14)
Compute derivatives and nonlinear packet jets	Nonlinear calculus (§ 16), scalable execution (§ 17)
Integrate polynomials or compare exact expressions	Finite integration (§ 19), typed expressions (§ 20)
Work with recurrences and generating functions	Sequences (§ 10), generating functions (§ 11)
Bound positive quadratic-form tails	Quadratic forms (§ 13)
Evaluate the supported special-function families	Special functions (§ 12)
Form finite quotients	Join semilattices (§ 14)
Study finite quantum models	Quantum and memory contracts (§ 22), Dirac modes (§ 22.6)

For complete programs with independent controls, turn to the worked-calculation chapter. For interface details, use the public API reference and thematic inventory. The glossary gathers the recurring mathematical and software terms.

PART I

Using the library

A calculation begins with a choice of objects, arithmetic and evidence. These chapters establish the working environment and the common language used throughout the manual.

1 First calculations

Komorebi Math provides reusable finite mathematical objects: operators, packet decompositions, coordinate changes, exact sequences, signal models, integration rules, and quantum realizations. Results carry the assumptions and evidence of the computation that produced them. This guide starts with a working environment and two small programs, then shows how to choose a domain and interpret its results.

The stable namespace contains 131 reviewed exports with a 1.x compatibility contract. The broader domain APIs document their own mathematical scope. Original software is distributed under the MIT License, with third-party notices retained. These instructions use a checkout or a supplied wheel.

Read this guide for first use, the worked examples (§ 27) for complete calculations, and the public reference (§ A) for domain interfaces.

1.1 Installation

The supported CPU targets are Python 3.11, 3.12 and 3.13 on Linux and Windows. The base dependency is NumPy 1.26 or newer. Start in the repository root and use the same interpreter for pip and the program. The platform policy (§ C) gives the dependency and support scope.

On Linux or macOS with Python 3.12 available:

SHELL

```
python3.12 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install .
python -m pip check
python -m komorebi_math info
python -m komorebi_math verify stable
```

The activation commands also apply to POSIX shells on macOS. macOS is outside the declared support matrix.

On Windows PowerShell:

POWERSHELL

```powershell
py -3.12 -m venv .venv
$py = (Resolve-Path .\.venv\Scripts\python.exe).Path
& $py -m pip install --upgrade pip
& $py -m pip install .
& $py -m pip check
& $py -m komorebi_math info
& $py -m komorebi_math verify stable
```

Use any supported Python minor. Activation is unnecessary when invoking the resolved Windows interpreter. When diagnosing an installation, use `python -m komorebi_math info` to check the package and interpreter together.

1.1.1 Optional dependencies

Install only the dependencies required by the workflow. These are the extras declared by the current package metadata.

Extra	Dependencies	Typical use
`exact`	SymPy >= 1.14	Symbolic linear algebra and Smith decompositions
`scientific`	SciPy >= 1.11, mpmath >= 1.3	Scientific and high precision numerical routines
`plot`	Matplotlib >= 3.8	Plotting examples
`yaml`	PyYAML >= 6.0	YAML import and export
`jax`	JAX >= 0.4	JAX directional differentiation
`torch`	PyTorch >= 2.1	PyTorch directional differentiation
`autodiff`	JAX and PyTorch	Both differentiation adapters
`test`	pytest 8.4.2	Source tests
`dev`	pytest, build, mypy, Ruff, MkDocs, PyMdown Extensions	Development and mathematical documentation tools
`all`	Exact, scientific, plot, and YAML dependencies	Broad mathematical dependencies

SHELL

```
python -m pip install ".[exact,scientific,yaml]"
```

Standard-library integer, rational, modular, and many exact sequence operations work without SymPy. The `all` extra does not include autodiff, CUDA, or development tools. Maintainer dependencies are described in the release guide.

Directed quadratic-form enclosures require the qualified arithmetic backend **mpmath 1.3.0**, even though the general scientific extra has a lower-bound dependency declaration:

SHELL

```
python -m pip install ".[scientific]" "mpmath==1.3.0"
```

CuPy is selected separately for the local CUDA version. Import presence does not establish that a GPU or an autodiff adapter works. The doctor command reports the environment, and hardware qualification has a separate scope:

SHELL

```
python -m komorebi_math doctor
```

1.1.2 Local wheel installation

A source checkout need not contain built distributions. Maintainers can create them with:

SHELL

```
python -m pip install "setuptools==82.0.1" wheel
python verification/build_distributions.py
```

Install the wheel in a separate environment:

SHELL

```
python -m pip install dist/komorebi_math-1.0.0-py3-none-any.whl
python -m pip check
python -m komorebi_math verify stable
```

The source distribution is `komorebi_math-1.0.0.tar.gz`. Installing it invokes a source build. A fully offline installation needs a local wheelhouse for all dependencies and build tools. See the release guide (§ E) for reproducible builds and clean installed-package checks.

1.2 First program: packet analysis and reconstruction

A packet system stores a finite family of projectors. The following two projectors separate the two coordinates of a vector. Their idempotence, pairwise annihilation, and sum-to-identity conditions are checked by the producer. NumPy execution gives a numerical certificate, even when the observed residuals are zero.

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.stable import Exactness, PacketSystem

packets = PacketSystem(
    [np.diag([1.0, 0.0]), np.diag([0.0, 1.0])],
    names=("left", "right"),
)
certificate = packets.certificate().require()
vector = np.array([2.0, 3.0])
coordinates = packets.analyze(vector)
reconstructed = coordinates.reconstruct()

np.testing.assert_array_equal(reconstructed, vector)
assert certificate.exactness is Exactness.NUMERICAL
print(reconstructed.real.tolist())
print(dict(certificate.residuals))
```

Expected output:

OUTPUT

```
[2.0, 3.0]
{'idempotence': 0.0, 'annihilation': 0.0, 'completeness': 0.0}
```

For a deliberately invalid family, construction normally raises. To inspect the failed record in a diagnostic example, pass `validate=False`, inspect its certificate, and call `.require()` at the acceptance boundary. Disabling validation does not make the failed family satisfy the packet axioms.

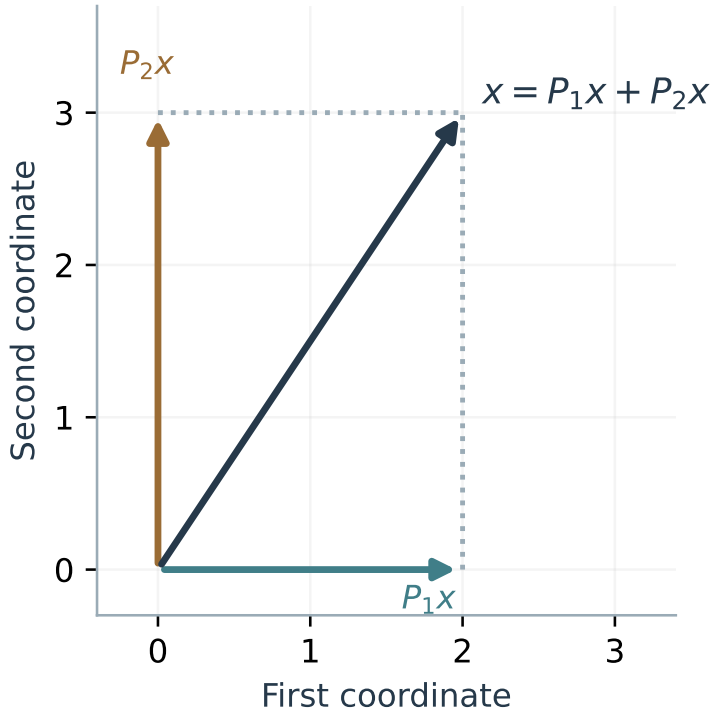


Figure 1.1: Two coordinate projectors decompose the vector into its horizontal and vertical packets. Their sum reconstructs the original vector.

1.3 Exact coordinate changes

Use integers and `Fraction` values when the intended scalar domain is rational. The stable exact transform below implements a shear and checks an inverse roundtrip without a symbolic backend.

PYTHON | Executable example

```python
from fractions import Fraction
from komorebi_math.stable import ExactCoordinateTransform, Exactness

Q = Fraction
transform = ExactCoordinateTransform(
    "A", "B", ((1, Q(1, 2)), (0, 1)),
    "GUIDE-EXACT-SHEAR", "exact rational shear",
)
vector = (Q(2), Q(3))
mapped = transform.apply(vector)
assert mapped == (Q(7, 2), Q(3))
assert transform.inverse().apply(mapped) == vector
assert transform.certificate().require().exactness is Exactness.EXACT
print(mapped)
```

Expected output:

OUTPUT

```
(Fraction(7, 2), Fraction(3, 1))
```

A valid transform record can still describe a singular map. Inspect the invertibility diagnostics before calling an inverse. The coordinate atlas (§4) covers arbitrary finite bases, polynomial families, recurrence coordinates, tensors, and formal charts.

1.4 Certificates and saved evidence

`Certificate` separates validity from evidence class. `EXACT` describes exact finite arithmetic under the producer's hypotheses. `NUMERICAL` describes a floating computation and its error rule. `CONDITIONAL` retains unchecked assumptions, and `HEURISTIC` or `PLANNED` describes weaker or unfinished claims. API stability is a separate property.

The constructor requires a Boolean validity flag. A proposed true certificate becomes invalid when a residual is nonfinite or nonreal, or its supplied tolerance is nonfinite or negative. An invalid report may retain an infinite residual as a failure diagnostic. The producer still owns the mathematical comparison rule: the base class does not recompute every identity or impose one universal norm on all residuals.

Residuals are copied into a read-only mapping. Use `dict(certificate.residuals)` for an ordinary dictionary display. Residual values must be supported immutable scalars; a mutable NumPy array, including a zero-dimensional array, cannot serve as live certificate evidence. Diagnostics are copied only at the outer dictionary level and are not deeply immutable. `.require()` accepts a live true certificate; it does not discharge unchecked assumptions.

Portable loading returns historical evidence. A stored true flag appears as `recorded_valid=True`, while `current_valid=False` and `revalidation_required=True`. Rerun the producing operation to obtain a current certificate. Equality with the old record and a valid SHA-256 envelope establish record consistency, not fresh mathematical validation or authorship.

PYTHON | Executable example

```python
from komorebi_math.stable import Certificate, Exactness, ValidationError
from komorebi_math.export import dumps_portable, loads_portable

original = Certificate(True, Exactness.EXACT, {"identity": 0})
restored = loads_portable(dumps_portable(original))
assert restored == original
assert restored.recorded_valid
assert not restored.current_valid
assert restored.revalidation_required
try:
    restored.require()
except ValidationError as error:
    assert "revalidation" in str(error)
else:
    raise AssertionError("a saved certificate cannot approve a new computation")
```

`ComputationResult.require()` returns its value after enforcing the certificate. Its `.valid` property uses current validity. Its `.exact` property reports only the evidence class, so it can be true even when the result is invalid or historical. `ConditionalResult` enforces the conditional label and exposes its certificate directly. Presentation JSON and typed portable persistence are different formats. See persistence schemas (§ 23).

1.5 Choosing an API and a mathematical domain

Use `komorebi_math.stable`, `komorebi_math.v1`, or `komorebi_math.api.v1` for the 131-export stable compatibility surface. They expose the same curated objects. Broader domain namespaces provide additional mathematical interfaces under their own documented contracts. An object does not acquire the stable compatibility promise merely by being importable from the top-level package.

Goal	Reading route
Operators, seeds, packets, transfer blocks, refinement	Public API (§ A), unified language (§ 3)
General analysis and synthesis, redundant frames	Analysis-synthesis systems (§ 5), frame algebra (§ 6)
Exact sequences and local OEIS comparisons	Sequence and OEIS tools (§ 10)
Generating functions, finite modular and digit algebra	Normalized generating functions (§ 11)
Filters, streaming, STFT, and finite signals	Production signal processing (§ 15)
Fourier jump recovery	Fourier recovery (§ 15.14)
Exact integration and typed expression evaluation	Integration (§ 19), typed expressions (§ 20)
Derivatives, jets, nonlinear trees, and lifting	Nonlinear calculus (§ 16)
Quadratic-form tails and factor spectra	Quadratic forms (§ 13)
Bernoulli-Hurwitz, Gamma, Meixner, and scalar Racah values	Special functions (§ 12)
Exact finite join congruences and quotients	Finite join semilattices (§ 14)
Quantum memory, closure, and spectral dimension	Quantum and memory contracts (§ 22)
Finite four-spinor Dirac modes	Dirac modes (§ 22.6)

The worked-example guide develops six mathematical workflows with independent controls. Fourier recovery, finite semilattices, and Dirac modes use explicit submodule imports. This makes the chosen mathematical interface explicit.

1.5.1 Conventions that affect a computation

Exact finite arithmetic and floating linear algebra have different acceptance rules. A result for a stored binary64 factor concerns that exact stored factor; it does not include an unprovided measurement or rounding model. A finite prefix matching a local OEIS record does not prove an infinite sequence identity. Scalar Racah evaluation does not establish positive orthogonality.

For periodic Fourier recovery, the normalized coefficient is

$$\hat{f}(n) = \frac{1}{T}\int_0^T f(x)e^{-2\pi inx/T}\,dx.$$

Derivative moments are `2*pi*1j*n*fhat[n]` for any positive period. Recovery needs the specified consecutive modes above the declared background bandlimit. Its numerical consistency checks do not establish that the unknown background really is bandlimited or that measured data are noiseless.

For the derivative Schrödinger equation `psi'=-i*H*psi`, the memory kernel is

$$K(t) = -H_{PQ}e^{-itH_{QQ}}H_{QP}.$$

The Hamiltonian self-energy retains its resolvent convention. In the open upper half-plane, `kernel.laplace(-1j*z)=-1j*self_energy(z)`. An infinite Laplace integral, a rational continuation, and a finite-horizon transform are separate operations:

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.dynamics.memory import MemoryKernel
from komorebi_math.stable import ValidationError

kernel = MemoryKernel([[1.0]], [[1.0]], [[1.0]])  # K(t)=exp(t)
np.testing.assert_allclose(kernel.rational_transform(0), [[-1.0]])
np.testing.assert_allclose(kernel.finite_laplace(1, 2), [[2.0]])
try:
    kernel.laplace(0)
except ValidationError:
    pass
else:
    raise AssertionError("the infinite Laplace integral diverges")
```

The Laplace half-plane test uses the full hidden realization and is conservative when hidden modes cancel. These matrix calculations remain numerical.

1.6 Command-line workflows and verification

Invoke the module through the resolved interpreter. `verify stable` and `release check` work with the base package. The complete `verify all` suite requires the `exact`, `scientific` and `yaml` extras.

SHELL

```shell
python -m komorebi_math --help
python -m komorebi_math features
python -m komorebi_math api check --stable
python -m komorebi_math api check --strict-signatures
python -m komorebi_math verify stable
python -m komorebi_math verify all
python -m komorebi_math release check
```

The CLI registry describes declared capabilities. It is not a live hardware report, and an installed finite suite is not the complete source test suite. The source-only examples and release tools require a checkout or unpacked source distribution. Two connected tours are available:

SHELL

```
python examples/library_tour.py --json
python examples/worked_examples.py
```

Source tests, documentation builds and distribution qualification are maintainer workflows. The release guide (§ E) gives the complete sequence, including dependency isolation, optional backend replay and platform records. Use the same interpreter for installation and every check.

1.7 Troubleshooting

Observation	Next action
Wrong version or unexpected imports	Print `sys.executable` and `komorebi_math.__file__`; install through that interpreter
Missing optional dependency	Install the extra required by the named operation
Directed quadratic forms reject the backend version	Install the qualified `mpmath==1.3.0` backend
A certificate fails `.require()`	Inspect its message, residual rule, checked hypotheses, and retained assumptions
A loaded certificate requires revalidation	Rerun its producer; retain the loaded record as historical evidence
Exact input is rejected	Use the API's declared integer or rational domain; avoid fractional truncation and Boolean indices
Numerical recovery is unstable	Inspect singular values, thresholds, and residuals; the declared model and data conditioning still matter
The full gate cannot build documentation	Install MkDocs and `pymdown-extensions==10.16.1`, then rerun the strict documentation command
Artifact parity fails after editing	Rebuild wheel and source distribution from the edited source

Include the package version, interpreter, dependency versions, minimal input, and complete certificate or exception when reporting an issue. Keep raw measurements and numerical tolerances with the result so another reader can replay the same finite computation.

2 The stable interface

`komorebi_math.stable`, `komorebi_math.v1` and `komorebi_math.api.v1` expose 131 object-identical symbols for the 1.x line. They support a complete finite workflow: construct operators and seeds, analyze coordinates, decompose data into packets, refine representations, certify results, save portable records and render mathematical documents.

2.1 Compatibility promise

A stable symbol retains its exported name, broad kind, documented signature and documented finite mathematical behavior throughout the 1.x line. Persistence promises also retain the declared codec identity and provide adjacent migrations when a codec version changes.

Additive stable names may appear in a later minor release. Removal or an incompatible change requires the deprecation process (§ D.6) and normally a major release. A correctness fix still has to state the corrected behavior and its effect on existing applications.

PYTHON | Executable example

```python
import komorebi_math.stable as stable
import komorebi_math.v1 as v1
import komorebi_math.api.v1 as api_v1

assert stable.__all__ == v1.__all__ == api_v1.__all__
```

2.2 Three independent checks

The structural manifest records exported names, kinds, implementation identities and callable signatures. Behavioral fixtures check committed mathematical examples. The persistence contract checks record tags, represented Python types, codec versions and migrations. Passing one does not replace the other two.

SHELL

```shell
python -m komorebi_math api check --stable
python -m komorebi_math verify stable
python -m komorebi_math release check
```

Programmatically:

PYTHON | Executable example

```
from komorebi_math.stable import check_stable_api, check_stable_contract

check_stable_api().certificate.require()
check_stable_contract().certificate.require()
```

2.3 Broader domain APIs

An object being importable from the top-level package or a domain module does not automatically place it under the stable contract. Explicit research APIs remain available with their documented scope. The separate `komorebi_math.beta` compatibility namespace is retained for existing applications, with its own signature checks.

The stable contract does not promise constant performance, availability of optional native backends, compatibility of arbitrary extension codecs or global analytic conclusions from finite calculations. See the platform policy (§ C) and persistence guide (§ 23).

3 The executable language

Komorebi Math uses one sequence of operations across its mathematical domains:

OUTPUT

```
seed
  -> address
  -> operate
  -> project
  -> transfer
  -> refine
  -> constrain
  -> complete
  -> reconstruct
```

The language organizes how finite data are generated, represented and reconstructed. A signal packet, a module summand, a symmetry sector and a residual deformation class still have distinct mathematical meanings. Their interfaces agree only where the corresponding operations and identities are preserved.

3.1 Seed

A seed is finite generating data for an operator action.

PYTHON | Contextual fragment

```
SeedSystem(operators, seeds)
```

Implemented operations include orbit hulls, generation certificates, finite free presentations, semisimple seed rank, rational single operator seed rank, and finite seed searches.

3.2 Address

An address names a legal operator path or packet coordinate.

PYTHON | Contextual fragment

```
OperatorGrammar
residue_analyze(values, modulus)
crt_many(residues, moduli)
polyphase_analyze(signal, order)
```

Typed paths, residue addresses, Chinese remainder addresses, mixed radix signal addresses, and packet tree paths are implemented.

3.3 Operate

PYTHON | Contextual fragment

```
OperatorRealization(grammar, spaces, operators)
OperatorAlgebra(generators)
StructuredOperator
MatrixSemigroup(generator)
```

Relations are checked exactly with an exact backend or numerically with a residual and tolerance. Structured operators and finite semigroups provide executable domain realizations.

3.4 Project

PYTHON | Contextual fragment

```
PacketSystem(projectors)
PacketIR.from_dense(matrix, layout)
ConditionalExpectation(weights, partition)
```

Projection covers algebraic packets, structured computational blocks, signal phases, and finite conditional expectation packets.

3.5 Transfer

PYTHON | Contextual fragment

```
PacketOperator
TransferGraph
PolyphaseConvolution
PacketSemigroup
```

Transfer blocks reconstruct operators, compose by packet matrix multiplication, propagate through paths, and specialize to convolution, semigroup, Markov, and memory systems.

3.6 Computational choice

The computational namespace implements packet intermediate representations, structured operators, low rank recompression, support propagation, Krylov methods, precision allocation, exact finite tree compilation, scheduling, and optional CuPy execution.

The compiler is exact for the supplied finite additive tree. A precision assignment is exact for the supplied integer budget. Threshold structure discovery remains heuristic. Cost and roofline results are declared models rather than hardware guarantees.

3.7 Signal realization

Signals realize the address, project, transfer, refine, and reconstruct stages through finite polyphase coordinates and packet trees.

PYTHON | Contextual fragment

```
coordinates = polyphase_analyze(signal, order)
filtered = polyphase_convolution(signal, kernel, order)
tree = HaarPacketTree.analyze(signal, depth)
reconstruction = retained_packet_reconstruct(signal, packet, approximation)
```

The signal implementation includes exact finite coordinate identities, filter bank reconstruction, additive best basis selection, truncated power singular packets, stable separated jump recovery under declared assumptions, nonlinear residue products, and Smith chart coordinates.

A recovery certificate does not infer unknown smoothness or noise assumptions from the data. It records which finite threshold conditions were checked and which model hypotheses remain external.

3.8 Dynamics realization

Dynamics realizes operate, transfer, refine, constrain, complete, and reconstruct through finite matrix evolutions and finite coarse models.

PYTHON | Contextual fragment

```
semigroup = MatrixSemigroup(generator)
lumping = MarkovLumping(generator, partition)
posterior = BayesianUpdate(prior, likelihood)
fractional = fractional_power(operator, exponent)
reduction = FeshbachReduction(operator, retained)
kernel = reduction.memory_kernel()
```

The implementation includes semigroup and resolvent defects, compatible finite towers, exact Markov lumping, conditional expectation packets, Bayesian sufficiency, fractional spectral calculus, Green operators, finite Feynman Kac transfer, Schur Feshbach elimination, rational memory kernels, and stable phase type survival models.

Finite matrix theorems do not claim an unbounded continuum construction. Completion certificates state the topology, compatibility, and stability assumptions that were actually checked.

3.9 Analytic realization

The analytic realization uses residue projection to turn one entire function into a finite cyclic differential system. Reciprocal, logarithmic, Appell, and theta families then operate on the resulting coefficient coordinates.

PYTHON | Contextual fragment

```
packets = cyclic_packets(coefficients, order)
section = signed_section(order, residue, z)
coefficients = reciprocal_coefficients(order, parameter, degree)
zeros = positive_zero_string(order, count)
```

Exact coefficient operations and numerical zero operations remain distinct. A zero certificate checks only the returned residuals, ordering, and requested finite computation.

3.10 Quantum realization

Quantum systems realize seed, project, transfer, constrain, and reconstruct through observable algebras, symmetry sectors, circuit paths, moving packets, channels, error correction, and effective dynamics.

PYTHON | Contextual fragment

```
packets = cyclic_symmetry_projectors(symmetry, order)
circuit = CircuitPacketExpansion(gates, packet_systems)
channel = QuantumChannel(kraus_operators)
code = QuantumCode(projector, errors)
transport = KatoTransport(moving_packet_system)
```

Every operator acts on a declared finite Hilbert space. Continuum limits and gravitational endpoints require additional mathematics and are not represented as executable conclusions.

3.11 Refine

PYTHON | Contextual fragment

```
PacketRefinement
RefinementTower
WeightedTransferGraph
LatticePacketData
SmithCongruenceProblem
SemigroupRefinement
```

Implemented refinement includes coarse and fine packet relations, residue and CRT refinement, weighted transfer paths, filtered inverses, integral separation, Smith congruences, operator defects, semigroup Duhamel defects, and finite compatible towers.

3.12 Constrain

PYTHON | Contextual fragment

```
LinearObstructionProblem
LinearLiftProblem
PolynomialLiftProblem
TransportGraph
recover_separated_jumps
MarkovLumping
BayesianPartition
```

The finite constraint layer contains cokernel obstructions, prime power correction spaces, lift trees, finite holonomy, signal recovery under declared threshold hypotheses, exact Markov lumping, and fixed Bayesian sufficiency.

The nonlinear namespace implements packet jets, Faà di Bruno composition, rooted nonlinear operator trees, graded operations, nonlinear Schur and Lyapunov-Schmidt elimination, finite filtered lifting, and obstruction spectral sequences. Smooth Kato transport is shared by the nonlinear and quantum namespaces through one implementation.

3.13 Complete

There is no single universal completion object. A Banach inverse limit, a finite lift forest, a compatible semigroup tower, and a global arithmetic system require different runtime representations.

The library implements finite semigroup towers, finite hidden state realizations, finite survival systems, finite Kato transport, Euclidean transfer, and bounded Hamiltonian tower diagnostics. These finite representations do not construct a general continuum semigroup.

3.14 Reconstruct

Implemented reconstruction includes:

PYTHON | Contextual fragment

```
PacketCoordinates.reconstruct()
PacketOperator.reconstruct()
ResidueCoordinates.reconstruct()
PolyphaseCoordinates.reconstruct()
PeriodicFilterBank.synthesize()
PrunedPacketCoefficients.reconstruct()
retained_packet_reconstruct(...)
FeshbachReduction.memory_kernel()
```

Signal reconstruction separates smooth approximation error from packet estimation error. Memory reconstruction separates retained dynamics from an auxiliary hidden realization.

3.15 Execution classes

Class	Meaning
exact	exact algebra or arithmetic under checked assumptions
numerical	floating point computation with residual and tolerance
conditional	valid when listed external assumptions hold
heuristic	discovery or estimation without a complete guarantee
planned	reserved namespace without a production implementation

Examples:

- a SymPy Smith solve is exact
- a NumPy cokernel solve is numerical
- a jump recovery threshold can be checked numerically while regularity and noise hypotheses remain conditional
- the finite exact-lumpability law is tested numerically for the supplied Markov matrix and partition
- the NumPy Feshbach resolvent certificate is numerical
- restricted automatic structure discovery is heuristic

3.16 Arithmetic and sequence realization

For arithmetic and sequences, the initial coefficient family supplies the seed. Residue and Mahler coordinates provide addresses, while convolution and recurrence evolution act on them. Local selection, triangular transport and prime power refinement connect the representations. Congruences and recurrence relations impose constraints, and reconstruction recovers the requested coefficients or terms from the finite model.

3.17 Deformation and Langlands realization

In deformation and arithmetic models, the unified stages acquire additional finite meanings. A residual class is a seed, a lift level is an address, reduction and transport are operators, packet cohomology and finite Hecke idempotents are projections, stable cores and support impose constraints, and a completion report records whether every required gate has been supplied.

PYTHON | Contextual fragment

```
forest = residue_forest(levels, prime=p)
report = forest.report()

packet = TameWeilPacket(FrobeniusOrbit(q, degree, exponent))
packet.relation_certificate().require()

completion = AutomorphicCompletionReport(gates)
result = completion.result()
```

The same words do not identify deformation support with a finite spectral packet. Fitting support is computed from a module presentation. Central idempotent packets are used only where a finite algebra actually decomposes as a product.

Finite-field and finite-group combinatorics have exact finite branches. Matrix certificates in the tame and Heisenberg models use numerical residuals when their arithmetic is floating complex. Ramification comparisons, automorphic induction plans, patching support, compatible propagation and final completion remain conditional whenever they rely on external arithmetic existence or comparison theorems.

3.18 Portable output realization

Portable output supplies the stage after reconstruction. A mathematical value may be normalized, rendered, written, and compiled without changing its execution classification.

OUTPUT

```
reconstruct
  -> normalize
  -> render
  -> write
  -> optionally compile LaTeX
  -> validate the written output
```

`MathDocument` is renderer independent. Ordinary prose is escaped. Equations contain trusted mathematical LaTeX. `RawLatex` is an explicit trusted escape hatch. `export_data` supports data formats and report formats. `compile_latex` is a local tool with shell escape disabled by default.

Export success certifies that the requested file was written and, for PDF, that the local compiler returned success and produced a nonempty PDF. It does not add a mathematical proof to the underlying value.

3.19 Advanced nonlinear realization

Nonlinear realization gives the constrain and complete stages a higher-order meaning. A derivative provider supplies the local multilinear data. Packet jets project each derivative slot. Faà di Bruno composes those jets. Rooted trees encode finite nonlinear iteration. Schur and Lyapunov-Schmidt reductions eliminate hidden or range variables. Filtered correction spaces organize successive obstructions. Finite spectral-sequence pages record which classes survive each filtration depth.

OUTPUT

```
derivative data
  -> packet jets
  -> higher composition or rooted trees
  -> obstruction projection
  -> correction or elimination
  -> filtered lift
  -> finite spectral sequence
  -> conditional completion record
```

The final arrow is conditional whenever it refers to an infinite inverse limit. The runtime certifies finite compatibility but does not infer completeness or the existence of an infinite branch.

PART II

Operators and coordinates

Coordinate maps, packet decompositions and transfer operators describe finite data through compatible representations. Their reconstruction laws determine what can be moved, compared and recovered.

4 Coordinate systems and recurrence models

`komorebi_math.coordinates` is a general coordinate calculus for finite mathematical objects. It separates the object being represented from the coordinates used to represent it. A coordinate change may therefore connect arbitrary vector bases, sequence realizations, polynomial families, formal parameters, finite fields, state-space models, tensor components, or numerical spectral systems.

Euler numbers in Fibonacci companion coordinates remain available as one worked bridge. They do not define the scope of the namespace.

4.1 Coordinate systems and atlases

`FiniteBasis` represents any linearly independent finite basis in a rational ambient space. Square bases span the complete ambient space. Rectangular bases describe exact subspaces.

PYTHON | Executable example

```python
from komorebi_math.coordinates import CoordinateAtlas, basis_from_matrix,
    ↪ standard_basis

standard = standard_basis(3, name="standard")
shear = basis_from_matrix(
    "shear",
    (
        (1, 1, 0),
        (0, 1, 1),
        (0, 0, 1),
    ),
)

change = standard.connection_to(shear)
coordinates = change.apply((3, 5, 7))
assert change.inverse().apply(coordinates) == (3, 5, 7)

atlas = CoordinateAtlas("three dimensional atlas")
atlas.register_basis(standard)
atlas.register_basis(shear)
atlas.register_transform(change, include_inverse=True)
atlas.resolve("standard", "shear").transform.certificate().require()
```

An atlas stores named systems and directed transformations. It composes a shortest path, compares declared paths, and certifies commuting diagrams exactly.

4.2 Normalized generating-function coordinates

A normalized series basis has elements

$$e_n(z) = \frac{z^n}{\gamma_n}.$$

Changing from normalization γ to η is diagonal:

$$b_n = \frac{\eta_n}{\gamma_n} a_n.$$

The coordinate namespace reuses the normalized bases from `komorebi_math.generating`. Ordinary, exponential, factorial-power, Fibonacci-factorial, Pochhammer, and custom declared normalizations can therefore be connected without introducing a second basis abstraction.

4.3 Combinatorial coordinates

The namespace provides exact triangular changes among common sequence coordinates:

- monomial and falling-factorial coordinates
- monomial and rising-factorial coordinates
- signed and unsigned Stirling transforms
- Lah transforms
- binomial and inverse-binomial transforms
- sample values and Newton forward-difference coordinates
- divisor zeta and Möbius coordinates
- subset zeta and Möbius coordinates

These are ordinary coordinate transforms with explicit inverses. They can be inserted into an atlas or composed with any other compatible exact transform.

PYTHON | Executable example

```python
from komorebi_math.coordinates import forward_difference_transform

values = (0, 1, 4, 9, 16, 25)
newton = forward_difference_transform(len(values))
assert newton.apply(values) == (0, 1, 2, 0, 0, 0)
assert newton.inverse().apply(newton.apply(values)) == values
```

4.4 Polynomial-family coordinates

A `PolynomialFamily` supplies a graded polynomial basis through a declared finite degree. The namespace also provides fixed-degree Bernstein bases and interpolation bases through separate constructors. The available families and basis constructions include:

- monomials
- falling and rising factorials
- binomial polynomials
- Euler polynomials
- Bernoulli polynomials
- Fibonacci and Lucas polynomials
- Pell polynomials
- Chebyshev families of the first and second kind
- Legendre polynomials
- probabilists' and physicists' Hermite polynomials
- Laguerre polynomials
- Bernstein bases at a fixed degree
- Newton and Lagrange interpolation bases
- custom Appell families

PYTHON | Executable example

```python
from komorebi_math.coordinates import get_polynomial_family, connection_transform

legendre = get_polynomial_family("legendre")
chebyshev = get_polynomial_family("chebyshev-t")
connection = connection_transform(legendre, chebyshev, 8)
connection.certificate().require()
```

The same machinery compiles derivative, integral, multiplication, affine-substitution, and finite-difference operators in arbitrary polynomial coordinates. Operators can then be transported between families by matrix conjugation or rectangular transport.

4.5 Recurrence coordinates

A constant-coefficient recurrence of order d gives finite coordinates consisting of:

1. the first d values
2. the unique recurrence-defect stream

For

$$a_{n+d} = c_0 a_n + \cdots + c_{d-1} a_{n+d-1} + \delta_n,$$

the coordinate vector is

$$(a_0, \dots, a_{d-1}, \delta_0, \dots).$$

This representation is exact for every supplied prefix at least as long as the recurrence order, whether or not that prefix satisfies the homogeneous recurrence. It gives invertible finite changes between recurrence models of different orders when the prefix length is at least both orders.

Named constructors include Fibonacci, Lucas, Pell, Jacobsthal, Tribonacci, and Padovan models. `custom_recurrence_model` accepts any finite constant-coefficient rule.

PYTHON | Executable example

```python
from komorebi_math.coordinates import get_recurrence_model

pell = get_recurrence_model("pell")
tribonacci = get_recurrence_model("tribonacci")
change = pell.coordinate_transform_to(tribonacci, 20)
change.certificate().require()
```

Euler-to-Fibonacci coordinates are one specialization of this general defect calculus. The forcing stream is retained because Euler numbers are not a homogeneous Fibonacci trajectory.

4.6 Realization and state-space coordinates

The coordinate calculus supports finite linear realizations and input-output state-space models. It provides:

- companion realizations
- Green kernels and variation of constants
- observability matrices and exact state recovery
- controllability and observability coordinate systems
- cyclic Krylov coordinates
- intertwiner spaces solving $BS = SA$
- exact similarity transforms
- Markov-parameter invariance under state changes

For a state transform $z = Tx$, the system

$$x' = Ax + Bu, \qquad y = Cx + Du$$

becomes

$$z' = (TAT^{-1})z + (TB)u, \qquad y = (CT^{-1})z + Du.$$

The implementation certifies the transformed dynamics and input-output equivalence.

4.7 Affine, barycentric, and formal coordinates

`AffineCoordinateTransform` represents

$$y = Ax + b$$

with exact rational data. `BarycentricCoordinateSystem` converts points to and from the coordinates of a nondegenerate simplex.

`FormalCoordinateChange` represents an invertible finite jet

$$w = \phi(z) = a_1 z + a_2 z^2 + \cdots + a_N z^N, \qquad a_1 \neq 0.$$

It supports exact truncation, composition, compositional inversion, pullback of scalar series, pushforward, and substitution matrices.

4.8 Tensor and operator transport

The namespace distinguishes vectors, covectors, operators, bilinear forms, and higher tensor components. Under $y = Tx$:

- vectors transform by T
- covectors transform by $T^{-\mathsf{T}}$
- square operators transform by TAT^{-1}
- bilinear forms transform by $T^{-\mathsf{T}}GT^{-1}$
- mixed tensors transform according to each covariant or contravariant index

Direct sums and tensor products of coordinate changes are also available. Pairing and bilinear-form invariance can be checked exactly.

4.9 Finite-field coordinates

`FiniteFieldCoordinateTransform` performs exact linear coordinate changes over a prime field. Rational transforms can be reduced modulo a prime when every denominator is invertible. Inversion, composition, application, and certificates use modular Gauss-Jordan elimination.

4.10 Spectral and packet coordinates

The numerical layer provides finite transforms for:

- the discrete Fourier transform
- the type-II discrete cosine transform
- Walsh-Hadamard coordinates
- complete Haar wavelet coordinates
- modal coordinates of diagonalizable matrices

These transforms carry numerical inverse and conditioning certificates.

The packet layer provides exact permutation coordinates for:

- residue or polyphase ordering
- digit-reversal ordering
- Chinese-remainder ordering
- tensor-axis permutations
- arbitrary stable sorted orders

This connects the coordinate calculus directly to `komorebi_math.signals`, packet systems, arithmetic residue decompositions, and multidimensional layouts.

4.11 Persistence

The trusted persistence layer stores data-only records for:

- coordinate systems and atlases
- finite bases
- exact and numerical transforms
- affine and barycentric charts
- formal coordinate changes
- finite-field transforms
- recurrence and state-space models
- realization and recovery records
- polynomial coordinate operators
- coordinate budgets

Files use the package's versioned canonical envelope and SHA-256 payload integrity. Loading never imports arbitrary classes or reconstructs executable callbacks.

4.12 Command line

POWERSHELL

```
python -m komorebi_math coordinates catalog
python -m komorebi_math coordinates basis --source-matrix "1,0;0,1" --target-matrix
    ↪ "1,1;0,1" --vector "3,5"
python -m komorebi_math coordinates polynomial --source legendre --target
    ↪ chebyshev-t --degree 8
python -m komorebi_math coordinates model --source pell --target tribonacci --count
    ↪ 20
python -m komorebi_math coordinates combinatorial --kind forward-difference --size
    ↪ 8 --values "0,1,4,9,16,25,36,49"
python -m komorebi_math coordinates spectral --kind dft --size 8
python -m komorebi_math coordinates packet --kind residue --length 12 --modulus 3
python -m komorebi_math coordinates formal --coordinate "0,1,1/2,1/6" --series
    ↪ "1,2,3,4"
python -m komorebi_math coordinates euler-fibonacci --count 20
python -m komorebi_math verify coordinates
```

The Euler-to-Fibonacci command supplies one named example. The general API applies equally to the other coordinate models.

4.13 Companion Coordinate Examples

Companion coordinates form one part of the general coordinate atlas described in General Coordinate Atlas (§ 4).

A constant-coefficient recurrence of order d provides a companion state model. Any finite prefix at least as long as that order can be represented by its initial state and unique defect stream. This applies to arbitrary custom recurrences and to the named Fibonacci, Lucas, Pell, Jacobsthal, Tribonacci, and Padovan models.

The Euler-to-Fibonacci construction shows why defect coordinates matter. Signed secant Euler numbers do not satisfy the homogeneous Fibonacci recurrence. Their exact Fibonacci companion representation therefore consists of the initial state and the forcing sequence

$$\delta_n = \mathcal{E}_{n+2} - \mathcal{E}_{n+1} - \mathcal{E}_n.$$

For arbitrary bases, polynomial families, combinatorial transforms, affine and formal charts, finite fields, state-space systems, tensor transport, and spectral or packet coordinates, use the general atlas documentation.

5 Analysis and synthesis

5.1 Definition

Let

$$A \in \mathbb{F}^{m \times n}, \qquad S \in \mathbb{F}^{n \times m}.$$

`AnalysisSynthesisSystem` represents the finite left-inverse law

$$SA = I_n.$$

The square case $m = n$ is an invertible coordinate transform. The redundant case $m > n$ is supported when A has full column rank and S is a left inverse. General construction checks dimensions and finite entries. Call `certificate().require()` to enforce the left-inverse condition.

5.2 Construction

PYTHON | Executable example

```
import numpy as np
from komorebi_math.stable import AnalysisSynthesisSystem

square = AnalysisSynthesisSystem.from_invertible(
    np.array([[1.0, 1.0], [1.0, -1.0]]) / np.sqrt(2.0)
)

redundant = AnalysisSynthesisSystem.canonical_dual(
    np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
)

square.certificate().require()
redundant.certificate().require()
```

`canonical_dual()` computes the Moore-Penrose left inverse when the analysis matrix has full column rank at the declared numerical tolerance. Input that fails that rank test is rejected. The resulting certificate is numerical, even when the displayed matrices contain integers.

5.3 Components versus projectors

For a coordinate index set I, define

$$C_I = SE_IA,$$

where E_I is the diagonal coordinate selector. A complete coordinate partition satisfies

$$\sum_I C_I = I_n.$$

In the square invertible case, $AS = I_m$, so the components are mutually annihilating projectors. In a redundant frame, generally $AS \neq I_m$. The component law still reconstructs, but individual terms need not satisfy $C_I^2 = C_I$.

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.stable import AnalysisSynthesisSystem

redundant = AnalysisSynthesisSystem.canonical_dual(
    np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
)

components = redundant.components()
assert np.allclose(sum(components), np.eye(redundant.source_dimension))
```

Use `component()` and `components()` in new code. `projector()` and `projectors()` are compatibility aliases and do not independently certify idempotence for a redundant system.

5.4 Transfer blocks

Given an operator T on the source space,

$$T = \sum_{\beta,\alpha} C_\beta T C_\alpha.$$

PYTHON | Executable example

```
import numpy as np
from komorebi_math.stable import AnalysisSynthesisSystem

redundant = AnalysisSynthesisSystem.canonical_dual(
    np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
)

operator = np.array([[2.0, -1.0], [3.0, 4.0]])
blocks = redundant.operator_blocks(operator, groups=((0, 2), (1,)))
blocks.certificate().require()
assert np.allclose(blocks.reconstruct(), operator)
```

5.5 Persistence

PYTHON | Executable example

```
import numpy as np
from komorebi_math.stable import AnalysisSynthesisSystem

redundant = AnalysisSynthesisSystem.canonical_dual(
    np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
)

from komorebi_math.stable import dumps_portable, loads_portable

restored = loads_portable(dumps_portable(redundant))
restored.certificate().require()
```

The codec validates the left-inverse law when reading the record. A portable payload cannot silently reconstruct an invalid analysis–synthesis system.

5.6 An explicit redundant coordinate space

For an analysis map from two coordinates to three, take

$$T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad S = \frac{1}{3}\begin{pmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{pmatrix}.$$

Then $ST = I_2$, while TS is the projection onto the range of T. Synthesis reconstructs every object in the two-dimensional space. Coordinate roundtrip retains only triples consistent with the analysis relation.

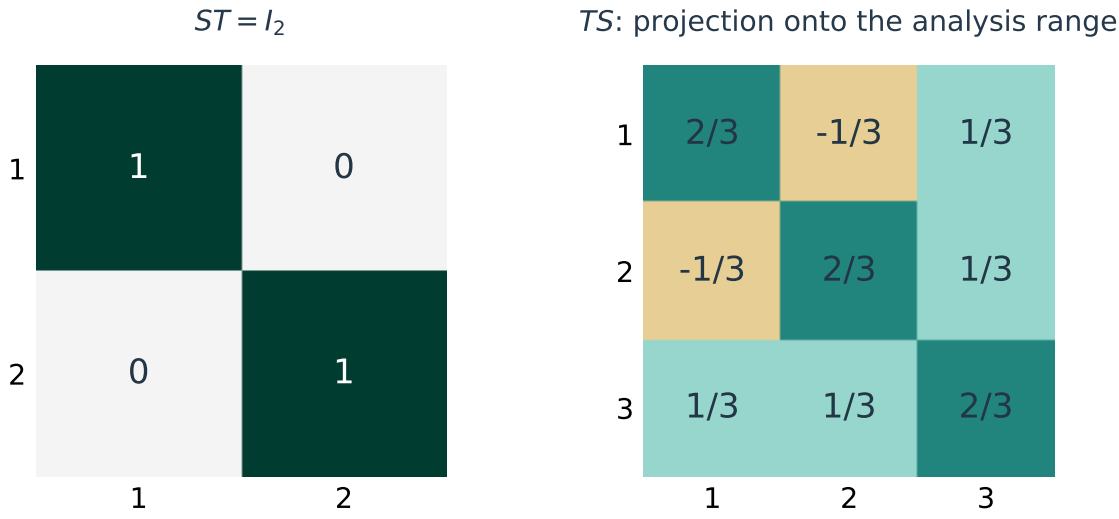


Figure 5.1: Object reconstruction is the identity. In the larger coordinate space, the roundtrip is an idempotent projection rather than the full identity.

6 Redundant frames

The redundant-frame algebra describes finite analysis and synthesis systems, including their coefficient-space representations. It does not assume that analysis coordinates form an orthonormal basis.

Let a finite presentation be a pair

$$A_X : X \longrightarrow K_X, \qquad S_X : K_X \longrightarrow X, \qquad S_X A_X = I_X.$$

The coefficient-space identity is the generally oblique projection

$$P_X = A_X S_X, \qquad P_X^2 = P_X.$$

Every coefficient vector has the canonical gauge decomposition

$$c = P_X c + (I - P_X)c, \qquad (I - P_X)c \in \ker S_X.$$

Two coefficient vectors represent the same physical vector exactly when their difference lies in $\ker S_X$. The positive semidefinite coefficient metric

$$H_X = S_X^* S_X$$

measures this quotient faithfully: $\langle c, H_X d\rangle = \langle S_X c, S_X d\rangle$, and gauge vectors have zero seminorm.

For another presentation (A_Y, S_Y) of the same physical space, the canonical bridge $R = A_Y S_X$ is an exact quotient isometry:

$$R^* H_Y R = H_X.$$

Moreover the reverse bridge composes with R to the corresponding oblique coefficient projection. Thus the bridges become genuine mutually inverse isometries after quotienting by the synthesis kernels, even though raw Euclidean coefficient norms can change drastically between presentations.

6.1 Maps and closure

For a physical operator $T : X \to Y$, its canonical coefficient lift is

$$\widehat{T} = A_Y T S_X.$$

It satisfies

$$P_Y \widehat{T} P_X = \widehat{T}, \qquad S_Y \widehat{T} A_X = T.$$

Consequently, physical maps are in bijection with the sandwiched coefficient matrices $P_Y \,\mathrm{Mat}(K_Y, K_X) P_X$. The inverse map is $L \mapsto S_Y L A_X$.

If two maps use the same middle presentation, their lifts compose by ordinary matrix multiplication. If the output presentation of the first map and the input presentation of the second map are different presentations of the same physical space, the bridge

$$R = A_{Y_{\text{in}}} S_{Y_{\text{out}}}$$

is required:

$$\widehat{U \circ T} = \widehat{U}\, R\, \widehat{T}.$$

The lifted physical identity is P_X, not the full identity on redundant coefficient space.

6.2 Coefficient blocks versus physical components

For a coefficient partition with selectors E_α, the small blocks

$$E_\beta \widehat{T} E_\alpha$$

assemble to $\widehat{T}$ and obey ordinary block-matrix multiplication when the middle partition is aligned. These are the compositional blocks in coefficient space.

The physical components

$$C_\beta T C_\alpha, \qquad C_\alpha = S E_\alpha A,$$

still reconstruct T, but in a redundant or oblique system the C_α need not be orthogonal projectors. A one-index multiplication rule for these components is therefore false in general. The verifier includes an explicit oblique counterexample so that the two block notions cannot be silently conflated.

6.3 Adjoints

The lift of the physical adjoint is

$$\widehat{T^*} = A_X T^* S_Y.$$

It is related to, but generally different from, the raw conjugate transpose of the coefficient lift:

$$\widehat{T}^* H_Y = H_X \widehat{T^*}.$$

Raw conjugate transpose becomes the correct lift after passing to the dual presentations (S^*, A^*). In Parseval presentations the distinction disappears.

6.4 Exact and numerical semantics

The numerical layer carries tolerance-aware certificates and immutable NumPy matrices. The exact layer uses immutable `Fraction` matrices and checks every identity by equality, with `Exactness.EXACT` certificates. Floating-point inputs are intentionally not treated as exact rational data.

This treatment is finite-dimensional. Infinite frame completion, measurable direct integrals and analytic convergence of an infinite expansion require additional hypotheses and constructions.

7 Metric packets

A metric packet distinguishes a bare quotient, a quotient equipped with geometric metadata, and a numerical compression of that metadata. This guide describes finite redundant frames and metric-decorated holonomy.

7.1 Redundant analysis–synthesis presentations

Let

$$A_i : X \to K_i, \qquad S_i : K_i \to X, \qquad S_i A_i = I_X.$$

The coefficient gauge is $\ker S_i$, and the quotient $K_i / \ker S_i$ represents the physical source space. The positive semidefinite coefficient metric

$$M_i = S_i^* S_i$$

is exactly the physical metric pulled back to coefficient space:

$$\langle c, M_i d \rangle = \langle S_i c, S_i d \rangle.$$

For two presentations of the same physical space, the canonical transition is

$$T_{ji} = A_j S_i.$$

It obeys

$$S_j T_{ji} = S_i, \qquad T_{ji}^* M_j T_{ji} = M_i.$$

Hence the induced quotient map is an exact isometry. If $P_i = A_i S_i$, then

$$T_{ij} T_{ji} = P_i, \qquad T_{ji} T_{ij} = P_j,$$

so the transitions become genuine inverses after quotienting the coefficient gauge directions. They also compose transitively:

$$T_{kj} T_{ji} = T_{ki}.$$

These identities require only the left-inverse laws $S_i A_i = I$. They do not require Parseval, orthogonal, or canonical-dual presentations.

7.2 Why Euclidean coefficient conditioning is representation dependent

If the quotient metric is discarded and raw Euclidean coefficient norm is used instead, the same physical vector can acquire arbitrarily different coefficient magnitudes under a nonunitary coordinate change.

For an invertible coefficient reparameterization G,

$$A' = GA, \qquad S' = SG^{-1}$$

still gives $S'A' = I$, while

$$M' = G^{-*}MG^{-1}.$$

Thus the metric-aware seminorm is unchanged,

$$\|Gc\|_{M'} = \|c\|_M,$$

while $\|Gc\|_2/\|c\|_2$ can be arbitrarily large or small. Scalar condition-number envelopes are therefore a compression of richer covariant metric information, not always the most fundamental packet.

7.3 Full-linear holonomy with transported metrics

Bare full-linear conjugacy data does not generally admit a continuous scalar that distinguishes every nontrivial holonomy from the identity. Metric metadata supplies a separate coordinate-covariant regime. If a positive-definite Hermitian root metric Q is part of the represented object and a full-GL coordinate change acts by

$$H' = gHg^{-1}, \qquad Q' = g^{-*}Qg^{-1},$$

then the metric Hilbert–Schmidt residual

$$\|H - I\|^2_{HS,Q} = \mathrm{tr}(Q^{-1}(H-I)^*Q(H-I))$$

is invariant in exact arithmetic:

$$\|H' - I\|_{HS,Q'} = \|H - I\|_{HS,Q}.$$

The invariant is a function of the pair (H, Q), not of the bare conjugacy class $[H]$. The semantics must therefore distinguish:

1. an active gauge symmetry with a fixed metric
2. a coordinate reparameterization that transports the metric

The bare `full_gl` numerical certificate fails closed. `metric_gluing_certificate` handles the transported-metric regime, and `transport_root_metric` supplies the required metric transformation.

7.4 Conditioning of the metric packet itself

Metric covariance does not eliminate floating-point conditioning. From

$$Q' = g^{-*} Q g^{-1}$$

one obtains the standard conservative bound

$$\kappa_2(Q') \leq \kappa_2(Q)\, \kappa_2(g)^2.$$

Therefore a coordinate change can make a previously well-resolved metric numerically singular. The implementation rejects positive-definite metrics whose smallest eigenvalue is not resolved relative to the declared graph tolerance.

This gives a hierarchy of represented information:

object $\to$ quotient $\to$ task quotient $\to$ covariant geometric metadata $\to$ conditioning envelope.

The fourth layer can remove coordinate distortion exactly. The fifth layer records when the fourth layer itself can no longer be represented reliably at finite precision.

7.5 Scope boundary

These contracts use finite-dimensional linear algebra and frame geometry: analysis and synthesis operators, dual frames, frame bounds, quotient spaces, congruence transport of metrics, and matrix condition numbers. Exact covariance requires the stated left-inverse identities and metric transport. Numerical certificates additionally require the metric to be resolved at the declared tolerance.

See redundant-frame algebra (§ 6), distortion cocycles (§ 8), and convex packets (§ 9) for related representations.

8 Distortion cocycles

The distortion API measures finite maps between spaces with positive metrics and records obstructions to a common metric completion. Its conventions build on metric packets (§ 7).

8.1 Intrinsic anisotropy packet

For an invertible finite-dimensional map

$$T : (V, Q_s) \longrightarrow (W, Q_t)$$

with positive-definite Hermitian metrics, choose factors $Q_s = R_s^* R_s$ and $Q_t = R_t^* R_t$. Define

$$B_Q(T) = R_t T R_s^{-1}$$

and

$$D_Q(T) = \kappa_2(B_Q(T)) = \frac{\sigma_{\max}(B_Q(T))}{\sigma_{\min}(B_Q(T))} \geq 1.$$

The singular values of `B_Q(T)` are independent of the chosen metric factors. If coordinates change by `G_s,G_t` and the metrics are transported covariantly, the new representative differs by unitary factors. Therefore `D_Q(T)` and the geometric-mean volume scale are coordinate invariants of the metric-decorated map.

Import the implementation explicitly from `komorebi_math.nonlinear.distortion`.

8.2 Distortion cocycle law

For composable metric maps,

$$D(T_2 T_1) \leq D(T_2) D(T_1),$$

hence

$$\delta(T) = \log D(T)$$

is subadditive:

$$\delta(T_2T_1) \le \delta(T_2) + \delta(T_1).$$

For a transport path, intermediate metric factors cancel exactly, so the same law applies edge by edge.

For a closed word with holonomy `H`, every metric satisfies

$$D_Q(H) \ge \frac{\max_i |\lambda_i(H)|}{\min_i |\lambda_i(H)|}.$$

Thus, with

$$\omega(H) = \log \frac{\max_i |\lambda_i(H)|}{\min_i |\lambda_i(H)|},$$

any edge distortion budget obeys

$$\sum_{e\in\gamma} \delta_e \ge \omega(H_\gamma),$$

where traversal multiplicity is counted without orientation cancellation. This is a necessary cycle/word obstruction.

8.3 Fundamental cycles alone are insufficient

Let

$$A = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad B_t = \begin{pmatrix} 0 & -t \\ 1/t & 0 \end{pmatrix}, \quad t > 1.$$

Both `A` and `B_t` have eigenvalues `+i, -i`; each is individually unitarizable and has spectral distortion lower bound one. But

$$AB_t = \mathrm{diag}(-1/t, -t),$$

so

$$\omega(AB_t) = 2\log t.$$

Consequently, for every common metric `Q`,

$$D_Q(A)D_Q(B_t) \geq t^2,$$

and therefore

$$\max(D_Q(A), D_Q(B_t)) \geq t.$$

The bound is attained by a metric with coordinate ratio `1/t`, so

$$\inf_Q \max(D_Q(A), D_Q(B_t)) = t.$$

For `t=9`, the exact optimum is `9`. A generating-cycle test can therefore miss an obstruction that appears only after word composition.

8.4 Conformal metric completion

A conformal metric completion of an invertible transport graph consists of positive metrics `Q_v` and positive edge length scales `lambda_e` satisfying

$$T_e^* Q_{t(e)} T_e = \lambda_e^2 Q_{s(e)}.$$

Rescale every vertex metric to determinant one and define

$$\widehat{T}_e = |\det T_e|^{-1/n} T_e.$$

Then the conformal equations are equivalent to

$$\widehat{T}_e^* Q_{t(e)} \widehat{T}_e = Q_{s(e)}.$$

After spanning-tree propagation this reduces to one root metric `Q` satisfying

$$\widehat{H}_\gamma^* Q \widehat{H}_\gamma = Q$$

for every closed word. Therefore:

> **Conformal metric-completion criterion.** A finite invertible transport graph admits a conformal positive-definite metric completion if and only if its determinant-normalized holonomy representation is unitarizable. Equivalently, its normalized holonomy image is conjugate into a unitary group; in finite dimensions this is equivalent to relative compactness of that image.

The equivalence with relative compactness uses the classical Haar-averaging construction for a compact group.

8.5 Metric-completion trichotomy

Normalize candidate root metrics by `det Q = 1`, and for a finite set of normalized holonomy generators define

$$F(Q) = \max_j \log D_Q(H_j), \qquad \Delta_* = \inf_{Q>0, \det Q=1} F(Q).$$

There are three operational cases.

1. **Completion:** `Delta_* = 0` and the infimum is attained. Then the normalized representation preserves a positive metric.
2. **Gap obstruction:** `Delta_* > 0`. Some anisotropy remains under every positive metric.
3. **Escape obstruction:** `Delta_* = 0` but the infimum is not attained. Every sequence `Q_k` with `F(Q_k) -> 0` must have `cond(Q_k) -> infinity` along a subsequence, and in fact no such near-minimizing sequence can have uniformly bounded condition number.

The third claim is a compactness theorem. If `det Q_k=1` and `cond(Q_k)` were uniformly bounded, all eigenvalues would be bounded above and away from zero. A convergent subsequence would then limit to a positive-definite `Q_*`; by continuity `F(Q_*)=0`, contradicting nonattainment.

This gives a precise role to conditioning: in the escape case, approaching metric completion forces the metric packet toward the boundary of the positive cone. A finite numerical trend illustrates this behavior but does not alone establish the limiting claim.

8.5.1 Unipotent model

For

$$H = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix},$$

all eigenvalues have modulus one, so the spectral lower bound is exactly one. There is nevertheless no positive-definite invariant metric. Taking

$$Q_s = \mathrm{diag}(s^2, 1)$$

gives

$$D_{Q_s}(H) \downarrow 1 \quad (s \downarrow 0), \qquad \kappa(Q_s) = s^{-2} \to \infty.$$

For sampled small positive `s`, numerical evaluation gives the diagnostic trend

$$(D_{Q_s}(H) - 1)\sqrt{\kappa(Q_s)} \to 1.$$

Thus the unipotent obstruction is a zero-gap/nonattainment obstruction rather than a positive spectral gap.

8.6 Curvature and metric compatibility

Metric compatibility is distinct from flatness. A connection can preserve a positive metric while having nonzero curvature.

On a trivial real rank-three bundle over a planar base, use the constant connection matrices

$$A_x = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad A_y = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Both are skew-symmetric, so the Euclidean metric is parallel. Yet

$$F_{xy} = [A_x, A_y] = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \end{pmatrix} \neq 0.$$

For a rectangular loop of side length `0.2`, a numerical evaluation gives nontrivial holonomy with Frobenius distance approximately `0.05637743157883952` from the identity and orthogonality residual `4.624480671433126e-17`.

So nonzero curvature and nontrivial holonomy are compatible with exact metric transport. For a fixed connection the relevant question is instead whether its holonomy preserves a positive metric. Curvature contributes to the holonomy algebra, but flatness is much stronger than metric compatibility.

8.7 Verification

The regression suite checks:

- coordinate covariance of metric distortion;
- submultiplicativity under composition;
- the exact two-generator mixed-word minimax example;
- the unipotent zero-gap/nonattainment model;
- determinant normalization preserving anisotropy.

Deterministic stress cases additionally compare coordinate covariance and composition bounds across varied finite matrices and positive metrics. These checks concern the represented numerical inputs and declared tolerances.

8.8 Scope

- The finite graph criterion uses classical unitarizability and compactness.
- Spectral word inequalities are necessary, not sufficient, for completion.
- Checking only fundamental holonomy generators is not sufficient; mixed words can expose obstructions.
- `Delta_* = 0` does not imply an invariant metric unless the infimum is attained.
- The API does not solve the global optimization problem for `Delta_*`.
- Nonlinear metric completion requires a holonomy-compatible parallel metric. A curvature value alone does not decide that existence question.

9 Convex packets

The geometry appropriate to a packet depends on its task. For exact invertible isometries, arbitrary Finsler norms do not enlarge the completion class. For one-sided semigroup control, they can be strictly stronger than quadratic metrics.

Import the implementation explicitly from `komorebi_math.nonlinear.convex_packets`. Related contracts are described in metric packets (§ 7) and distortion cocycles (§ 8).

9.1 Convex geometric packets

A finite-dimensional norm is equivalent to a centrally symmetric convex body `K` containing the origin in its interior. The unit ball is the geometric packet. For a linear map `A`,

$$\|A\|_K = \inf\{c > 0 : AK \subseteq cK\}.$$

One explicit family provided by the API is

$$\|x\|_{F,\infty} = \|Fx\|_\infty,$$

where `F` is invertible. Its unit ball is a parallelotope. Under a coordinate change `x'=Gx`, the packet transports as `F'=FG^{-1}`, so vector and operator norms are coordinate-covariant.

9.2 Exact invertible isometry: no Finsler enlargement

Let `Gamma <= GL(V)` preserve some norm exactly:

$$\|gx\| = \|x\| \qquad (g \in \Gamma).$$

Then every `g` is a linear isometry of a bounded convex body. Hence `Gamma` is bounded, its closure is compact, and averaging any Euclidean inner product over the compact closure gives a positive-definite invariant inner product. Thus:

> **Finite-dimensional isometry theorem.** An invertible linear group admits an invariant norm if and only if it admits an invariant positive-definite quadratic norm.

The same applies to exact conformal transport after removing the scalar factor from each generator. Therefore replacing inner products with arbitrary Finsler norms does not remove the holonomy metric-completion obstruction.

9.3 Forward semigroups are different

For a finite family `A={A_1,...,A_m}` and `r>0`, let `S_r` be the semigroup generated by `A_i/r`.

> **Product-bounded completion theorem.** There exists a norm with `||A_i|| <= r` for every generator if and only if `S_r` is product-bounded.

If the norm exists, all normalized products are nonexpansive and hence bounded. Conversely, if `S_r` is bounded in a reference Euclidean norm, define

$$\|x\|_* = \sup_{S \in S_r \cup \{I\}} \|Sx\|_2.$$

This is a finite norm and each `A_i/r` is nonexpansive in it.

The classical Rota-Strang variational formula gives

$$\rho(\mathcal{A}) = \inf_{\|\cdot\|} \max_i \|A_i\|,$$

where `rho` is the joint spectral radius. When this intrinsic growth rate is positive, normalization by it gives two cases for arbitrary norm packets:

1. **convex completion:** the normalized semigroup is product-bounded and an extremal norm is attained;
2. **convex escape:** the infimum is one but no extremal norm is attained.

The unipotent Jordan block is an escape example: its joint spectral radius is one, but its powers grow linearly, so no norm makes it nonexpansive. Diagonal coordinate packets can nevertheless make its one-step bound approach one while the packet condition number diverges.

For compact irreducible matrix families, Barabanov norms provide an important attainment mechanism. These facts are established joint-spectral-radius machinery.

9.4 Sharp convex-versus-quadratic separation

Consider

$$A_1 = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}, \qquad A_2 = \begin{pmatrix} 0 & 1 \\ 0 & -1 \end{pmatrix}.$$

In the infinity norm,

$$A_1(x,y) = (x,x), \qquad A_2(x,y) = (y,-y),$$

so

$$\|A_1\|_\infty = \|A_2\|_\infty = 1.$$

Thus the square is an exact forward-invariant convex packet and the joint spectral radius is one.

Now let

$$Q = \begin{pmatrix} a & c \\ c & b \end{pmatrix} > 0, \qquad d = ab - c^2 > 0.$$

Both maps are rank one. Their squared quadratic operator norms are

$$\|A_1\|_Q^2 = \frac{b(a+b+2c)}{d}, \qquad \|A_2\|_Q^2 = \frac{a(a+b-2c)}{d}.$$

If both were strictly less than 2, adding the resulting inequalities would force

$$(a-b-c)^2 + 3c^2 < 0,$$

which is impossible. Hence

$$\max(\|A_1\|_Q, \|A_2\|_Q) \geq \sqrt{2}$$

for every positive metric. `Q=I` attains equality, so

$$\boxed{\inf_{Q>0} \max_i \|A_i\|_Q = \sqrt{2}}$$

while the optimal arbitrary norm bound is exactly one.

This example is the standard Ando–Shih separation used in the joint spectral radius and Lyapunov literature. It shows that a quadratic gap need not be an obstruction in a richer geometry class.

9.5 John ellipsoid compression

For a symmetric convex body `K` in `R^n`, John's theorem gives an ellipsoid `E` with

$$E \subseteq K \subseteq \sqrt{n}\, E.$$

If `A K subseteq r K`, then

$$AE \subseteq AK \subseteq rK \subseteq r\sqrt{n}\, E,$$

so the associated quadratic norm satisfies

$$\|A\|_E \leq \sqrt{n}\, r.$$

Consequently the best ellipsoidal joint bound obeys

$$\rho(\mathcal{A}) \leq \rho_{\text{ell}}(\mathcal{A}) \leq \sqrt{n}\, \rho(\mathcal{A}).$$

This is the classical ellipsoidal approximation guarantee. The two-dimensional pair above attains the factor `sqrt(2)`, so the generic dimension bound is sharp already for the two-dimensional example.

For the implemented parallelotope packet `||Fx||_inf`, the John ellipsoid is explicit:

$$E_F = \{x : \|Fx\|_2 \leq 1\}, \qquad Q_F = F^T F.$$

The implementation therefore exposes both the exact convex operator norm and its compressed quadratic counterpart.

9.6 Geometry classes and obstruction hierarchy

A positive quadratic gap must be labeled by its admissible geometry class. It need not be an obstruction in a richer convex packet category.

For forward dynamics, a useful hierarchy is

quadratic packet $\subset$ parallelotope/polyhedral packet $\subset$ arbitrary symmetric convex packet.

The intrinsic growth obstruction is the joint spectral radius. Above that floor, failure to attain the bound is an escape or compactness issue. An extra gap in a restricted packet class is a **model-class obstruction**.

For invertible exact isometry/conformal holonomy, however, the zero-obstruction class does not enlarge: invariant Finsler structure can be averaged back to an invariant quadratic structure.

The choice of geometry therefore depends on the task. Quadratic metrics can be too restrictive for one-sided semigroup certification, while invariant quadratic and arbitrary norm structures give the same existence criterion for exact finite-dimensional group isometry.

9.7 Verification boundary

The regression suite checks:

- coordinate covariance of parallelotope packets;
- explicit John metric compression;
- exact infinity-norm nonexpansivity of the Ando–Shih pair;
- the sharp `sqrt(2)` quadratic lower bound under randomized positive-metric pressure tests;
- convex escape of the unipotent family.

Additional deterministic trials compare parallelotope bounds with their John-ellipsoid compression. These finite checks test the implementation over their sampled inputs. The mathematical statements use classical convex geometry, joint spectral radius, and switched-system theory.

PART III

Arithmetic, functions and signals

The same finite working language reaches recurrences, generating functions and signal decompositions. Each construction retains its own scalar conventions, admissible inputs and reconstruction conditions.

10 Sequences and local comparisons

The `komorebi_math.sequences` namespace generates exact sequence families and compares their finite windows with explicit local corpora. Its arithmetic conventions determine which coefficients are represented, while its comparison records distinguish a complete indexed match from agreement on a shorter overlap. These operations work offline and need no additional dependency.

10.1 Related APIs

Operation	Package implementation
Bernoulli numbers and signed Euler coefficients	`arithmetic.euler`, `sequences.catalog`
Fibonacci and Lucas recurrences and companion jumps	`arithmetic.recurrence`
Generic Hurwitz products and inversion	`arithmetic.hurwitz`
Ordinary and normalized formal series, reciprocals, powers	`generating.series`, `generating.finite_series`
Catalan, Bell, partitions, Apéry, and automatic sequences	`sequences.catalog`, `generating.sequences`
Residue channels, refinement, and polyphase reconstruction	`refinement.residue`, `refinement.polyphase`

`LinearRecurrence` orders companion states from oldest to newest and uses `x[n+d] = sum(c[i] * x[n+i])`. The convenience functions in this namespace use the same convention.

10.2 Exact families

PYTHON | Executable example

```python
from komorebi_math.sequences import (
    reciprocal_coefficients, reciprocal_sequence,
    universal_denominator_coefficients,
    kbonacci_sequence, kbonacci_closed_form,
    fubini_sequence, derangement_sequence,
)

assert reciprocal_coefficients(2, 6) == (1, 1, 5, 61, 1385, 50521)
assert kbonacci_sequence(3).first(7) == (0, 0, 1, 1, 2, 4, 7)
assert kbonacci_closed_form(3, 6) == 7
assert fubini_sequence().first(5) == (1, 1, 3, 13, 75)
assert derangement_sequence().first(6) == (1, 0, 1, 2, 9, 44)
```

For spacing `N` and nonnegative integer power `r`, the normalization is

$$D_N(t) = \sum_{q\geq 0} \frac{(-1)^q t^{Nq}}{(Nq)!}, \qquad D_N(t)^{-r} = \sum_{n\geq 0} \frac{A_N^{[r]}(n) t^{Nn}}{(Nn)!}.$$

`reciprocal_coefficients(N, count, power=r)` returns exactly `count` compressed coefficients starting at `n=0`. Power zero gives `(1,0,0,...)`. `universal_denominator_coefficients(N, degree)` returns dense EGF coefficients through the inclusive ordinary degree, including the zero residue lanes. It does not return ordinary Taylor coefficients. The compressed inversion recurrence uses Python integers throughout. The dense Hurwitz implementation provides an independent normalization control.

The k-bonacci family has `k-1` initial zeros and then one. Its finite binomial formula is tested against the recurrence and modular companion execution. Fubini numbers use ordered set partitions. Derangements start at `D_0=1`. All integer sizes, offsets, and terms reject booleans and fractional coercions.

10.3 Explicit local corpora

PYTHON | Executable example

```python
from komorebi_math.sequences import (
    OEISRecord, compare_oeis_prefix, find_local_oeis_matches, load_local_oeis,
)

reference = OEISRecord("A000045", (1, 1, 2, 3, 5, 8), offset=1)
match = compare_oeis_prefix((2, 3, 5), reference, query_offset=3, minimum_terms=3)
assert match.passed
assert match.index_alignment_verified

# An explicit file can be internal-format text, stripped text, or archive JSON.
# .gz files are supported. No download occurs.
# corpus = load_local_oeis("dated_snapshot.json", format="json")
# matches = find_local_oeis_matches(query_terms, corpus, query_offset=0)
```

`parse_oeis_internal` reads `%S/%T/%U` fields, preferring signed `%V/%W/%X` fields when supplied. Continuations are assembled in field order. `%N` provides the name and `%O` supplies the starting index. Missing offsets remain `None`. Lines that do not match the internal field syntax are ignored, including lines with malformed identifiers. Inspect the returned records before comparing them. `parse_oeis_stripped` reads the terms and optional names. The stripped format contains no index offset, so the loader preserves an unknown offset.

Without `query_offset`, matching compares the first stored positions and does not claim verified index alignment. With a declared query offset, the matcher aligns actual indices. A

reference with an unknown offset cannot validate an explicit indexed query. A query beginning before the available reference is reported as incomplete rather than silently dropping its leading terms.

`agrees_on_overlap` means every available compared term agrees and the minimum length is met. `complete_query_match`, and its `passed` alias, additionally mean that the entire supplied query was compared. If a reference ends early, the status is `reference-exhausted-prefix` and `passed` is false. Candidate search can retain that incomplete evidence, but excludes any known mismatch even if an earlier prefix met the minimum. Every result concerns a finite window. Neither a match nor a failed local search establishes infinite identity, originality, priority, or absence from the full OEIS.

10.4 Local data and licensing

No OEIS corpus or atlas is bundled with the library. Users load a local snapshot explicitly, and JSON provenance is retained on `LocalOEISCorpus.provenance`. The original software is licensed under MIT. OEIS data are attributed to the OEIS Foundation under CC BY-SA 4.0, separate from the software license. A software grant does not change the terms of separately supplied data.

11 Normalized generating functions

The generating-function namespace represents finite coefficient data in an explicit normalized basis. Products, reciprocals and basis changes retain the chosen coefficient domain. Local masks, carry compilers and Cartier operators then turn selected coefficient calculations into finite state transitions.

11.1 Normalized bases

A normalized monomial basis is determined by nonzero factors γ_n:

$$e_n(z) = \frac{z^n}{\gamma_n}, \qquad \gamma_0 = 1.$$

For

$$F(z) = \sum_n a_n e_n(z), \qquad G(z) = \sum_n b_n e_n(z),$$

the product coefficient is

$$(FG)_n = \sum_{k=0}^{n} \frac{\gamma_n}{\gamma_k \gamma_{n-k}} a_k b_{n-k}.$$

The package provides ordinary, exponential, factorial power, q-factorial, Fibonacci factorial, Pochhammer, finite table and callback-defined bases. Portable bases use closed `Basis Descriptor` records. A callback basis can be used directly in a finite computation. When a portable series is requested, the values of that basis are saved as a finite table.

PYTHON | Executable example

```
from komorebi_math.generating import (
    FiniteNormalizedSeries,
    RationalDomain,
    factorial_power_basis,
)

basis = factorial_power_basis(3)
left = FiniteNormalizedSeries(basis, RationalDomain(), (1, 2, -3, 5))
right = FiniteNormalizedSeries(basis, RationalDomain(), (2, -1, 4))
product = left * right
product.certificate().require()
```

11.2 Coefficient domains

`FiniteNormalizedSeries` uses an explicit exact coefficient domain:

- integers
- rational numbers
- residue rings $\mathbb{Z}/M\mathbb{Z}$
- prime fields $\mathbb{F}_p$
- truncated p-adic rings $\mathbb{Z}/p^r\mathbb{Z}$

The implementation checks invertibility before reciprocal, logarithmic, exponential, composition, or basis-transfer operations that require division. Same-basis divided-power products in characteristic p do not invert $n!$. Their binomial structure constants remain valid even when $n!$ itself vanishes modulo p.

11.3 Declarative coefficient sources

`CustomFunctionSpec` separates a coefficient source from its normalization. Constructors are provided for:

- explicit finite tables
- constant-coefficient recurrences
- rational generating functions
- hypergeometric term ratios
- implicit algebraic branches
- Euler products
- finite composition graphs
- factorial-power Bell families
- weighted factorial-power Bell families

PYTHON | Executable example

```python
from komorebi_math.generating import (
    FunctionRegistry,
    algebraic_function,
    rational_function,
)

registry = FunctionRegistry()
registry.register(
    rational_function("fibonacci", (0, 1), (1, -1, -1))
)
registry.register(
    algebraic_function(
        "catalan",
        {(0, 0): 1, (0, 1): -1, (1, 2): 1},
        1,
```

```
    )
)

assert registry.get("fibonacci").series(10).coefficients[10] == 55
```

Closed-form recognition routines perform bounded exact searches. A returned relation is verified on the supplied finite data. If no relation is returned, the conclusion concerns only the declared search region.

11.4 Local masks

For a radix $p \geq 2$, the local-mask transform is

$$\mathcal{M}_p[R](z) = \prod_{j \geq 0} R\left(z^{p^j}\right).$$

Every formal unit series has the unique local mask

$$R(z) = \frac{F(z)}{F(z^p)}.$$

The finite APIs provide exact truncated product, exact mask inversion, functional-equation certificates, analytic finite products with conservative tail bounds, Frobenius defect masks, first lifts modulo p^2, bounded polynomial coboundary solving, and Euler or cyclotomic orbit collapse.

11.5 Finite mask carry compiler

For a finite mask $R(z) = \sum_t r_t z^t$, the coefficient of $\mathcal{M}_p[R]$ has an exact finite carry representation. The transition matrices are

$$T_d(q, q') = r_{d+pq'-q}.$$

PYTHON | Executable example

```
from komorebi_math.generating import compile_finite_mask_carry_plan

plan = compile_finite_mask_carry_plan((1, 1, 1), 2)
plan.certificate(500).require()
coefficients = plan.sequence(500)
```

The overfull binary mask $1+z+z^2$ produces Stern's diatomic sequence shifted by one index. Ordinary Cauchy products of digit-local sequences are compiled by `compile_digit_cauchy_product`, which retains the required addition carry state.

11.6 Rational Cartier compiler

If

$$B(z)F(z) = A(z)F(z^p)$$

over $\mathbb{F}_p$, the package builds a bounded-dimensional Cartier linear representation and, when requested, its reachable minimized output automaton.

PYTHON | Executable example

```
from komorebi_math.generating import compile_cartier_linear_representation

representation = compile_cartier_linear_representation(
    numerator=(1, 1),
    denominator=(1, -1),
    prime=5,
)
value = representation.evaluate(10**12)
```

Evaluation reads the base-p digits of the index from least to most significant. After compilation, the number of matrix transitions therefore grows logarithmically with the index.

11.7 Bell congruence compilers

The factorial-power Bell family is implemented with exact integer polynomial recurrences. Under the theorem hypotheses, the global polynomial modulo p reduces to the ordinary weighted Bell polynomial indexed by the base-p digit sum.

The package provides:

- pointed-subindex certificates
- weighted and unweighted digit-sum reductions
- the exact binary population-count parity automaton
- Bell period bounds
- finite Bell residue automata

11.8 Prime power arithmetic

`PrimePowerUnitCompiler` computes the p-free factorial unit and exact binomial residues modulo p^r. `PrimePowerKernelCompiler` adds factorial powers and optional unit twists.

Kummer layer matrices count subindices by borrow number. This gives exact support for $\binom{n}{k}^s$ modulo p^r, including zero-borrow and one-borrow formulas and carry-layer convolution records.

Gaussian binomial valuations use the all-prime mixed-radix decomposition. The two-adic branch uses radix two for odd q. Support modulo prime powers follows directly from the valuation.

11.9 Persistence

Portable generating-function records contain data only. They use the package persistence envelope, explicit codec versions, canonical JSON and SHA-256 payload integrity. Loading a record does not execute an arbitrary import path or class name supplied by that record.

Supported records include normalized bases, budgets, formal series, domain-aware finite series, frozen custom functions, finite mask plans, Cartier automata and linear representations, Kummer distributions, and carry-layer convolution results.

11.10 Signal bridge

A finite ordinary series can be interpreted as an FIR transfer polynomial under an explicit $z \mapsto z^{-1}$ convention. `series_to_fir_adapter` records the convention, taps, response, and round-trip certificate. The bridge does not identify every formal series with a stable causal filter and does not hide truncation or numerical conversion.

11.11 Resource contracts

Operations that accept `GeneratingBudget` check operation-specific limits on degree, coefficient size, matrix dimension, reachable states, local combinations, and modulus size. Exceeding an enforced limit raises `ResourceLimitError`. Checks may require preliminary input conversion or arithmetic, so the budget is not an overall memory or elapsed-time bound.

The `maximum_serialized_bytes` and `maximum_recursion_depth` fields are retained in the budget record but are not currently enforced by generating-function operations. Portable persistence has its own explicit `max_depth` parameter. Bound serialized input and output sizes separately when needed.

12 Special functions

The `komorebi_math.special_functions` namespace brings together numerical Gamma shape transforms, exact rational residue packets and terminating scalar formulas. Importing it does not load mpmath, SciPy or SymPy. Numerical evaluation needs the `scientific` extra. The exact rational functions need no additional dependency.

PYTHON | Executable example

```python
from fractions import Fraction
from komorebi_math.special_functions import (
    bernoulli_packet_integer,
    gamma_sech_ambiguity,
    beta_binomial_refinement,
    meixner_pollaczek,
    q_racah_scalar,
)

# Exact rational Bernoulli polynomial packet, with B_1(x)=x-1/2.
packet = bernoulli_packet_integer(4, Fraction(3, 7), 3, 1)

# Unit log-Gamma windows with two positive shapes.
overlap = gamma_sech_ambiguity("1.25", "2.75", ".8", ".4", dps=60)

# p_(n,alpha+gamma)(x+y) expands into these normalized child amplitudes.
coefficients = beta_binomial_refinement(4, "1.25", "2.75", dps=60)
polynomial = meixner_pollaczek(4, 4, ".6", dps=60)

# A terminating scalar value with explicit exact rational parameters.
scalar = q_racah_scalar(2, 3, "2/7", "3/5", "4/9", "5/8", "2/3")
```

12.1 Exact rational functions

`bernoulli_packet_integer(n,a,M,r)` evaluates `M**(n-1)*B_n((a+r)/M)` with rational arithmetic. It accepts any rational translation, including zero and negative values. It uses the library's Bernoulli-number implementation. Summing the `M` residues reconstructs `B_n(a)` ; parent packets also reconstruct from their child residues. Degree and residue must be nonnegative integers, `M>=1`, and `r<M`.

`q_pochhammer`, `q_racah_lattice`, `q_racah_scalar`, and `racah_scalar` accept integers, `Fraction` values, and exact rational strings. They reject floating inputs. The q functions require `0<q<1`. The scalar sums retain terms through `min(n,y)` and reject a zero denominator in every retained term, before any cancellation. No value is assigned to a singular retained term.

These functions evaluate scalar formulas. They do not assert positive orthogonality or identify those formulas with the finite matrix recoupling construction. An exact counterexample illustrates the distinction: for `q=1/2`, `alpha=beta=1/2`, `gamma=4`, `delta=1`, and `n,y` in `{0,1}`,

both lattice sites equal 3 and every table entry equals 1. The termination condition `gamma*q=q**(-1)` therefore does not establish a positive orthogonal family.

12.2 Numerical families and conventions

Numerical functions return mpmath scalar values. Each call creates an isolated precision context. The `dps` parameter is the requested decimal precision and must be at least 15. The process-wide mpmath precision is unchanged. Shapes must be positive finite real values. Real arguments must be finite, and booleans are excluded from all scalar and integer domains.

Shape functions use extra isolated working digits to resolve cancellation at large parameters, then round the returned result to the requested precision. More than 10,000 working decimal digits, or excessive positive window-exponent work, raises `ResourceLimitError`. This work limit is not a uniform bound on elapsed time, and extra precision does not make the output a directed enclosure.

Functions	Object and convention
`log_gamma_window`, `log_gamma_fourier`	Unit window `g_s(y)=Gamma(s)**(-1/2)*exp(s*y/2-exp(y)/2)`; unitary Fourier transform with `exp(-i*omega*y)`
`gamma_sech_ambiguity`, `shape_affinity`	Centered overlap `integral g_alpha(y+h/2)*g_gamma(y-h/2)*exp(i*eta*y) dy`; zero-shift affinity
`meixner_density`, `meixner_characteristic`	Fourier energy density and characteristic function `sech(h/2)**s`
`beta_logit_density`	Density of `log(U/(1-U))` for `U~Beta(alpha, gamma)`
`meixner_pollaczek`	Normalized polynomial with generating function `(1-i*z)**(-s/2+i*x)*(1+i*z)**(-s/2-i*x)` before normalization
`beta_binomial_refinement`, `shape_refinement_coefficient`	Positive two-child and many-child refinement amplitudes; degree vectors and shape vectors must have the same positive length
`shape_transfer_singular_value`	`sqrt((s)_n/(s+t)_n)` for `s>0` and increment `t>0`
`bernoulli_hurwitz`, `bernoulli_hurwitz_packet`	Entire-order `-z*zeta(1-z,a)` and residue refinement for real `a>0`; exact removable values at `z=0`

The Bernoulli evaluator retains tiny nonzero orders instead of substituting the zero-order value. It temporarily increases its isolated working precision to resolve the Hurwitz pole cancellation, with a 10,000-digit working cap. The integer-order polynomial function has a broader translation domain than the complex-order Hurwitz function.

These are numerical evaluations, not interval enclosures. `dps` alone does not certify an error bound. Integration tests compare independent quadrature, generating-function coefficients, exact residue reconstruction, polynomial addition, and refinement-tree composition in a stated finite range. They do not certify arithmetic depth asymptotics, unresolved signed estimates, or all proofs in the source manuscripts.

12.3 Scope and related APIs

The `coordinates`, `analytic`, `generating`, and `sequences` namespaces provide residue packets, finite analysis and synthesis, normalized generating functions, and classical polynomials. This namespace supplies the Gamma/Meixner shape family, Bernoulli–Hurwitz order interpolation, and a finite scalar q-Racah bridge. Infinite arithmetic transfer estimates, a full representation-theoretic matrix recoupling engine, and a continuous-series spectral transform are outside its implemented contract.

Exact counterexamples and independent numerical comparisons support the stated conventions. Bounded numerical tests do not establish universal identities or replace mathematical proofs.

13 Positive quadratic-form tails

The optional `komorebi_math.quadratic_forms` namespace evaluates positive quadratic-form tails using the directed backend of the Komorebi Quadratic-Form Engine, K-COS. The represented random variable is

$$Q = \sum_{j=1}^{r} \lambda_j Z_j^2, \qquad Z_j \sim N(0,1)$$

with independent standard Gaussian variables and strictly positive coefficients. The COS remainder bound requires `r > 2`. Automatic planning requires the threshold to exceed the exact sum of the supplied coefficients. This is a research API with explicit numerical and mathematical assumptions. Original project software is licensed under MIT.

From a source checkout, install the scientific extra and qualified backend:

SHELL

```
python -m pip install ".[scientific]" "mpmath==1.3.0"
```

Directed operations require `mpmath==1.3.0`, the qualified arithmetic implementation used by the source audit. Importing the package or this namespace does not import mpmath. A missing or unqualified arithmetic backend raises a `BackendUnavailableError` with an installation instruction when an operation is requested.

PYTHON | Executable example

```
from komorebi_math.quadratic_forms import certified_positive_logsf

tail = certified_positive_logsf(
    "30",
    ("0.25" for _ in range(32)),
    relative_accuracy="1e-9",
    initial_K=64,
    max_K=512,
    initial_L="10",
    max_L=14,
    initial_interval_dps=40,
    max_interval_dps=80,
    working_dps=120,
)
assert tail.certified
print(tail.log10_interval)
```

Inputs are interpreted through their decimal literals. Use strings or `Decimal` values when exact decimal intent matters. A float input contributes its `str(value)` decimal, not its exact

binary fraction. Iterables are materialized once, so generators work through every refinement.

`certified=True` means the returned literal endpoints prove `p_upper / p_lower - 1 <= relative_accuracy`, conditional on the arithmetic trust base and stated mathematical hypotheses. Exhausted valid resource caps return `certified=False` with a failure reason. Invalid domains raise an error. Caps constrain each attempted transform and precision, rather than elapsed time or total work. Low-rank or poorly scaled spectra can require many modes.

The authoritative enclosure is the outward decimal string `log10_interval`. Float fields such as `log10_lower`, `log10_upper`, and `width_decades`, and point midpoints, are display summaries. They can round inward or collapse a narrow interval. Certificate decisions use the literal endpoints with directed arithmetic. `directed_relative_accuracy` supplies the same width decision for a separately supplied outward bracketed literal string. Interval objects are rejected because ordinary string conversion can silently collapse their width at the ambient precision. The caller is responsible for supplying valid outward endpoints of the intended interval.

Operation	Supported purpose
`certified_positive_logsf`	Automatically refine an above-mean positive central tail and prove a requested multiplicative width
`design_positive_tail_parameters`	Choose admissible positive tilt/support design points for an above-mean query
`interval_positive_logsf`	Evaluate a specified finite tilted COS transform with support/alias and omitted-mode error bounds
`even_df_scaled_chi_square_logsf_interval`	Independent finite Erlang-tail control for a nonnegative threshold and positive even degrees of freedom
`directed_relative_accuracy`	Decide a multiplicative width from outward literal log10 endpoints
`certify_float64_factor_spectrum`	Enclose the eigenvalues of a stored binary64 Gram factor using directed residual checks
`certified_float64_factor_logsf`	Combine spectrum and tail enclosures with a whole-interval width decision

The explicitly parameterized evaluator also checks positive tilt, a support endpoint above the threshold, and a second tilt strictly between the first tilt and every MGF singularity. Its `passed_positive_sum` field reports whether the analytic and arithmetic error interval proves a positive lower bound for the tail sum. That field does not assert that a requested relative tolerance was achieved.

13.1 Stored-factor interpretation

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.quadratic_forms import certify_float64_factor_spectrum

spectrum = certify_float64_factor_spectrum(
    np.diag([1.0, 1.5, 2.0, 2.5]),
    candidate_digits=40,
    interval_dps=50,
)
print(spectrum.lower, spectrum.upper)
```

The bridge converts a real input matrix to binary64 and treats those stored entries as exact dyadic numbers. It certifies the sorted eigenvalues of `A @ A.T`. The eigensolver supplies a candidate only. Directed orthogonality and reconstruction residuals establish the coefficient intervals. The unknown matrix before conversion and any uncertainty in input data are outside the contract. Complex and nonfinite matrices are rejected.

This implementation uses a dense full basis and is intended for correctness on modest matrices. `VerifiedSpectrum.rank` is the number of coefficient intervals, including zero eigenvalues, rather than the algebraic matrix rank. The automatic matrix-tail bridge requires a threshold above the exact stored trace and at least three strictly positive lower coefficient bounds. Unresolved spectral rank and invalid planner domains can raise an informative error. When valid input domains cannot be resolved at the supplied working or interval precision, the automatic controller returns `certified=False` with its failure reason. Conservative comparison-spectrum obstructions trigger bounded candidate refinement. Reported candidate precision is the actual last attempt, including on exhaustion. The public options preserve the supplied caps.

Public operations hold a reentrant lock for their entire backend computation and restore the caller's point and interval precision in a `finally` block. Concurrent calls through this namespace are serialized. Arbitrary concurrent manipulation of mpmath global contexts by external code is outside this guarantee. Private implementation modules are not the supported entry points.

13.2 Arithmetic trust and scope

The enclosure claim assumes correctness of mpmath 1.3.0 interval arithmetic and transcendental evaluations, its exact binary endpoint tuples, Python integer arithmetic, and directed Decimal serialization. Finite-transform rounding is enclosed, and analytic support/alias and COS remainder bounds are added explicitly. This numerical claim is distinct from a Lean proof of the Python engine. The companion formalization covers finite algebra and arithmetic consequences of supplied bounds. It does not establish Gaussian laws, COS inversion, or correctness of the numerical libraries.

Public entry points validate inputs and serialize access to backend precision contexts. Spectrum iterables are consumed once, and adaptive support-scale refinements respect the supplied cap. A display-only ratio may be infinite when it overflows, without changing the enclosed tail bound. Private backend modules are implementation details.

Signed and noncentral quadratic-form engines, scalable streaming engines, and unconstrained root-finding helpers are outside this public surface. The project license covers its original software. Dependencies retain their own terms.

Validation includes directed-contract regressions, exact finite-gamma controls for five equal-coefficient spectra, ordinary-tail SciPy comparisons, two known/higher-precision stored-factor controls, a complete matrix-tail control, malformed-input checks, and success/failure/concurrency precision checks. These bounded tests establish behavior on their named cases and do not substitute for the stated arithmetic trust base.

14 Finite join quotients

The `komorebi_math.core.semilattice` namespace constructs finite join semilattices from explicit operation tables. It computes the least congruence containing supplied relations and produces quotient certificates whose derivations can be replayed independently.

PYTHON | Executable example

```python
from komorebi_math.core.semilattice import FiniteJoinSemilattice

# Subsets of a two-element set, labeled by bit masks.
source = FiniteJoinSemilattice(
    tuple(tuple(left | right for right in range(4)) for left in range(4))
)
result = source.generated_congruence(((0, 1),))
result.certificate().require()
assert result.classes == ((0, 1), (2, 3))
assert result.projection == (0, 0, 1, 1)
assert result.quotient.join_table == ((0, 1), (1, 1))
```

`FiniteJoinSemilattice` requires a nonempty square table whose entries are strict integers in the source label range. It checks idempotence, commutativity and associativity. Its storage is an immutable tuple snapshot. Python and NumPy integer labels are accepted. Booleans, floating labels, truncation, ragged tables and invalid entries are rejected.

`generated_congruence(relations)` returns the least equivalence relation closed under joining with every source element and containing the requested pairs. Each successful merge records a finite derivation: an input pair, or a join context of an earlier merge. `Semilattice Quotient.certificate()` independently replays those derivations and checks the quotient homomorphism and requested relations. Every derived merge belongs to any congruence containing the input relations. Conversely, the verified output is itself such a congruence. Together these finite checks establish leastness for this supplied finite table.

`quotient(partition)` validates a supplied equivalence partition, allowing arbitrary integer class labels, and constructs its quotient. It does not claim that a manually supplied partition is least for any set of relations.

The output contains canonical class minima in `partition`, consecutive quotient labels in `projection`, original members in `classes`, and the validated quotient table in `quotient`. The certificate has exact status because table validation, union-find and witness replay use Python integer comparisons. Residual floats are only the zero or one display of that exact decision.

For source size `n`, table storage is `O(n^2)` and axiom validation is `O(n^3)`. With `m` supplied relations, congruence closure performs at most `n-1` successful merges and queues `O(m+n^2)`

pairs. Union-find operations have nearly constant amortized cost. Constructing and validating the quotient retains a cubic worst-case table-validation cost. The API is suitable for finite tables whose storage and validation costs are acceptable.

The regression suite compares all pairs of relations on two different size-four semilattices against an independent enumeration of every equivalence partition. It also checks relation order independence, join homomorphisms, a size-eight Boolean context closure, forged witnesses, integer validation and caller mutation. These computations do not certify a universal matroid colimit, nonadditive packet reconstruction theorem or doctrine closure statement.

15 Signal processing and recovery

The signal-processing namespace combines polyphase decomposition, filter banks, wavelet packets, singular packets, and jump recovery with a general finite digital signal-processing layer. The implementation is available from `komorebi_math.signals` and depends only on NumPy.

Every routine operates on an explicitly supplied finite array or a finite ordered stream of chunks. Certificates check algebraic identities, reconstruction residuals, declared parameter ranges and agreement between batch and streaming execution. Statistical assumptions, application models and execution deadlines require evidence beyond these finite checks.

15.1 Scope

The digital signal-processing layer contains ten connected components:

1. deterministic analysis windows and overlap diagnostics
2. finite framing, STFT analysis, inverse synthesis, and spectrograms
3. periodograms, Welch estimates, cross spectra, coherence, and transfer estimates
4. linear-phase FIR design, direct filtering, and causal streaming execution
5. integer upsampling and downsampling with rational polyphase resampling
6. finite LMS, normalized LMS, and recursive least-squares adaptation
7. multilevel Haar decomposition and coefficient shrinkage
8. chunk-invariant streaming STFT analysis
9. finite FIR/IIR recursion, continuation state, and biquad design
10. direct FFT and stateful overlap-add convolution

These components work alongside the packet, polyphase, lifting, singular-recovery and nonlinear signal APIs.

15.2 Windows and finite overlap geometry

`get_window` constructs boxcar, Hann, Hamming, Blackman, Bartlett, Kaiser, and Tukey windows under explicit periodic or symmetric conventions. An array supplied by the caller is accepted as a custom window only when its length and entries are valid.

PYTHON | Executable example

```
from komorebi_math.signals import (
    canonical_dual_window,
    get_window,
    overlap_add_certificate,
    window_metrics,
)

window = get_window("hann", 128, periodic=True)
metrics = window_metrics(window, sample_rate=1024.0)
assert metrics.coherent_gain == 0.5

overlap_add_certificate(
    window,
    32,
    squared=True,
    require_constant=True,
).require()

dual = canonical_dual_window(window, 32).require()
```

For a window `w` of length `L` and hop `H`, `overlap_profile` computes the periodic residue-class profile

$$P_r = \sum_{n \equiv r \pmod{H}} w_n$$

or its squared counterpart. `overlap_add_certificate` can require positivity only or constant overlap. `canonical_dual_window` uses the finite residue-class denominator to construct a synthesis window whose cross-overlap profile is one whenever that denominator is nonzero.

The dual-window certificate is a finite identity. It does not assert a frame theorem for an unspecified infinite signal space.

15.3 Framing, STFT, and inverse synthesis

`frame_signal` defines one uniform finite framing convention. The last frame is included only when complete unless `pad_end=True`. Centering and boundary padding are explicit metadata.

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals import stft

rng = np.random.default_rng(12001)
signal = rng.normal(size=1001)

transform = stft(
    signal,
    sample_rate=8000.0,
    frame_length=127,
    hop_length=31,
    fft_length=256,
    window="hann",
    center=True,
    pad_end=True,
)

reconstructed = transform.inverse().require()
transform.roundtrip_certificate(signal).require()
assert np.allclose(reconstructed, signal, atol=2e-15)
```

The STFT record stores all metadata required by normalized overlap-add synthesis. For analysis window `w`, synthesis window `v`, frame index `m`, and hop `H`, reconstruction divides by the represented finite denominator

$$D[n] = \sum_m w[n - mH]v[n - mH].$$

A reconstruction certificate fails when this denominator vanishes on a required sample. Constant framewise detrending is deliberately marked noninvertible because the removed means are not retained.

Real input supports one-sided Fourier coordinates. Complex input requires a two-sided transform. Complex analysis windows also require two-sided coordinates. Inverse and roundtrip certificates check finite coefficients and synthesis output in addition to overlap coverage. A numerical overflow cannot pass a certificate merely because both an error and its reference scale become infinite.

15.4 Spectrograms and spectral estimates

`spectrogram` derives magnitude, power, spectrum, or density values from the same finite STFT record. `periodogram` and `welch_psd` expose finite power-spectrum records with frequency coordinates, scaling metadata, integration helpers, and nonnegativity certificates.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.signals import periodogram, welch_psd

sample_rate = 1024.0
index = np.arange(1024)
signal = 2.0 * np.sin(2.0 * np.pi * 91.0 * index / sample_rate)

line = periodogram(
    signal,
    sample_rate=sample_rate,
    window="boxcar",
    detrend="none",
)
assert line.dominant_frequency() == 91.0
line.certificate.require()

estimate = welch_psd(
    signal,
    sample_rate=sample_rate,
    frame_length=256,
    hop_length=128,
    average="mean",
)
estimate.certificate.require()
```

For a deterministic relation `y = g x`, `cross_spectrum` reports input and output power, cross power, magnitude-squared coherence, and the regularized H1 estimate

$$\widehat{H}_1(f) = \frac{S_{yx}(f)}{S_{xx}(f) + \varepsilon}.$$

PYTHON | Executable example

```
import numpy as np

time = np.arange(1024) / 1000.0
signal = np.sin(2 * np.pi * 60.0 * time)
from komorebi_math.signals import cross_spectrum

report = cross_spectrum(
    signal,
    -2.25 * signal,
```

```
    frame_length=256,
    hop_length=128,
    detrend="none",
)
report.certificate.require()
```

Coherence is clipped to its finite numerical range. A high observed coherence is a property of the supplied record and estimator. It is not a causal-identification theorem.

`parseval_certificate` separately checks the finite DFT energy identity for real or complex records. Coherence normalization avoids multiplying the two auto spectra before normalization, preserving the supplied-record coherence when those finite spectra are small enough that their product would underflow.

15.5 FIR design and filtering

`design_fir` implements odd-length real linear-phase windowed-sinc designs. Convenience constructors are available for lowpass, highpass, bandpass, and bandstop filters.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.signals import design_lowpass, fir_filter

lowpass = design_lowpass(
    100.0,
    num_taps=101,
    sample_rate=1000.0,
    window=("kaiser", 8.0),
)
lowpass.certificate(tolerance=1e-9).require()

filtered = fir_filter(np.arange(1000.0), lowpass, mode="causal")
response = lowpass.response(points=1001)
assert lowpass.group_delay_samples == 50.0
```

The finite modes are:

- `full`: complete finite convolution
- `causal`: first `len(signal)` samples of complete convolution
- `same`: centered signal-length slice
- `valid`: samples with complete kernel overlap

`StreamingFIR` retains exactly `num_taps - 1` prior input samples. Its `process` method emits one causal output per accepted input sample. `flush` emits the remaining full-convolution tail once. FIR arithmetic promotes fixed-width integer data to a floating or complex computation type before products and sums. A complex history remains complex when a subsequent chunk contains only real samples.

PYTHON | Executable example

```
import numpy as np

time = np.arange(1024) / 1000.0
signal = np.sin(2 * np.pi * 60.0 * time)
from komorebi_math.signals import design_lowpass, stream_fir_filter

lowpass = design_lowpass(100.0, num_taps=101, sample_rate=1000.0)

result = stream_fir_filter(
    signal,
    lowpass,
    chunk_sizes=(113, 287, 19, 605),
)
result.certificate.require()
```

The stream certificate compares the chunked output with the direct causal finite convolution. It does not make a wall-clock latency claim.

15.6 Recursive FIR and IIR filtering

`LinearFilter` normalizes a finite rational transfer function and executes it in direct-form II transposed form. The returned `FilterOutput` contains both samples and the exact finite continuation state required by the next chunk. The computation type includes the supplied initial state. Real input chunks and real filter coefficients therefore preserve any imaginary continuation state from preceding complex samples.

PYTHON | Executable example

```
import numpy as np

time = np.arange(1024) / 1000.0
signal = np.sin(2 * np.pi * 60.0 * time)
from komorebi_math.signals import design_biquad, stream_lfilter

biquad = design_biquad("lowpass", 120.0, q=0.8, sample_rate=1000.0)
biquad.stability_certificate().require()
streamed = stream_lfilter(biquad, signal, (1, 17, 300, 706))
streamed.certificate.require()
```

The stability certificate checks that every numerically computed pole lies strictly inside the unit disk by the declared tolerance. This is a finite coefficient-level BIBO stability diagnostic. It does not certify robustness to coefficient quantization not represented in the supplied model.

15.7 FFT and overlap-add convolution

`fft_convolve` computes finite convolution through zero-padded FFT multiplication. `OverlapAddConvolver` processes bounded blocks, carries the exact overlap tail, and emits the remaining tail once at flush.

PYTHON | Executable example

```python
import numpy as np

time = np.arange(1024) / 1000.0
signal = np.sin(2 * np.pi * 60.0 * time)
from komorebi_math.signals import design_lowpass, certified_overlap_add_convolve

lowpass = design_lowpass(100.0, num_taps=101, sample_rate=1000.0)

result = certified_overlap_add_convolve(
    signal,
    lowpass.taps,
    block_length=256,
)
result.certificate.require()
```

The certificate compares overlap-add output with direct finite convolution under the same crop convention. It establishes numerical equality within tolerance, not an unconditional speedup.

15.8 Rational polyphase resampling

The resampling convention is explicit:

1. reduce `up/down` by their greatest common divisor
2. insert `up - 1` zeros between samples
3. apply the declared finite anti-imaging and anti-aliasing FIR
4. compensate the symmetric filter delay
5. retain every `down`-th sample
6. return exactly `ceil(N up / down)` samples

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals import resample_poly_result

signal = np.linspace(0.0, 1.0, 257)
result = resample_poly_result(signal, 5, 3)
result.certificate.require()
assert result.signal.size == 429
```

`RationalResamplingPlan.explicit_reference` evaluates the corresponding zero-insertion and direct-convolution construction. The certificate compares the polyphase implementation with that independent finite reference.

The default filter is a finite windowed-sinc design. It is not presented as a universally optimal resampling filter.

15.9 Adaptive filters

The adaptive layer provides finite LMS, normalized LMS, and RLS recursions. Each result stores output, error, final coefficients, complete coefficient history, and finite diagnostics.

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals import fir_filter, nlms_filter

rng = np.random.default_rng(1205)
source = rng.normal(size=3000)
target = np.array([0.7, -0.35, 0.15])
desired = fir_filter(source, target, mode="causal")

identified = nlms_filter(source, desired, 3, step_size=0.8)
identified.certificate.require()
assert np.allclose(identified.final_weights, target, atol=1e-8)
```

Complex signals use conjugate inner products and coefficient updates. Parameter-range validation and finite-recursion checks are explicit. Persistent excitation, stochastic independence, and convergence for a chosen step size or forgetting factor remain unchecked assumptions in the result certificate.

15.10 Multilevel Haar denoising

`haar_decompose` repeatedly applies the normalized finite Haar filter bank. A decomposition certificate verifies reconstruction and normalized energy conservation. Haar pair arithmetic and energy diagnostics promote fixed-width integer data before summation. Reconstruction and energy use separate relative scales, so the numerical certificate has the same

meaning for an otherwise identical scaled signal. `FiniteFilterBank.report` and `PeriodicFilterBank.frame_bounds` include unobserved input directions: an analysis operator with fewer rows than input coordinates has lower frame bound zero.

`wavelet_denoise` supports:

- hard thresholding
- soft thresholding
- nonnegative-garrote thresholding
- fixed thresholds
- universal thresholds
- finite SURE threshold selection
- common or level-dependent noise estimates

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals import denoising_risk_certificate, wavelet_denoise

rng = np.random.default_rng(1206)
clean = np.repeat(np.array([0.0, 2.0, -1.0, 1.5]), 128)
noisy = clean + rng.normal(scale=0.3, size=clean.size)

result = wavelet_denoise(
    noisy,
    levels=4,
    rule="universal",
    mode="soft",
)
result.certificate.require()
denoising_risk_certificate(clean, noisy, result.signal).require()
```

The shrinkage certificate verifies finite reconstruction and coefficient-magnitude monotonicity. The observed-risk certificate requires a supplied clean reference. Universal and SURE rules do not establish application-specific optimality without their noise-model assumptions.

The scalar and vector jump-recovery APIs reject nonfinite samples and nonfinite grid declarations. Vector jump contrasts promote integer data before subtraction, preventing unsigned downward jumps from wrapping to small positive contrasts. Their guarantees remain conditional on the declared smoothness, noise, jump-size, and separation hypotheses.

15.11 Streaming STFT

`StreamingSTFT` maintains only the overlap and unframed samples required by the next finite frame. `process` emits every complete new frame. `flush(pad_end=True)` emits at most one terminal padded frame, exactly matching `frame_signal`.

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals import stream_stft

rng = np.random.default_rng(1207)
signal = rng.normal(size=1000)

result = stream_stft(
    signal,
    chunk_sizes=(123, 456, 421),
    frame_length=128,
    hop_length=32,
    fft_length=256,
)
result.certificate.require()
```

The certificate compares coefficients and time coordinates against a batch STFT with identical parameters. Chunk boundaries do not alter the represented finite transform.

15.12 Verification

Run the installed digital signal-processing checks with:

POWERSHELL

```
komorebi verify dsp
```

The source tests cover numerical conventions, chunk invariance, finite reconstruction, and comparison with convention-matched reference calculations.

15.13 Explicit boundaries

The implemented scope includes the biquad families described above. General high-order IIR design, arbitrary multirate filter-bank compilation, continuous wavelet transforms, audio-file codecs, real-time acquisition, adaptive-filter convergence theorems, and a hardware-validated GPU DSP path are outside this API.

15.14 Fourier recovery of periodic jump packets

`komorebi_math.signals.fourier_recovery` provides finite jump reconstruction using NumPy. It combines high-mode Fourier moments, Hankel rank estimation, annihilating polynomials, and periodic integration.

The target is a real periodic signal `f = s + j`. Its smooth component `s` must have Fourier degree at most the supplied `background_degree`. Its jump component `j` has finitely many distinct active locations, real amplitudes that sum to zero, and mean zero. These are model assumptions. The library does not infer them from arbitrary data.

For a period `T`, use normalized Fourier coefficients

$$\hat{f}(n) = \frac{1}{T}\int_0^T f(x)e^{-2\pi inx/T}\,dx.$$

Above the declared background degree, the derivative moments satisfy

$$m_n = 2\pi in\hat{f}(n) = \sum_{\ell=1}^{L} a_\ell z_\ell^n, \qquad z_\ell = e^{-2\pi i\tau_\ell/T}.$$

The factor `2π i n` holds for every period under this normalization. A different factor requires a corresponding change in the Fourier convention.

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals.fourier_recovery import recover_fourier_jumps

period = 2 * np.pi
locations = np.array([0.7, 2.2, 4.8])
amplitudes = np.array([2.0, -0.75, -1.25])
background_degree = 3
maximum_jumps = 5

# These modes lie above the background's bandlimit.
measurements = {
    n: np.sum(amplitudes * np.exp(-1j * n * locations)) / (2j * np.pi * n)
    for n in range(background_degree + 1,
                   background_degree + 1 + 2 * maximum_jumps)
}
result = recover_fourier_jumps(
    measurements,
    background_degree=background_degree,
    maximum_jumps=maximum_jumps,
    period=period,
)
result.certificate.require()
assert result.order == 3
```

```
assert np.allclose(result.packet.locations, locations)
assert np.allclose(result.packet.amplitudes, amplitudes)
reconstructed_edge = result.packet.evaluate(np.linspace(0, period, 1000))
```

The API requires `2 * maximum_jumps` consecutive positive Fourier modes starting at `background_degree + 1`. A missing mode produces a validation error. A decimated sample subset cannot substitute for this block. Polyphase offset lifting is a distinct algorithm and is not silently inferred here.

`hankel_moment_matrix` constructs `H[i,j] = m[i+j]`. The order gate counts singular values above `rank_tolerance * largest_singular_value`. A supplied `known_order` must not exceed the observed numerical rank at that threshold. The selected leading Hankel solve gives a monic annihilating polynomial. Polynomial roots give candidate nodes, which are projected to the unit circle. An all-moment least-squares fit gives candidate amplitudes. Their imaginary parts and total sum are checked before canonical real, mean-zero synthesis.

The zero-order branch returns an empty `PeriodicJumpPacket` whose evaluation is identically zero. An explicitly declared zero order with a substantial nonzero moment block fails the numerical certificate. Negative partners supplied for the requested positive-mode block must satisfy the real Fourier conjugacy condition. Measurements outside that block and its negative partners are not checked or used.

`PeriodicJumpPacket` reduces locations modulo the period, sorts them, copies its arrays, and keeps them read-only. Active locations must be distinct. Active amplitudes must be nonzero and have zero total within the supplied tolerance. A correction within that tolerance is removed before integration. Evaluation uses the right-continuous representative and subtracts its piecewise-constant mean using a closed formula evaluated in floating-point arithmetic. Its zero Fourier coefficient is zero.

The certificate verifies finite moment-fit, recurrence, node-circle, real-amplitude, and zero-sum consistency. It records the Hankel and Vandermonde condition numbers. The target's bandlimit, finite jump model, correctness of the rank threshold, and suitability of the noise and conditioning remain unchecked assumptions. A passing residual certifies consistency of the represented finite model. It does not certify the true number or locations of edges in an arbitrary noisy signal, a uniform stability theorem, or a model-free image method.

The module implements the finite Fourier model described here. Plotting and benchmarking frameworks, the largest-singular-ratio order heuristic, multi-offset phase lifting, greedy image dictionary selection, and infinite packet image reconstruction are outside its API. Its numerical certificates are not Lean proof certificates.

Run the focused regressions with:

SHELL

```
python -m pytest -q tests/test_fourier_recovery.py
```

15.15 Polyphase additive route packets

Perfect reconstruction in polyphase and filter-bank routing is additive and depends on cancellation. Convex or maximum-based branch aggregation generally changes that reconstruction. For the normalized Haar bank and input `(1, 0)`, the two branch contributions to the first reconstructed sample are `1/2` and `1/2`, while those to the second are `1/2` and `-1/2`. Sum semantics reconstruct `(1, 0)`. Max/convex semantics would produce `(1/2, 1/2)`.

A route packet therefore pairs its continuation state with a declared carrier algebra. See carrier-change obstructions (§ 15.16) for the conditions under which an aggregation law can be replaced.

For causal FIR polyphase routing the carrier is additive. If `E_h` is the polyphase route packet of `h`, with

OUTPUT

```
E_h[target, source][q] = h[N q + target - source],
```

then a cascade satisfies

OUTPUT

```
E_(g*h)[target, source]
  = sum_middle E_g[target, middle] * E_h[middle, source],
```

where $*$ on scalar entries is convolution in the coarse-lag variable. Intermediate phase/tap histories can therefore be summed exactly into a finite matrix of lag kernels. This is the standard polyphase matrix multiplication law expressed as a route-packet contract.

`komorebi_math.signals.route_packets.PolyphaseRoutePacket` implements this finite additive routing law.

15.16 Carrier-change obstructions

A carrier translation preserves downstream semantics only when it respects the operations and observations used by the computation. The cancellation in polyphase routing (§ 15.15) gives a concrete example.

15.16.1 Factorization criterion

Suppose one raw computation space `X` carries two exact algebraic semantics

OUTPUT

```
s : X -> S,    t : X -> T.
```

If both are homomorphisms for the expression grammar, then an exact carrier map `phi : image(s) -> T` with `t = phi o s` exists exactly when

OUTPUT

```
s(x) = s(y)  =>  t(x) = t(y).
```

Equivalently, the kernel congruence of `s` is contained in that of `t`. If the condition holds, define `phi(s(x)) = t(x)`. Kernel inclusion makes this well-defined, and the homomorphism law follows from those of `s` and `t`. If the condition fails, any collision `s(x)=s(y)` with `t(x)!= t(y)` is a complete carrier-change obstruction witness.

When neither semantic factors through the other, the joint map

OUTPUT

```
x -> (s(x), t(x))
```

is the coarsest raw quotient sufficient to recover both semantics. Its kernel is `ker(s) intersect ker(t)`. `finite_factorization_audit` implements the finite fiber test without presenting sample success as a proof of the universal claim.

A useful compiler interpretation follows. A query may always be applied *after* an exact source computation. It can be pushed down through intermediate nodes and used as a replacement carrier only when the corresponding operation squares commute. Signed nonzero-support is a valid final query of a signed sum, but it cannot be pushed through addition as Boolean OR because cancellation breaks that square.

For a source carrier `(M, plus_M, times_M)` and target carrier `(N, plus_N, times_N)`, a translation `phi : M -> N` is exact for every finite expression built from those operations only when it commutes with the required primitive operations (and with any branch actions / observations used by the query). This is the ordinary homomorphism requirement expressed as a packet contract.

Cancellation barrier

Let `R` be a nonzero ring and `S` a nontrivial zerosumfree semiring, meaning

OUTPUT

```
a plus_S b = 0  =>  a = b = 0.
```

There is no unital semiring homomorphism `R -> S`. Indeed,

OUTPUT

```
0 = phi(1 + (-1)) = 1_S plus_S phi(-1),
```

which would force `1_S = 0` by zerosumfreeness.

Thus signed additive routing cannot be exactly replaced by Boolean OR, max-plus, or another zerosumfree/idempotent carrier. The Haar cancellation example (§ 15.15) is an instance of this algebraic obstruction.

By contrast, on a zerosumfree domain such as the nonnegative reals the support map `x -> (x != 0)` does preserve `+` as Boolean OR and `*` as Boolean AND. The obstruction is cancellation.

Idempotence barrier

There is a dual obstruction. If the source carrier has idempotent addition, then `1 plus 1 = 1`. A unital homomorphism into a nontrivial additively cancellative carrier would force

OUTPUT

```
1 + 1 = 1,
```

and cancellation gives `1 = 0`, contradicting nontriviality. Thus Boolean or max-like semantics cannot be pushed into ordinary additive semantics while also recovering route multiplicity. This is the algebraic form of the duplicate-route counterexample (§ 15.15).

The loss is not bounded by a fixed finite residual alphabet on unbounded inputs. For raw packets containing `n >= 1` indistinguishable unit histories, Boolean support is always `true` while additive multiplicity is `n`. Any residual that recovers exact multiplicity from Boolean support must therefore distinguish arbitrarily many states.

15.16.2 Signed cancellation packet

`core.carrier_change.SignedMassPacket` stores positive and negative mass separately. Its observation

OUTPUT

```
value = positive - negative
```

preserves both addition and multiplication exactly under the packet operations. The quantity

OUTPUT

```
total_variation - abs(value) = 2 min(positive, negative)
```

is the exact unsigned-mass loss caused by cancellation.

This gives an exact completion for the scalar signed-routing query. Universal minimality among signed representations is a separate question.

15.16.3 Sum-product to max-plus

For scores `x_i` and temperature `tau > 0`, define

OUTPUT

```
m = max_i x_i
r = sum_i exp((x_i - m) / tau).
```

Then

OUTPUT

```
tau log(sum_i exp(x_i / tau)) = m + tau log r.
```

`TropicalResidualPacket` stores `(m, log r, term_count, tau)`. Its tropical projection keeps only `m`; its correction is `tau log r`.

The packet has exact compositional operations:

- merging alternatives maps `m` by `max` and reconstructs the finite-temperature sum exactly;
- multiplying positive expressions maps `m` by `+`, and the corrections add.

Moreover

OUTPUT

```
0 <= correction <= tau log(term_count).
```

The upper bound is sharp: if all `n` scores tie at the maximum, the correction is exactly `tau log n`. Hence finite-temperature max-plus replacement has no uniform error bound when fan-in is unbounded. The quantity

OUTPUT

```
effective_multiplicity = exp(correction / tau)
```

lies in `[1, n]` and records how much alternative-path mass the tropical projection has discarded.

The max-plus carrier is therefore the zero-temperature limit, while the residual records the exact finite-temperature scalar correction.

This is the standard Maslov-dequantization and log-sum-exp construction, expressed as a compositional carrier-change contract.

15.16.4 Query boundary of the residual completion

The tropical residual packet is exact for the paired scalar semantics `(finite-temperature sum-product value, tropical max value)`. It is not a provenance packet. For example, at `tau=1`, the score multisets

OUTPUT

```
[0, 0, log(1/2)]
[0, log(3/4), log(3/4)]
```

have the same maximum `0`, the same partition sum `5/2`, and the same total term count `3`, but the first has two maximizing branches and the second has one. Thus exact scalar carrier completion does not determine argmax multiplicity or identity. Those are separate future-query semantics and need additional provenance state.

15.17 Linear realizations and Hankel rank

Field-valued semantics admit linear state representations that can be much smaller than a discrete observer storing exact future behavior.

For a weighted series `f : Sigma* -> K`, define its residual after prefix `u` by

```
f_u(v) = f(uv).
```

A discrete future-equivalence observer identifies prefixes only when the full residual functions are equal. A linear realization instead stores a vector in the span of the residual functions. Over a field, the dimension of this span is the rank of the Hankel matrix `H[u,v] =`

`f(uv)` and equals the state dimension of a minimal weighted finite automaton. This is the classical weighted-automata and rational-series correspondence.

15.17.1 A polyphase example

Take the polyphase route packet (§ 15.15) for the two-tap filter `h=(1,1)`, polyphase order `N`, and evaluate the coarse-lag polynomial variable at `z=2`. The resulting route matrix is

`M_N = I + S`, with `S^N = 2 I`.

For the scalar downstream query

```
f_N(n) = e_0^T M_N^n e_0,
```

the `N x N` Hankel block has full rank `N`. Thus the `N`-phase linear route state is already minimal for this query.

For `N=2`,

```
M = [[1,2],[1,1]]
```

and

```
f(n) = 1,1,3,7,17,41,...,
```

which satisfies

```
f(n+2)=2 f(n+1)+f(n).
```

All discrete residual functions `k -> f(n+k)` are distinct, so a deterministic Moore observer storing exact future behavior requires infinitely many states. The linear/Hankel representation requires dimension two.

This example illustrates why weighted automata and linear state-space models can compress numerical series more effectively than deterministic state equality.

15.17.2 Architectural consequence

The continuation quotient must be typed by the observation algebra:

- set or Boolean semantics: future-equivalence classes, as in Myhill–Nerode theory
- field-linear semantics: the residual vector-space quotient and Hankel rank
- other semirings: the appropriate semimodule/congruence notion, when it is well behaved.

15.17.3 API boundary

The polyphase route packet supplies the finite linear realization used here. The recurrence subsystem supplies companion-state coordinates. A generic weighted-automaton constructor is outside these APIs. Minimality depends on the observation algebra, as described in carrier-change obstructions (§ 15.16).

PART IV

Nonlinear calculus and integration

Derivatives, jets, finite measures and typed expressions organize more elaborate calculations. Exact algebraic identities remain distinct from sampled numerical checks and assumptions supplied by the caller.

16 Nonlinear calculus

Finite nonlinear packet calculus resolves directional derivatives into input and output packets. The same derivative interface supports composition, finite tree expansions, correction equations, and transport. Each construction records its arithmetic model and the assumptions needed to interpret its certificate.

16.1 Derivative providers

A derivative provider declares source and target dimensions, exactness, map evaluation, directional derivatives, and the supported derivative orders.

PYTHON | Executable example

```
from komorebi_math.nonlinear import DerivativeProvider
```

The concrete providers are:

Provider	Use
`CallableDerivativeProvider`	declared analytic derivatives
`PolynomialDerivativeProvider`	finite symmetric coefficient tensors
`FiniteDifferenceDerivativeProvider`	numerical directional finite differences
`SympyDerivativeProvider`	optional exact symbolic derivatives
`ComposedDerivativeProvider`	composition through Faà di Bruno

For `PolynomialDerivativeProvider`, tensors satisfy

OUTPUT

```
F(x) = sum_r T_r[x,...,x] / r!
```

Thus `T_r` is the declared `r`-th derivative tensor at the origin, not the ordinary polynomial coefficient.

16.2 Packet jets

Given source packets `P_a`, target packets `Q_b`, and an order-`r` derivative,

OUTPUT

```
J[b; a1,...,ar](h1,...,hr)
  = Q_b D^r F(x)[P_a1 h1,...,P_ar hr].
```

`PacketJetFamily` constructs these blocks lazily.

PYTHON | Contextual fragment

```python
family = PacketJetFamily(provider, point, source_packets, target_packets, order=3)
jet = family.jet("output", ("left", "middle", "right"))
value = jet.evaluate((h1, h2, h3))
family.reconstruction_certificate((h1, h2, h3)).require()
```

`packet_jet_support` evaluates addresses under an explicit limit. A complete scan is exact or numerical according to the provider. A truncated scan is conditional because untested addresses may be nonzero.

16.3 Faà di Bruno composition

For `F = outer o inner`, the order-`n` directional derivative is a sum over set partitions of `{0,...,n-1}`. Each block feeds one derivative of the inner map into a derivative of the outer map.

PYTHON | Contextual fragment

```python
result = faa_di_bruno(outer, inner, point, directions)
result.certificate.require()
```

The result records every retained term and combinatorial counts. `packet_faa_di_bruno` adds packet labels at the input, intermediate, and output stages.

16.4 Rooted trees

A leaf represents the seed. An internal vertex of arity `r` represents the declared `r`-linear operation. `PolynomialFixedPointProblem` evaluates all finite trees up to a depth bound and compares the sum with the corresponding Picard iterate.

PYTHON | Contextual fragment

```
problem = PolynomialFixedPointProblem(seed, {2: bilinear, 3: trilinear})
result = problem.tree_expansion(3, max_trees=100_000)
result.certificate.require()
```

Tree enumeration has explicit limits because the number of trees grows rapidly. `majorant_convergence` separately checks the supplied bounds for a scalar invariant ball and a contraction criterion. The operation norms must be supplied by the caller rather than inferred from the callable.

16.5 Graded operations

`FiniteAbelianGroup` supplies exact labels and addition. `GradedPacketOperation` checks that an `r`-linear operation maps grades `(a1,...,ar)` into their group sum.

The higher Maurer-Cartan utility evaluates a finite truncation

OUTPUT

```
sum_r l_r(x,...,x) / r!
```

and resolves the result by grade. It is a finite identity, not an assertion that unspecified higher operations exist.

16.6 Correction and elimination

16.6.1 Hilbert obstruction

`HilbertObstructionProjector` separates a residual into the range of a linearized operator and its orthogonal obstruction component. When the obstruction vanishes, it returns the least-norm correction.

16.6.2 Right-inverse correction

`RightInverseCorrectionProblem` implements a contraction argument with explicit right inverse, radius, and Lipschitz bound. The certificate checks the supplied finite inequalities.

16.6.3 Lyapunov-Schmidt reduction

`LyapunovSchmidtReduction` computes numerical kernel, complement, range, and obstruction spaces from the base derivative. It solves the range equation and exposes the reduced obstruction map.

16.6.4 Nonlinear Schur elimination

`NonlinearSchurElimination` eliminates hidden variables locally. It returns the implicit first derivative and the reduced first and second derivatives. Exact providers produce exact SymPy matrices. Numerical providers use finite matrix solves and residual certificates.

16.7 Filtered lifting

`FilteredVectorSpace` represents a finite decreasing filtration over `F_p`. It provides level bases, associated-graded bases, coordinates, and lifts.

`FilteredCorrectionProblem` solves the equation at one filtration depth on the associated graded and returns the correction coset. `verify_finite_filtered_branch` checks a supplied sequence of approximations through a finite terminal depth.

`conditional_projective_limit_completion` records, but does not prove, the external infinite hypotheses.

16.8 Filtered spectral sequences

`FiniteFilteredCochainComplex` requires an explicit finite complex and a filtration by subcomplexes. It computes finite pages from `Z_r` and `B_r` subspaces, induced differential ranks, and limiting dimensions.

PYTHON | Contextual fragment

```python
result = complex_.spectral_sequence()
result.certificate.require()
for page in result.pages:
    print(page.page, page.total_dimension, page.total_differential_rank)
```

The finite terminal filtration permits an exact dimensional convergence check.

16.9 Smooth transport

`MovingPacketSystem` validates the orthogonal packet axioms at each evaluated time. Its derivative certificate compares the declared derivatives with centered finite differences and checks the differentiated projector identities at the sampled point. Differentiability beyond that point remains an assumption. The canonical generator is

OUTPUT

```
K(t) = sum_a dot(P_a(t)) P_a(t).
```

`KatoTransport` integrates this generator by midpoint exponential steps. It also forms products of direct polar rotations and compares nested meshes.

The same classes are re-exported from `komorebi_math.quantum`.

17 Scalable nonlinear execution

Directional evaluation separates a derivative from its storage representation. Sparse tensors, automatic differentiation adapters, and compiled packet plans can therefore serve the same mathematical interface while exposing their different costs and assumptions.

17.1 Directional derivative contract

Every derivative provider implements the same finite directional interface. At a point `x` and directions `v_1, ..., v_r`, the call

PYTHON | Contextual fragment

```
provider.derivative(r, x, (v_1, ..., v_r))
```

returns the ordered directional derivative

$$D^r F(x)[v_1, \dots, v_r].$$

The protocol does not require construction of a dense Jacobian or higher-order tensor. Dense polynomial, sparse polynomial, finite-difference, declared callable, SymPy, JAX, and PyTorch implementations can therefore be consumed by the same packet-jet machinery.

17.2 Sparse symmetric tensors

A dense order-`r` derivative tensor has shape

OUTPUT

```
(target_dimension, source_dimension, ..., source_dimension)
```

with `r` source axes. For a symmetric derivative, coefficients related by a permutation of the source axes are equal. `SparseSymmetricTensor` stores canonical nondecreasing source-index tuples and expands their distinct permutations only when reconstructing dense data or contracting with directions.

PYTHON | Executable example

```python
from fractions import Fraction
from komorebi_math.nonlinear import SparseSymmetricTensor

tensor = SparseSymmetricTensor.from_mapping(
    order=3,
    source_dimension=3,
    target_dimension=1,
    entries={(0, 0, 1, 2): Fraction(7, 11)},
)
tensor.certificate().require()
```

For an entry indexed by `(o, i_1, ..., i_r)`, contraction computes the sum over the full distinct permutation orbit. Repeated indices are handled without multiplicity errors.

17.2.1 Storage diagnostics

The object reports:

- `stored_entry_count`
- `expanded_nonzero_count`
- `dense_entry_count`
- `storage_ratio`

These are deterministic counts. They do not include Python object overhead, allocator behavior, cache effects, or backend-specific memory layouts.

17.3 Automatic differentiation adapters

`AutomaticDifferentiationProvider` accepts a JVP function with the form:

PYTHON | Contextual fragment

```python
def jvp(function, point, direction):
    return primal_value, directional_tangent
```

Nested JVPs provide higher-order directional derivatives. The generic adapter is useful for custom dual-number systems, local differentiable programming runtimes, or finite validation JVPs.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.nonlinear import AutomaticDifferentiationProvider

def function(x):
    return np.array([x[0] ** 3 + x[0] * x[1]])

def central_jvp(function, point, direction):
    h = 1e-6
    return function(point), (function(point + h * direction) - function(point - h
        ↪  * direction)) / (2 * h)

provider = AutomaticDifferentiationProvider(function, central_jvp, 2, 1,
    ↪  max_order=1)
```

The accuracy and differentiability properties of a custom JVP remain the caller's responsibility. A finite-difference JVP supplies a numerical approximation rather than an automatic derivative.

17.3.1 JAX and PyTorch

PYTHON | Executable example

```
from komorebi_math.nonlinear import JaxDerivativeProvider, TorchDerivativeProvider
```

Both adapters are lazy. Merely importing these names does not import the optional framework. The provider validates source and target dimensions and guards the declared maximum order.

Framework tracing rules still apply. A function that escapes to NumPy, mutates traced values, or performs unsupported control flow can fail in the underlying framework. Komorebi Math preserves and reports that failure rather than silently replacing the derivative.

17.4 Packet-jet plan compilation

A packet-jet family combines a provider, a base point, input packets, output packets, and a finite order. The naive address count is

$$N_{\text{out}} N_{\text{in}}^{r}.$$

Plan compilation performs two independent reductions:

1. **Certified support pruning.** A complete support certificate may prove some packet addresses zero.
2. **Input-group reuse.** Addresses with the same ordered input packet tuple share the same projected directions and one provider derivative call.

PYTHON | Contextual fragment

```
support = packet_jet_support(family)
plan = compile_packet_jet_plan(family, support)
result = execute_packet_jet_plan(plan, family, directions)
result.certificate.require()
```

An incomplete or absent support certificate never justifies pruning. This conservative rule prevents finite probing from being promoted into an exact structural zero claim.

17.5 Execution certificates

The batch result records:

- active packet evaluations
- reconstructed derivative
- direct derivative
- reconstruction residual
- provider derivative-call count
- naive derivative-call count
- number of pruned blocks
- exactness inherited from the provider and packet system

The certificate validates the finite identity

$$D^r F(x)[v_1, \dots, v_r] = \sum_{b, a_1, \dots, a_r} Q_b D^r F(x)[P_{a_1} v_1, \dots, P_{a_r} v_r]$$

within the declared tolerance for numerical data.

17.6 CuPy execution boundary

The CuPy classes accept device-aware providers and packet data when a compatible runtime is present. `cupy_runtime_status()` returns a structured status before construction. No implicit CPU fallback is used by a class whose contract promises CuPy execution.

A matched benchmark contract describes the case signature and timing scope. An observed speedup requires measurements from the corresponding benchmark.

17.7 Complexity boundaries

Sparse storage is beneficial only when derivative tensors contain structural zeros or repeated symmetric coefficients. Projection caching reduces repeated packet projections and provider calls, but the output projection and reconstruction costs remain. Snapshot capture is exponential in derivative order at fixed source dimension. These APIs expose finite structure and reuse. They do not remove the general combinatorial growth of high-order derivatives.

18 Streamed packet jets

18.1 Purpose

A finite packet-jet snapshot stores every retained standard-basis coefficient in one in-memory record. That representation is appropriate for small and moderate derivatives. Its dense capture cost is still

$$N_{\text{basis}} = d_{\text{source}}^{r},$$

where `r` is the derivative order. A high-order snapshot can therefore be too large to capture, serialize, or reload as one object.

The directory-backed stream format stores packet jets incrementally. It writes coefficients in chunks with independent digests, records progress after each chunk, and supports resuming an interrupted capture. A reader verifies and decodes at most one chunk at a time during iteration or evaluation.

Streaming bounds the number of coefficients held in memory at once. The exponential basis count remains.

18.2 Two write modes

18.2.1 Direct capture

`capture_packet_jet_stream` evaluates a live `PacketJet` on the ambient standard basis. `evaluation_budget` limits the additional basis evaluations performed by one call.

PYTHON | Contextual fragment

```
from komorebi_math.nonlinear import capture_packet_jet_stream

partial = capture_packet_jet_stream(
    jet,
    "quadratic.kmstream",
    chunk_size=256,
    evaluation_budget=500,
)
assert not partial.completed

complete = capture_packet_jet_stream(
    jet,
    "quadratic.kmstream",
    chunk_size=256,
    resume=True,
)
complete.certificate.require()
```

An incomplete write lives at `<destination>.partial`. The requested destination appears only after a complete manifest has been written and revalidated.

A direct resume compares the packet address, dimensions, provider name, exactness, point, tolerance, and chunk size with the recorded metadata. Arbitrary Python callables do not have a portable semantic hash. The caller must therefore keep the provider implementation fixed across a resume. The certificate records that limitation.

18.2.2 Existing snapshot

`save_packet_jet_snapshot_stream` divides an already captured snapshot into coefficient chunks. `coefficient_budget` limits the additional coefficients written by one call.

PYTHON | Contextual fragment

```
from komorebi_math.nonlinear import save_packet_jet_snapshot_stream

result = save_packet_jet_snapshot_stream(
    snapshot,
    "snapshot.kmstream",
    chunk_size=1024,
)
result.certificate.require()
```

Snapshot mode fingerprints the complete coefficient record and capture certificate. Resuming with a different snapshot is rejected.

18.3 Directory format

A complete stream has the following structure:

OUTPUT

```
snapshot.kmstream/
  manifest.json
  chunks/
    chunk-00000000.json
    chunk-00000001.json
    ...
```

Each JSON file has its own envelope:

JSON

```
{
  "schema": "komorebi-math.packet-jet-stream.chunk",
  "schema_version": 1,
  "payload_sha256": "...",
  "payload": {}
}
```

The manifest records:

- packet address and dimensions
- provider label and exactness
- base point and tolerance

- direct or snapshot capture mode
- source fingerprint
- dense basis count
- completed basis and coefficient counts
- canonical chunk records
- omitted-zero residual bound
- final capture certificate for complete streams
- a stream content digest

The content digest covers metadata, progress, chunk payload digests, and the final certificate digest. Chunk payload digests cover coefficient values and ordinal ranges.

18.4 Atomic progress

A chunk is written to a temporary file in the target directory, flushed, and atomically renamed. Only after that rename succeeds is the manifest advanced. Before resuming work, the writer reads every committed prefix chunk and checks its digest. An interrupted write can leave an orphan chunk, but resume removes chunks beyond the committed manifest length only after the committed prefix has passed verification.

Completion uses a staging directory. Promotion to the requested destination is atomic when the source and destination are on the same filesystem. Explicit overwrite uses a temporary backup and restores it if promotion fails.

Filesystem durability still depends on operating-system and storage semantics. The implementation does not claim transactional guarantees across hardware failure, network filesystems, or an adversarial storage layer.

18.5 Bounded reads

PYTHON | Contextual fragment

```
from komorebi_math.nonlinear import open_packet_jet_snapshot_stream

reader = open_packet_jet_snapshot_stream(
    "snapshot.kmstream",
    verify_chunks=True,
)

for record, coefficients in reader.iter_chunks():
    process(record, coefficients)

selection = reader.select(start=100, stop=200, max_coefficients=100)
selection.certificate.require()
```

`iter_chunks` and `iter_coefficients` retain one decoded chunk at a time. `select` and `load_snapshot` require explicit coefficient guards because they materialize data in memory.

18.6 Complete and partial evaluation

PYTHON | Contextual fragment

```python
result = reader.evaluate(directions)
result.certificate.require()

left = reader.evaluate(directions, start=0, stop=100)
right = reader.evaluate(directions, start=100)
```

A complete evaluation receives a complete-snapshot certificate. A bounded ordinal range is a declared partial sum. Its certificate lists coefficients outside the selected range as unchecked. The API never labels a partial range as the complete multilinear map.

18.7 Integrity and authenticity

SHA-256 digests detect accidental modification and any modification that does not also replace the affected digest chain. They are not signatures. An attacker who can replace the payloads, manifests, and digests can construct another internally consistent stream.

Use an authenticated filesystem, an external signature, or a signed release manifest when provenance or authorship matters.

18.8 Resource boundaries

The stream layer has explicit guards for:

- maximum dense basis evaluations during direct capture
- per-call evaluation or coefficient budgets
- positive chunk size
- materialized coefficient count
- selection ranges and dimensions

The complete manifest still grows linearly with the number of chunks. Choose a chunk size that balances restart granularity, manifest size, and filesystem overhead.

18.9 Command line inspection

POWERSHELL

```
komorebi stream inspect .\snapshot.kmstream --verify-chunks
komorebi stream verify .\snapshot.kmstream
```

Use `--allow-incomplete` only when intentionally inspecting a `.partial` capture. An incomplete stream cannot be materialized as a complete `PacketJetSnapshot`.

19 Finite measures and integration

The integration API supplies a common operation for explicit finite measures:

PYTHON | Contextual fragment

```
result = integrate(integrand, measure, provider=optional_provider)
```

The integrand may be an `ExactPolynomial` or a callable. Measures have an explicit finite dimension and expose total mass and monomial moments.

19.1 Positive measures

- `AtomicMeasure` represents a finite positive weighted sum of point masses.
- `IntervalMeasure` represents a constant nonnegative density on a bounded interval, including zero-density and degenerate zero measures.
- `ProductMeasure` supplies exact factorized moments and finite Fubini checks.
- `PushforwardMeasure` implements integration by precomposition. Exact affine maps pull exact polynomials back using rational arithmetic.

Negative weights are rejected by positive measures.

19.2 Quadrature functionals

`QuadratureRule` and `ExactQuadratureRule` are finite linear functionals. Their weights may be signed, so they are not automatically measures. An exact rule computes its finite sum exactly, but this does not prove equality with a target integral.

`degree_of_exactness` compares rule moments with measure moments. Only an exact moment match through the integrand degree allows `certify_polynomial_integral` to return an exact integral certificate.

`CompositeSimpson` evaluates exact polynomials and arbitrary callables over bounded interval factors, but those paths make different claims:

- `rigorous` is available only for a provenance-matched univariate `ExactPolynomial` and the same exact rational `IntervalMeasure`. The bound created by `polynomial_derivative_bound` must match both objects, and Simpson nodes, weights, polynomial evaluation, summation, and the final error bound are all computed with exact rational arithmetic.
- `conditional` applies to an arbitrary callable when a derivative bound is explicitly assumed. Applicability to that callable and interval, as well as floating-point node, weight, evaluation, summation, and roundoff enclosure, remain visible unchecked assumptions.

- `estimated` applies to a fine/coarse Richardson estimate without a rigorous provenance chain, including the ordinary arbitrary-floating-callable path.
- `unknown` is used when no integral error theorem exists.

An internally rigorous polynomial bound cannot be attached to an arbitrary callable, even if that callable appears to implement the same formula. The caller must pass the matching `ExactPolynomial` itself to retain rigorous provenance.

19.3 Exact identities

The exact polynomial layer certifies:

- the fundamental theorem of calculus
- integration by parts with the boundary term and constant density
- finite Fubini identities for rational interval and atomic products
- affine pushforward and one-dimensional Jacobian density laws
- oriented polynomial bounds without treating orientation as a positive measure
- coordinate covariance of the integration covector

For a polynomial family with derivative matrix D, integration covector ℓ, density ρ, and endpoint evaluation covectors e_a, e_b,

$$\ell D = \rho(e_b - e_a).$$

If C maps source coefficients to target coefficients, then

$$\ell_{\text{target}} C = \ell_{\text{source}}.$$

This is the coordinate law that closes integration with the existing polynomial coordinate calculus.

`polynomial_integration_covector` records and replays the exact graded basis witness before issuing an exact certificate. A covector constructed directly from arbitrary weights remains a structurally valid finite linear functional, but its relationship to the named basis is conditional and explicitly unchecked.

19.4 Existing-calculus adapters

- normalized `FormalSeries` values convert to exact ordinary polynomials
- arbitrary exact polynomial-family coordinates integrate directly
- `DerivativeProvider` values integrate numerically through a measure
- `AtomicIntegratedDerivativeProvider` differentiates a fixed finite weighted sum
- analysis and synthesis maps are checked for commutation with atomic integration

The derivative adapter assumes fixed weights. A parameter-dependent measure has an additional derivative-of-measure term and is not silently accepted. Provider dimensions must be positive integers, every value and derivative output must have shape `(target_dimension, )`, and exact object arithmetic is preserved. The frame commutation certificate incorporates the certificate of the underlying analysis and synthesis system, including its validity, assumptions, and diagnostics. Zero-valued samples cannot make an invalid system pass.

19.5 Verification

Run the installed integration checks with:

CONSOLE

```
python -m komorebi_math verify integration
```

The source tests include independent direct calculations, exact moment checks, boundary counterexamples, and controls for unsupported quadrature claims.

19.6 Scope

This finite integration API does not claim general Lebesgue integration, automatic measurability, countably additive infinite-space construction, improper or stochastic integration, generic nonlinear change of variables, manifold integration, Radon–Nikodym automation, or dominated-convergence proofs.

19.7 Exact integration through a coordinate change

This example follows one integral through measure transport, polynomial pullback, coordinate change, exact evaluation, and certified numerical evaluation.

19.7.1 Base measure and chart

Let $\mu = dx$ on $[0, 1]$, and define

$$\phi(x) = 2x + 1.$$

The pushforward $\nu = \phi_{\#}\mu$ is supported on $[1, 3]$. Relative to ordinary Lebesgue measure dy, its density is

$$d\nu(y) = \frac{1}{2}\, dy.$$

The factor is $1/|\phi'| = 1/2$. This density statement uses the Jacobian. The abstract pushforward identity itself does not:

$$\int g\, d(\phi_{\#}\mu) = \int g \circ \phi\, d\mu.$$

19.7.2 Integrand and exact pullback

Take

$$f(y) = y^4 - 2y + 1.$$

The exact affine pullback is

$$f(2x+1) = 16x^4 + 32x^3 + 24x^2 + 4x.$$

Therefore

$$\begin{aligned}\int f\, d\nu &= \frac{1}{2}\int_1^3 (y^4 - 2y + 1)\, dy \\ &= \int_0^1 (16x^4 + 32x^3 + 24x^2 + 4x)\, dx \\ &= \frac{16}{5} + 8 + 8 + 2 \\ &= \boxed{\frac{106}{5}}.\end{aligned}$$

The integration API evaluates both sides using rational arithmetic and certifies their exact equality.

19.7.3 The integral as a coordinate covector

Let c be the coefficient vector of f, and let ℓ be the vector of basis-function integrals. Then

$$\int f\, d\nu = \ell c.$$

The same polynomial gives the following exact pairings in three coordinate bases.

Basis	Coordinates of f	Integration covector	Pairing
Monomial	$(1, -2, 0, 0, 1)$	$(1, 2, 13/3, 10, 121/5)$	$106/5$
Euler	$(1/2, 0, 3, 2, 1)$	$(1, 3/2, 7/3, 15/4, 31/5)$	$106/5$
Fibonacci	$(3, -2, -3, 0, 1)$	$(1, 2, 16/3, 14, 191/5)$	$106/5$

If C maps old coefficients to new coefficients, the covector law is

$$\ell_{\text{new}} C = \ell_{\text{old}}.$$

The example certifies this contragredient transport exactly. It also checks the basis-independent fundamental theorem identity

$$\ell D = \frac{1}{2}(e_3 - e_1).$$

19.7.4 Quadrature with a rigorous error bound

The example passes the exact polynomial itself, its exact rational interval, and the fourth-derivative bound derived from that same polynomial and interval to composite Simpson quadrature. With 16 subintervals, exact rational nodes, weights, evaluations, and summation return

$$Q = \frac{130253}{6144} = 21.200032552083333\ldots.$$

The exact integral is $106/5 = 21.2$. Since $f^{(4)} = 24$, the provenance-matched exact calculation derives the rigorous error bound

$$|I - Q| \le \frac{1}{30720} = 3.255208333333333\ldots \times 10^{-5}.$$

Here the observed error is exactly $1/30720$, so it lies inside the bound. The rigorous label depends on the complete provenance chain: a matching `ExactPolynomial`, the same exact rational `IntervalMeasure`, and exact rational evaluation and bound arithmetic. If no derivative bound is supplied, the fine/coarse statement is estimated. If an arbitrary floating callable is used with an assumed derivative bound, the result is conditional rather than rigorous because bound applicability and floating-point roundoff are not proved.

19.7.5 Why sampled agreement is not a proof

The polynomial

$$h(x) = x^2 \left(x - \frac{1}{2}\right)^2 (x-1)^2$$

vanishes at every node of the three-node Simpson rule on $[0, 1]$. The sampled quadrature value is zero, but

$$\int_0^1 h(x)\,dx = \frac{1}{840}.$$

The exactness checker finds that Simpson is certified only through degree three, while h has degree six. It rejects the attempted exact-integral certificate.

19.7.6 Run it

From a source checkout or source distribution:

POWERSHELL

```
python examples/measure_integration.py
```

Machine-readable output:

POWERSHELL

```
python examples/measure_integration.py --json
```

20 Typed finite expressions

The typed expression calculus composes finite polynomial and coordinate operators using exact rational values. It supplies explicit types, replayable transformations, canonical normal forms, and decidable equality within its finite semantic domain.

The implemented semantic domain is finite and rational:

$$\mathbb{Q}, \qquad \mathcal{P}_n^B(x) = \operatorname{span}_{\mathbb{Q}}\{b_0(x), \dots, b_n(x)\}, \qquad \operatorname{Hom}_{\mathbb{Q}}(X, Y).$$

Every coefficient, matrix entry, endpoint, density, and affine parameter in the exact fragment is an integer or a reduced `Fraction`.

20.1 Exact polynomial spaces

A polynomial coordinate space is not identified by a family name. It is identified by

$$(x, n, B), \qquad B = (b_0, \dots, b_n),$$

where each basis polynomial has exact rational coefficients,

$$b_k(x) = \sum_{j=0}^{k} B_{jk} x^j, \qquad B_{kk} \neq 0.$$

Thus the coefficient matrix

$$B_n = (B_{jk})_{0 \le j,k \le n}$$

is triangular with a nonzero diagonal and is exactly invertible over $\mathbb{Q}$. The display label is metadata. Two spaces with the same label and dimension but different coefficient witnesses are different types.

A shared family label does not establish type equality. The type checker compares the complete variable, degree, and basis witness. Its structural fingerprint is a deterministic digest of that canonical exact data. Exact type equality still comes from the underlying witness, not from assuming that a cryptographic hash can never collide.

The primary type objects are `RationalSpace`, `PolynomialCoordinateSpace`, and `LinearMapType`. A linear map type records its source and target spaces explicitly.

20.2 Typed expressions

The expression layer contains immutable nodes for exact values, exact maps, symbols, typed `let` bindings, identities, zero maps, composition, sums, rational scaling, and map application. The constructors reject an expression before it can enter the graph when any of the following fails:

- a vector length does not equal its space dimension
- a matrix shape does not equal target dimension by source dimension
- adjacent composition endpoints do not agree
- a sum contains unlike types
- a map is applied to a value outside its source
- a bound value does not have the symbol's declared type

Composition is stored outermost to innermost. If

$$A : X \to Y, \qquad B : Y \to Z,$$

then `compose(B, A)` denotes $B \circ A : X \to Z$. The `then(A, B)` helper accepts the same maps in the order in which they act.

`Symbol` and `Let` provide a finite first-order binding mechanism. They are not a lambda calculus and do not introduce higher-order functions. Evaluation of a closed term checks scope and declared types. Free symbols remain explicit and cannot be silently replaced by similarly named objects. Binder identity is object identity, not dataclass field equality, so a free symbol with the same printed name and declared type as a binder does not become bound accidentally.

20.3 Replayable polynomial primitives

For a basis matrix B_n, a coefficient vector c_B represents the monomial coefficient vector

$$c_M = B_n c_B.$$

If S and T are exact bases through the same degree, the coordinate change from source to target is

$$C^{(n)}_{S\to T} = B^{-1}_{T,n} B_{S,n}.$$

It follows immediately that

$$C^{(n)}_{T\to U} C^{(n)}_{S\to T} = C^{(n)}_{S\to U}, \qquad C^{(n)}_{T\to S} C^{(n)}_{S\to T} = I_{n+1}.$$

The other primitive matrices are also reconstructed from their exact structural data rather than accepted from a label or a certificate supplied by a caller. The public primitive constructors cover:

- coordinate change
- the tight derivative
- a basepoint antiderivative
- point evaluation
- constant-density interval integration
- affine substitution
- finite difference
- the endpoint boundary functional
- constant injection

`replay` recomputes the matrix. When a caller supplies a claimed matrix, exact certification compares every entry with that replay.

20.3.1 Derivative and basepoint antiderivative

For $n \geq 1$, the tight derivative is

$$D_B^{(n)} : \mathcal{P}_n^B \longrightarrow \mathcal{P}_{n-1}^B.$$

It has shape $n \times (n+1)$, not $(n+1) \times (n+1)$. Let

$$J_{a,B}^{(n)} : \mathcal{P}_{n-1}^B \longrightarrow \mathcal{P}_n^B$$

be the unique primitive whose value at a is zero. Then

$$D_B^{(n)} J_{a,B}^{(n)} = I_n.$$

The reverse product is not the identity. If u_B is the coordinate column of the constant polynomial 1, and $e_{a,B}$ is evaluation at a, then

$$J_{a,B}^{(n)} D_B^{(n)} = I_{n+1} - u_B e_{a,B}.$$

This rank-one defect records the lost integration constant.

20.3.2 Evaluation, integration, and the fundamental theorem

Evaluation and integration are exact covectors. For constant density $\rho \geq 0$,

$$e_{a,B}c_B = p(a), \qquad \ell_{B,[a,b],\rho}c_B = \rho \int_a^b p(x)\,dx.$$

They transport contragrediently under coordinate changes:

$$e_{a,T}C_{S\to T} = e_{a,S}, \qquad \ell_{T,[a,b],\rho}C_{S\to T} = \ell_{S,[a,b],\rho}.$$

The derivative and antiderivative naturality squares are

$$D_T^{(n)}C_{S\to T}^{(n)} = C_{S\to T}^{(n-1)}D_S^{(n)},$$

and

$$J_{a,T}^{(n)}C_{S\to T}^{(n-1)} = C_{S\to T}^{(n)}J_{a,S}^{(n)}.$$

The finite exact fundamental theorem is the row identity

$$\ell_{B,[a,b],\rho}^{(n-1)}D_B^{(n)} = \rho(e_{b,B}^{(n)} - e_{a,B}^{(n)}).$$

These are matrix equalities over $\mathbb{Q}$. No sampling tolerance is used.

20.4 Exact normal forms and decidable equality

A closed, well-typed map expression normalizes to its type and one exact rational matrix. A closed value expression normalizes to its space and one exact rational vector, or one rational scalar in $\mathbb{Q}$. `Fraction` normalization gives each entry a unique numerator and positive denominator. Matrix dimensions and the complete space witnesses are part of the normal form.

Consequently, equality is decidable for this closed exact language:

1. type-check both expressions
2. replay every primitive
3. evaluate finite sums, scales, applications, bindings, and products
4. compare the resulting types and rational entries

There is no numerical threshold and no dependence on a printed family name. For expressions with unresolved free symbols, canonical structural equality is decidable, but the expression calculus does not decide every universally quantified identity that a user might intend those symbols to satisfy.

20.5 Canonical data and digests

Types and expression graphs have a closed, versioned JSON representation. The portable entry points are `type_to_dict`, `type_from_dict`, `expression_to_dict`, `expression_from_dict`, `expression_to_json`, `expression_from_json`, and `expression_digest`.

The decoder accepts only the declared expression schema and exact rational tokens. It does not import arbitrary classes, call arbitrary functions, or deserialize arbitrary Python objects. Deterministic field order, normalized Unicode text, normalized fractions, exact basis witnesses, and canonical child order make repeated serialization stable. Noncommutative composition order is never sorted away.

20.6 Certified rewrites

A rewrite is more than a before-and-after pair. A `RewriteStep` records the rule name, theorem identifier, before digest, after digest, checked side conditions, and its certificate. Built-in exact rewrites are replayed in the finite rational model. Type preservation and equality of exact normal forms are checkable.

Certificate composition never strengthens a child claim. Invalid evidence remains invalid. Numerical, conditional, heuristic, and planned inputs cannot become exact merely because a surrounding algebraic identity is exact. Any unchecked side condition makes the composed conclusion at most conditional.

The guarded rewrites include coordinate cocycles, derivative and antiderivative laws, coordinate naturality, evaluation and integration transport, and the finite fundamental theorem. Inverse cancellation requires a replayed exact inverse. A square shape or a certificate whose message says “exact” is not enough.

20.7 Matrix-chain planning

Once an expression has become a compatible chain of dense matrices, the planner may choose a multiplication parenthesization. For matrices with dimension chain

$$p_0 \times p_1, p_1 \times p_2, \dots, p_{r-1} \times p_r,$$

the dynamic program uses

$$m[i,j] = \min_{i \le k < j} \left(m[i,k] + m[k+1,j] + p_i p_{k+1} p_{j+1} \right).$$

This proves the minimum dense scalar-operation count among binary parenthesizations that preserve the matrix order. It does not optimize across algebraic rewrites, sparsity patterns, low-rank structure, memory traffic, parallel scheduling, device transfer, or hardware-specific kernels. Equal-cost plans may exist even when deterministic tie-breaking selects one of them.

20.8 Resource boundary

`ExpressionBudget` checks a finite expression DAG before execution. It bounds node count, depth, traversal steps, polynomial degree, exact-entry count, coefficient bit length, serialized structural size, and symbol length. Cycles and malformed children are rejected. These guards are part of the semantics: exceeding a declared limit does not produce a partial exact certificate.

Graph checking, normalization, simplification, and plan construction memoize shared nodes in their applicable lexical context, so a DAG is not evaluated as an exponentially expanded tree. Exact intermediate and final values are rechecked against entry, coefficient-bit, and serialized-size limits. The dense matrix-chain dynamic program also has a fixed work guard, and expression compilation charges planning work to the caller's step budget.

Portable JSON necessarily expands shared nodes into occurrences. Before an envelope is returned, the encoder replays that generated tree through the same strict decoder and the same budget. Thus an encoder cannot emit a transport it would reject itself because of expanded node, edge, exact-entry, label, or type witness counts. Runtime validation also accepts only the exact built-in node and type classes and replays constructor invariants, preventing subclasses or mutated frozen objects from spoofing a certified denotation.

20.9 Scope

The expression calculus does not claim:

- arbitrary symbolic algebra or arbitrary user-defined rewrite theories
- nonlinear, infinite-dimensional, or function-space theorem proving
- analytic convergence or equality of general functions
- floating-point error analysis or numerical stability
- general Lebesgue, stochastic, manifold, or distributional calculus
- a universal optimizer

It is a finite exact linear language for a deliberately bounded polynomial coordinate calculus. Within that boundary, types, replay, normal forms, and equality are explicit and mechanically checkable.

20.10 Exact typed expressions across two bases

This example follows one exact polynomial through antiderivation, two coordinate changes, differentiation, and definite integration. Every vector and matrix is over $\mathbb{Q}$. The complete operator compiles to I_4, while the final scalar is $5/4$.

The second half expresses the affine integration result $106/5$ inside the finite expression language and records four failure cases that must not be rewritten away.

20.10.1 The two graded bases

Let the standard Fibonacci polynomials satisfy

$$F_0(x) = 0, \qquad F_1(x) = 1, \qquad F_{n+1}(x) = xF_n(x) + F_{n-1}(x).$$

Komorebi Math uses the graded basis $G_k = F_{k+1}$. Through degree four,

$$\begin{aligned} G_0 &= 1, \\ G_1 &= x, \\ G_2 &= x^2 + 1, \\ G_3 &= x^3 + 2x, \\ G_4 &= x^4 + 3x^2 + 1. \end{aligned}$$

The Euler basis is defined by

$$\frac{2e^{xt}}{e^t + 1} = \sum_{n \geq 0} E_n(x) \frac{t^n}{n!}.$$

The required prefix is

$$\begin{aligned} E_0 &= 1, \\ E_1 &= x - \frac{1}{2}, \\ E_2 &= x^2 - x, \\ E_3 &= x^3 - \frac{3}{2}x^2 + \frac{1}{4}, \\ E_4 &= x^4 - 2x^3 + x. \end{aligned}$$

These coefficient lists are the type witnesses. The words “Fibonacci” and “Euler” are only display labels.

20.10.2 Input in Fibonacci coordinates

Take the degree-three Fibonacci basis element

$$p(x) = G_3(x) = x^3 + 2x.$$

In the ordered basis (G_0, G_1, G_2, G_3), its coordinate vector is

$$v_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}_{F_3}.$$

Here the subscript F_3 means the Fibonacci coordinate space through degree three. It does not mean the standard Fibonacci polynomial $F_3(x)$.

20.10.3 Basepoint antiderivative in the Fibonacci basis

Let $J_{0,F}^{(4)} : \mathcal{P}_3^F \to \mathcal{P}_4^F$ be the primitive that vanishes at zero. Its exact matrix is

$$J_{0,F}^{(4)} = \begin{bmatrix} 0 & -\frac{1}{2} & 0 & -\frac{1}{2} \\ 1 & 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{4} \\ 0 & 0 & \frac{1}{3} & 0 \\ 0 & 0 & 0 & \frac{1}{4} \end{bmatrix}.$$

Applying it gives

$$v_1 = J_{0,F}^{(4)} v_0 = \begin{bmatrix} -\frac{1}{2} \\ 0 \\ \frac{1}{4} \\ 0 \\ \frac{1}{4} \end{bmatrix}_{F_4}.$$

Reconstruction confirms

$$-\frac{1}{2}G_0 + \frac{1}{4}G_2 + \frac{1}{4}G_4 = \frac{1}{4}x^4 + x^2.$$

This is exactly the primitive of $x^3 + 2x$ with value zero at the basepoint.

20.10.4 Change from Fibonacci to Euler coordinates

The exact degree-four coordinate change is

$$C_{F_4\to E_4} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{3}{2} & \frac{3}{2} & 3 \\ 0 & 1 & 1 & \frac{7}{2} & 5 \\ 0 & 0 & 1 & \frac{3}{2} & 6 \\ 0 & 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Therefore

$$v_2 = C_{F_4\to E_4} v_1 = \begin{bmatrix} \frac{5}{8} \\ \frac{3}{2} \\ \frac{7}{4} \\ \frac{1}{2} \\ \frac{1}{4} \end{bmatrix}_{E_4}.$$

The represented polynomial has not changed. The vector v_2 still reconstructs to $x^4/4+x^2$.

20.10.5 Differentiate in Euler coordinates

Euler polynomials are an Appell family, so the tight derivative has the simple matrix

$$D_E^{(4)} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{bmatrix}.$$

It maps the degree-four space to the degree-three space. The next vector is

$$v_3 = D_E^{(4)} v_2 = \begin{bmatrix} \frac{3}{2} \\ \frac{7}{2} \\ \frac{3}{2} \\ 1 \end{bmatrix}_{E_3}.$$

Indeed,

$$\frac{3}{2}E_0 + \frac{7}{2}E_1 + \frac{3}{2}E_2 + E_3 = x^3 + 2x.$$

20.10.6 Return to Fibonacci coordinates

The degree-three change back to Fibonacci coordinates is

$$C_{E_3\to F_3}=\begin{bmatrix}1 & -\frac{1}{2} & -1 & \frac{7}{4}\\ 0 & 1 & -1 & -2\\ 0 & 0 & 1 & -\frac{3}{2}\\ 0 & 0 & 0 & 1\end{bmatrix}.$$

It returns the original vector exactly:

$$v_4=C_{E_3\to F_3}v_3=\begin{bmatrix}0\\0\\0\\1\end{bmatrix}_{F_3}=v_0.$$

No tolerance or floating comparison appears in this calculation.

20.10.7 The compiled operator is exactly I_4

The whole map on $\mathcal{P}_3^F$ is

$$\mathcal{O}=C_{E_3\to F_3}D_E^{(4)}C_{F_4\to E_4}J_{0,F}^{(4)}.$$

Exact matrix multiplication gives

$$\mathcal{O}=\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}=I_4.$$

This is the concrete naturality form of $DJ_0=I$. The coordinate changes do not alter the represented operator.

20.10.8 Integrate the recovered polynomial

The exact integration covector on $[0,1]$ in the Fibonacci degree-three basis is

$$\ell_{F_3,[0,1]} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{4}{3} & \frac{5}{4} \end{bmatrix}.$$

The entries are the integrals of G_0, G_1, G_2, G_3. Hence

$$\ell_{F_3,[0,1]} v_4 = \frac{5}{4}.$$

Directly,

$$\int_0^1 (x^3 + 2x)\,dx = \frac{1}{4} + 1 = \boxed{\frac{5}{4}}.$$

20.10.9 Affine and weighted integration

Let $\mu = dx$ on $[0,1]$, let $y = 2x + 1$, and let $\nu = \phi_{\#}\mu$. Then

$$d\nu(y) = \frac{1}{2}\,dy \quad \text{on } [1,3].$$

For

$$f(y) = y^4 - 2y + 1,$$

exact affine substitution gives

$$f(2x+1) = 16x^4 + 32x^3 + 24x^2 + 4x.$$

The monomial coefficient vector is therefore

$$c_x = \begin{bmatrix} 0 & 4 & 24 & 32 & 16 \end{bmatrix}^{\mathsf{T}}.$$

Pairing with the $[0,1]$ moment row gives

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix} c_x = \frac{106}{5}.$$

The direct weighted moment row on $[1,3]$ is

$$\ell_{[1,3],1/2} = \begin{bmatrix} 1 & 2 & \frac{13}{3} & 10 & \frac{121}{5} \end{bmatrix}.$$

With $c_y = (1, -2, 0, 0, 1)^{\mathsf{T}}$,

$$\ell_{[1,3],1/2} c_y = \frac{106}{5}.$$

Thus the affine pullback and pushforward-density paths agree exactly.

20.10.10 Guarded failure cases

Same label, different basis

Consider

$$B = (1, x, x^2), \qquad B' = (1, x, x^2 + x).$$

They may both be displayed as `custom basis`, and both have dimension three, but their exact witnesses differ. Their structural fingerprints differ, so a coordinate map certified for B cannot be reused for B'.

The false reverse identity

The equation $DJ_0 = I$ is valid on the smaller source space. The tempting reverse equation $J_0 D = I$ is false. Apply it to the constant polynomial:

$$J_0 D(1) = J_0(0) = 0 \neq 1.$$

The correct formula is

$$J_0 D = I - u e_0.$$

The rank-one term removes the constant value at the basepoint.

Omitted pushforward density

If the factor $1/2$ is omitted from the integral on $[1, 3]$, every moment doubles and the result becomes

$$\int_1^3 (y^4 - 2y + 1)\, dy = \frac{212}{5}.$$

This is not another representation of $106/5$. It is a different measure. The density is therefore part of the integration primitive's exact structural data.

Singular inverse

The matrix

$$S = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

has determinant zero. An exact inverse does not exist, regardless of its label or square shape. The basis constructor's graded leading-coefficient condition prevents such a matrix from becoming a valid polynomial basis witness, and the exact inverse routine rejects it. No cancellation rewrite may be certified from this matrix.

20.10.11 Independent replay

The standalone verifier implements the basis formulas and all matrix arithmetic locally with `Fraction`. It imports neither the expression engine nor the existing polynomial-family implementation.

POWERSHELL

```
python verification/verify_expression_math.py
```

Machine-readable output can be written with

POWERSHELL

```
python verification/verify_expression_math.py --quiet --output
    ↪ qualification/expression-math.json
```

Its 15 independent checks verify the exact basis and integration identities used in this example. Distribution and platform validation are described in the release process (§ E).

21 Finite carrier completion

`komorebi_math.deformation.finite_frobenius_separator` works with the supplied finite trace table. If its selected trace matrix has full column rank, the constituent multiplicity vector is uniquely recoverable from those rows. Rank-deficient data do not separate every declared constituent.

PYTHON | Executable example

```python
from komorebi_math.deformation import finite_frobenius_separator

separator = finite_frobenius_separator({"place_a": (1, 0), "place_b": (0, 1)})
separator.certificate.require()
assert separator.recover((2, 3)) == (2, 3)
```

21.1 Certificate contract

The Boolean `separating`, certificate validity, and exact column-rank condition agree:

$$\text{valid} \iff \text{separating} \iff \operatorname{rank}(A) = r,$$

where `A` is the supplied finite trace matrix and `r` is the constituent count. A rank-deficient result has an invalid certificate. Both `certificate.require()` and `recover(...)` reject that result.

The certificate has `Exactness.EXACT` because rank and recovery use exact finite algebra. Its checked assumptions concern the supplied finite table and exact column rank.

21.2 Arithmetic scope

A global Galois or automorphic identification requires hypotheses beyond the finite trace table. The unchecked assumption `Chebotarev density beyond the supplied table` is recorded explicitly. Any global conclusion requires the relevant external arithmetic hypotheses.

PART V

Models, portable results and complete programs

Finite quantum models provide another setting for operators and packets. Persistence and export make calculations usable beyond one interpreter, while complete programs show how the mathematical checks fit together.

22 Quantum and memory models

Quantum elimination and numerical span calculations depend on explicit sign, transform and rank conventions. This chapter states those conventions and explains how trajectory comparisons, finite integrals and changes of generators test the represented finite models.

22.1 Schrödinger memory convention

`QuantumFeshbachReduction.self_energy(z)` retains the Hamiltonian convention

$$\Sigma(z) = H_{PQ}(zI - H_{QQ})^{-1}H_{QP}.$$

`memory_kernel()` returns the kernel for the derivative equation obtained from `psi' = -i H psi`. With zero hidden initial state,

$$\psi_P'(t) = -iH_{PP}\psi_P(t) + \int_0^t K(t-s)\psi_P(s)\,ds, \qquad K(t) = -H_{PQ}e^{-itH_{QQ}}H_{QP}.$$

Both coupling maps in the returned `MemoryKernel` carry `-i`. The kernel's `RationalMemoryRealization` therefore reconstructs the full Schrödinger trajectory after the retained and hidden coordinates are ordered consistently. In the open upper half plane, the convention is `kernel.laplace(-1j*z) = -1j*self_energy(z)`.

For `H=[[0,1],[1,0]]`, the retained amplitude from initial state `[1,0]` is `cos(t)`. A positive derivative kernel would instead produce `cosh(t)`. The regression checks the direct matrix exponential and the analytic cosine.

22.2 Infinite and finite Laplace transforms

`MemoryKernel.laplace(z)` requires the sufficient half plane

$$\operatorname{Re} z > \max \operatorname{Re}\operatorname{spec}(D).$$

This domain checks the full hidden realization. It is conservative when unstable hidden modes cancel from `B exp(tD) C`. The eigenvalue and residual checks use floating point arithmetic and retain numerical certificate status.

`rational_transform(z)` separately returns `B solve(zI-D,C)` whenever the resolvent exists. It makes no claim that the infinite Laplace integral converges. For the kernel `exp(t)`, its value at zero is `-1` as an algebraic continuation, while `laplace(0)` rejects the divergent integral and `certificate(0)` is invalid.

`finite_laplace(z,horizon)` evaluates the finite integral by the upper right block of an augmented matrix exponential. It does not invert `zI-D`, so it also works at a resolvent pole. For `K(t)=exp(t)`, `z=1` and horizon `2`, the integral is `2`. For the corresponding size two Jordan kernel `K(t)=t exp(t)`, it is `2` as well. The finite integral remains a numerical result, with no rigorous floating point enclosure implied.

22.3 Constraint span closure

`constraint_closure` preserves its coefficients, residual matrix and maximum residual in the declared generator coordinates. Its validity decision uses the spectral norm of the quotient commutator map in a Hilbert Schmidt orthonormal basis of the resolved span. A unitary change of orthonormal basis acts unitarily on the input tensor coordinates, so this norm is invariant.

Multiplying the two Pauli generators `X,Z` by `1e-6` does not close their span. Its raw commutators shrink quadratically with the generator scale, while the resolved span is unchanged. The span obstruction rejects this example over the tested uniform scales from `1e-100` through `1e100`.

The SVD rank threshold and singular spectrum are certificate diagnostics. Extremely ill conditioned generating families remain a finite precision rank boundary. Numerical span resolution does not certify an exact symbolic rank.

22.4 Spectral dimension

`spectral_dimension` cancels the common exponential factor associated with the smallest eigenvalue before computing the heat trace ratio. It therefore returns `2000` for the scalar identity at time `1000`, even though the heat trace itself underflows.

For a finite nonnegative operator the limit at time zero is zero. The large time limit is zero when a zero eigenvalue is present. A strictly positive operator instead has asymptotic spectral dimension `2*t*lambda_min`.

22.5 Mutable inputs and finite validation

Lindblad generators and memory kernels snapshot their constructor matrices. Caller mutation of the original arrays does not change the represented model. The snapshots are read only. Lindblad operations also recheck the declared form and reject reassigned nonfinite Hamiltonians, jumps or computed outputs before they can produce a positive certificate.

Shared quantum Hermitian, unitary, projector and density validation rejects nonfinite or negative tolerances. These are numerical tolerance decisions, without an exact theorem claim about unrounded real inputs.

22.6 Dirac evolution in a fixed mode

The `komorebi_math.quantum.dirac` namespace implements four-spinor Clifford coordinate actions and free Dirac evolution for one explicit Fourier mode. The model retains the chosen momentum, mass and time as finite numerical data.

PYTHON | Executable example

```python
from komorebi_math.quantum.dirac import (
    central_mode_momentum, dirac_symbol, dirac_propagator,
)

q = central_mode_momentum([1, 0, 0], mesh=1 / 12)
H = dirac_symbol(q, mass=1.0)
U = dirac_propagator(q, time=0.4, mass=1.0)
```

The units are `c = hbar = 1`. The torus is `(R/Z)^3` with Fourier phase `exp(2*pi*i*n.x)`, so continuum physical momentum is `2*pi*n`. The spinor coordinates have the source order `e0, e1,e2,e3`.

`dirac_matrices()` returns independent read-only snapshots of `alpha_x,alpha_y,alpha_z, beta` in the declared spinor coordinates. These are numerical Python matrices. Their evaluation does not provide a Lean proof certificate.

`central_mode_momentum(mode,mesh)` accepts a strict integer triple. It computes `p*sinc(mesh* p/pi)` for `p=2*pi*mode`, with the continuum value at zero mesh. This formula handles the removable zero-mesh limit and small meshes. Away from that limit, the implementation evaluates `sin(mesh*p)/mesh` directly to avoid changing a large represented phase through a divide/multiply roundtrip. It still operates on the supplied floating-point phase, so it cannot recover phase information already lost in input conversion or multiplication. The central symbol is periodic in momentum; distinct Fourier labels can have the same central symbol. There is no Nyquist restriction or claim of recovering labels from that symbol. No sampled finite-grid Hilbert space is constructed.

`dirac_symbol(momentum,mass)` takes a real physical momentum triple and returns `H = sum q_ j*alpha_j + mass*beta`.

`dirac_propagator(momentum,time,mass)` uses the scalar-square identity `H^2 = E^2 I`, with `E = hypot(mass,q_x,q_y,q_z)`, to evaluate `cos(time*E) I - i*sin(time*E) H/E`. At zero energy it returns the identity. Normalizing the real coefficients before constructing `H/E` handles tiny positive energies without unstable complex division. Real negative mass, mesh and time are allowed. Shapes, scalar reality, strict mode labels and finite values are validated. Unrepresentable energy or phase is rejected.

These are numerical finite-matrix results. They do not certify infinite Fourier support, an unbounded Dirac domain, growing cutoffs, sampled-grid norms, noisy parameters or a uniform continuum-limit theorem. Those require separate mathematical hypotheses and proofs.

The dedicated tests verify all four Clifford involutions and their anticommutators, coordinate actions, Hermiticity, scalar-square identities and unitarity. Seventy-two varied momentum/mass/time cases compare the closed formula to an independent eigensystem exponential. Further controls cover zero and tiny energy, zero and tiny mesh, group composition, caller mutation, strict integer/boolean guards, invalid shapes and finite-range rejection.

23 Portable persistence

23.1 Purpose

Presentation export converts data into readable reports. Portable persistence reconstructs supported objects through an explicit schema and codec registry. Adjacent schema migrations preserve supported older records, while chunked streams handle large finite packet-jet snapshots.

Use `to_serializable` for presentation-oriented export. Use `dumps_portable` when a supported object must be reconstructed as the same declared mathematical record.

23.2 Envelope

A portable document contains:

JSON

```
{
  "schema": "komorebi-math.portable",
  "schema_version": 1,
  "payload": {},
  "payload_sha256": "..."
}
```

The digest is computed over canonical JSON for `payload`. Object keys are sorted, separators are fixed, non-ASCII text is retained, and non-finite JSON numbers are rejected.

The digest detects accidental or unkeyed payload modification. It is not a digital signature and does not establish authorship, authorization, confidentiality, or resistance to an attacker who can replace both payload and digest.

23.3 Codec records

Objects registered in the codec table use:

JSON

```
{
  "__komorebi_type__": "nonlinear.packet_jet_snapshot",
  "__version__": 2,
  "data": {}
}
```

Decoding accepts only a registered tag. Current versions decode directly. Older versions must traverse a complete registered chain of adjacent migrations from `n` to `n + 1`; missing steps and future versions fail closed. The decoder never resolves Python import paths from the input.

23.4 Built-in scalar and container forms

The encoder supports:

- `None`, booleans, integers, strings, and finite floats
- exact fractions
- finite complex values
- NumPy scalar values that convert to supported Python scalar types
- NumPy arrays with shape and dtype metadata
- tuples, lists, sets, frozensets, and mappings
- bytes and `pathlib.Path` values
- explicitly registered portable codecs

Mappings are represented by sorted key-value pairs so heterogeneous key types remain deterministic.

Object arrays are recursively encoded, preserving nested lists, tuples, and arrays as individual cells. Numeric arrays use finite scalar data and restore the declared shape. Structured and subarray dtypes are unsupported. The decoder validates dimensions and recursion depth before reconstruction.

Integer array payloads must contain genuine integers within the declared dtype's range; fractional truncation and integer overflow are rejected. Floating and complex conversions must remain finite. Extended NumPy scalar precision that cannot convert to a supported Python scalar is rejected explicitly, rather than rounded silently or serialized recursively.

23.5 Built-in mathematical codecs

The built-in registry includes codecs for:

- `Certificate`
- `OrderedRootedTree`
- `PacketJetAddress`
- `PacketJetCoefficient`
- `PacketJetSnapshot`
- `SparseSymmetricTensor`
- `LiftBranch`
- `LiftTreeNode`
- `FiniteLiftTree`
- `SpectralSequenceEntry`
- `SpectralSequencePage`
- `FilteredSpectralSequenceResult`

The packet-jet snapshot codec is version 2. Its provider field is normalized and coefficient ordering is explicit. Version 1 remains readable through the registered deterministic migration. A changed representation must receive a new version and an explicit migration path. Silent reinterpretation of old payloads is prohibited.

23.5.1 Certificate lifecycle after loading

`core.certificate` uses codec version 2. Version 1 remains readable through an explicit adjacent migration. A portable certificate is a historical record of a producer's earlier check, not a command to rerun that check. On load:

- `recorded_valid` preserves the Boolean recorded by the producer.
- `valid` is false.
- `revalidation_required` is true.
- `require()` fails until the producer is actually rerun and produces a fresh in-memory certificate.

The payload digest establishes integrity of the stored record relative to its envelope. It does not establish that mutable external inputs, software assumptions, external theorems, or producer logic still justify the recorded mathematical conclusion. Fresh integrity checks may themselves return new live certificates, but those certificates cover only the integrity properties that the checks actually rerun.

The version-2 certificate decoder also rejects unknown decoded fields, nonboolean recorded validity, nonfinite or nonnumeric residuals, nonfinite/negative tolerance, non-string assumptions, and malformed theorem/message fields rather than coercing them.

23.6 Codec migrations

PYTHON | Executable example

```
from komorebi_math.export import register_portable_migration

register_portable_migration(
    tag="my_project.record",
    from_version=1,
    to_version=2,
    migrator=lambda data: {**data, "schema_name": "record-v2"},
)
```

A migration advances exactly one version. The persisted payload selects only a registered type tag and declared version; it cannot provide code or an import path. Migration functions are trusted application code and must validate their own schema.

23.7 Packet-jet stream format

`capture_packet_jet_stream` and `save_packet_jet_snapshot_stream` use a directory format rather than the single-document portable envelope. Each chunk and the manifest have independent canonical-JSON SHA-256 envelopes. Incomplete direct captures remain at `<destination>.partial` until final validation and atomic promotion.

The stream reader can decode one chunk at a time, but the complete basis count remains exponential in derivative order. See Resumable packet-jet streams (§ 18) for the format, recovery procedure, partial-evaluation semantics, and integrity boundary.

23.8 File writes

`save_portable` writes and flushes a temporary file in the destination directory, then publishes the complete file atomically. With the default `overwrite=False`, publication rejects an existing destination even when another writer creates it concurrently. This uses a no-replacement rename on Windows and a hard link on POSIX, so the POSIX destination filesystem must support hard links. With `overwrite=True`, publication atomically replaces the destination. Temporary files are cleaned after failure where the platform permits.

23.9 Custom codecs

PYTHON | Executable example

```
from dataclasses import dataclass
from komorebi_math.export import register_portable_codec

@dataclass(frozen=True)
class Record:
    value: int

register_portable_codec(
    Record,
    tag="my_project.record",
    version=1,
    encoder=lambda value: {"value": value.value},
    decoder=lambda data: Record(int(data["value"])),
)
```

Codec decoders are trusted application code. Register a codec only for a type whose schema, validation, and versioning policy are controlled by the caller. `replace=True` is intended for tests and deliberate migrations, not routine import-time replacement.

23.10 Safe marker decoding

`from_serializable` reverses only the built-in presentation markers for fractions, complex numbers, and NumPy arrays. Unknown `__type__` dictionaries remain ordinary dictionaries. This prevents the broad export decoder from becoming a class-instantiation mechanism.

24 Export and LaTeX

The export package turns Python values, NumPy arrays, dataclasses, exact fractions, certificates and computation results into portable data or renderer-independent mathematical documents.

24.1 Supported formats

OUTPUT

```
json
yaml
csv
tsv
text
markdown
html
latex
pdf
npy
npz
```

YAML requires the optional `yaml` extra. PDF export requires a local LaTeX engine. The package detects `lualatex`, `xelatex`, `pdflatex`, and `latexmk`.

24.2 Export ordinary data

PYTHON | Executable example

```python
from fractions import Fraction
from komorebi_math.export import ExportOptions, export_data

values = [
    {"index": 0, "value": Fraction(1, 1)},
    {"index": 1, "value": Fraction(1, 2)},
]

result = export_data(
    values,
    "values.json",
    options=ExportOptions(overwrite=True),
)
result.certificate.require()
```

The same object can be exported to CSV, Markdown, HTML, LaTeX, or PDF by changing the suffix.

24.3 Build a mathematical document

PYTHON | Executable example

```
from komorebi_math.export import DocumentMetadata, ExportOptions, MathDocument,
    ↪ export_data

document = MathDocument(
    DocumentMetadata(
        title="Exact finite calculation",
        author="Komorebi P.",
        abstract="A report generated from executable mathematics.",
    )
)
document.paragraph("Ordinary text is escaped by each renderer.")
document.equation(r"E_n = \sum_{r=0}^{n/2} c_r A_{n-2r}", numbered=True)
document.table(("n", "value"), ((0, 1), (2, -1), (4, 5)))

export_data(document, "report.tex",
    ↪ options=ExportOptions(overwrite=True)).certificate.require()
```

To compile the document, pass `"report.pdf"` to `export_data` with a local LaTeX engine installed. PDF export writes its generated `.tex` file beside the PDF by default. Set `keep_tex=False` to remove it after successful compilation.

24.4 Compile an existing LaTeX source

PYTHON | Contextual fragment

```
from komorebi_math.export import LatexCompileOptions, compile_latex

result = compile_latex(
    "paper.tex",
    "build",
    options=LatexCompileOptions(
        engine="pdflatex",
        passes=2,
        keep_auxiliary=False,
    ),
)
pdf = result.require()
```

Shell escape is disabled by default. Compilation has an explicit timeout and writes a `.komorebi.log` file.

24.5 Bundle several formats

PYTHON | Executable example

```python
from komorebi_math.export import ExportOptions, export_bundle

bundle = export_bundle(
    {"values": [1, 1, 2, 3, 5, 8]},
    "fibonacci_bundle.zip",
    formats=("json", "markdown", "latex"),
    options=ExportOptions(overwrite=True),
)
bundle.certificate.require()
```

The ZIP includes a SHA256 manifest. Add `"pdf"` to the formats when a local LaTeX engine is available.

24.6 Custom document class

PYTHON | Executable example

```python
from komorebi_math.export import ExportOptions, LatexRenderOptions

options = ExportOptions(
    overwrite=True,
    latex=LatexRenderOptions(
        document_class="report",
        include_table_of_contents=True,
    ),
)
```

A custom class such as `aomart` can be supplied when it is installed in the local TeX distribution.

24.7 Safety boundary

Ordinary text is escaped. `Equation` stores trusted mathematical LaTeX. `RawLatex` is an explicit escape hatch for trusted source only. Shell escape remains disabled unless a Python caller deliberately enables it for a direct engine.

24.8 PowerShell workflows

Invoke the installed package through the selected Python interpreter:

POWERSHELL

```
python -m komorebi_math export sequence fibonacci --terms 30 --compile --output
    ↪ .\out\fibonacci.json --force
```

The PowerShell guide (§ C.8) covers clean installation, exported reports and local LaTeX compilation.

24.9 Reproducibility boundary

ZIP member ordering, ZIP timestamps, and the bundle manifest are normalized. JSON, CSV, Markdown, HTML, LaTeX, NPY, and NPZ content is deterministic for deterministic inputs and options. PDF bytes can still vary with the selected TeX engine, TeX distribution, fonts, and compiler version. The manifest records the exact PDF that was produced rather than claiming byte identity across installations.

25 Behavioral fixtures and profiling

Behavioral fixtures and runtime profiles answer different questions. A fixture checks whether a fixed finite case still produces its declared result. A profile records the time and memory observed for a specified workload. Keeping those roles distinct makes both records easier to interpret.

25.1 Closed behavioral fixtures

`run_behavioral_golden_fixture` executes a fixed registry of case kinds. A fixture cannot name a module, class, or callable to import. Unknown case kinds are rejected.

The closed evaluator keeps regression fixtures within a fixed set of supported operations. Inputs and expected values are explicit JSON data. Exact rational values use canonical numerator and denominator strings.

PYTHON | Executable example

```
from komorebi_math.golden import run_behavioral_golden_fixture

report = run_behavioral_golden_fixture()
report.certificate.require()
```

The report records the fixture path, fixture digest, observed values, expected values, and a certificate for every case.

Changing an expected value is not a routine test update. It is a semantic compatibility event and should be reviewed with the corresponding implementation and mathematical contract.

25.2 Exact structural complexity counts

`estimate_packet_jet_complexity` reports deterministic counts for one finite family and support record:

- total family addresses
- active and pruned addresses
- naive and planned provider calls
- projected input vectors
- output projections
- dense standard-basis evaluations
- dense tensor elements
- derivative-call reduction ratio

These values are operation counts derived from the selected finite structure. They are not elapsed-time predictions.

PYTHON | Contextual fragment

```
support = packet_jet_support(family)
estimate = estimate_packet_jet_complexity(family, support)
estimate.certificate.require()
```

25.3 Observational runtime profiles

`profile_operation` and `profile_packet_jet_runtime` record local observations using `perf_counter_ns` and `tracemalloc`.

PYTHON | Contextual fragment

```
profile = profile_packet_jet_runtime(
    family,
    directions,
    repetitions=5,
    include_snapshot=True,
    include_serialization=True,
)
```

A profile includes:

- Python and platform metadata
- operation scope
- repetition and warm-up counts
- minimum, median, and maximum duration
- maximum traced peak bytes
- exact structural counts beside measured stages

The timing is valid only for the recorded process and workload. It does not establish a portable speedup, a backend comparison, or a hardware-independent complexity theorem. `tracemalloc` observes Python-managed allocations and does not measure every native, framework, or device allocation.

25.4 Device validation boundary

Run `python -m komorebi_math verify gpu` to check the optional installed CUDA paths on a compatible host. The release guide (§ E) describes how to bind hardware results to exact distribution files. A local timing profile does not substitute for device qualification.

26 Extensions and companion C integration

Separately distributed packages can register themselves through the entry point group:

TOML

```toml
[project.entry-points."komorebi_math.extensions"]
my_extension = "komorebi_my_extension:extension"
```

The entry point must provide an `ExtensionSpec` or a callable that takes no arguments and returns one.

PYTHON | Executable example

```python
from komorebi_math.extensions import ExtensionSpec


def register() -> None:
    pass


extension = ExtensionSpec(
    name="komorebi-example",
    version="0.1.0",
    api_version="1",
    modules=("komorebi_example",),
    optional_dependencies=(),
    description="Example extension",
    register=register,
)
```

The diagnostic command

POWERSHELL

```powershell
komorebi doctor
```

reports extension-loading failures without preventing the core package from importing.

Extensions must not silently change the meaning of a core object. New backends and feature records must be registered under distinct names. Conditional mathematical constructions must return conditional certificates with their unmet assumptions.

26.1 Companion C integration

The Companion C visual atlas supplies finite decompositions and independent numerical controls for the Python library. Each adopted construction needs a package-native convention, typed interface and explicit certificate scope.

26.1.1 Shared mathematical structure

The shared operator framing is

$$P_\alpha = U^{-1} E_\alpha U, \qquad x = \sum_\alpha P_\alpha x, \qquad T = \sum_{\alpha,\beta} P_\beta T P_\alpha,$$

and the general analysis-synthesis obligation

$$SA = I$$

on the declared object or model space.

This framing is reflected in the stable semantic contract for complete decomposition, polyphase refinement, reconstruction, and certificates.

26.1.2 Cross-checks relevant to the current package

The Companion C numerical ledger includes finite checks for:

- packet synthesis
- operator-block synthesis
- polyphase synthesis
- two-to-four refinement
- mixed-radix packet synthesis
- dyadic and ternary residue refinement
- perfect-reconstruction filter banks
- Fourier and wavelet coordinate synthesis
- coordinate transport
- CRT packet synthesis
- overlap-add synthesis
- quantum and modal synthesis

The corresponding polyphase, mixed-radix, lattice, persistence and reconstruction identities are represented in package-native behavioral fixtures.

26.1.3 Scope of the stable interface

The following Companion C areas should remain outside the stable API until separately reviewed:

- standalone plotting and page-layout helpers
- duplicate Fourier, wavelet, DCT, DST, Walsh, Gabor, and sampling implementations
- visual-only diagnostics without package certificates
- PDE, multigrid, Krylov, and low-rank examples that do not yet map to one stable interface
- heuristic comparison plots and model-specific RMSE claims

26.1.4 Admission to the stable API

A Companion C construction may enter the stable package only after it has:

1. a package-native mathematical convention
2. a typed public interface
3. exactness or numerical certificate semantics
4. deterministic regression tests
5. a persistence decision
6. API tier classification
7. documentation that states its finite scope

The stable API admits a construction only after its mathematical and software contracts have been reviewed.

27 Worked calculations and independent controls

This chapter develops six mathematical workflows through twelve complete programs. Each section introduces the required imports, admissible inputs and result, then compares a small calculation with an independent control. The programs can be executed as printed and adapted to other explicit finite inputs.

Use the explicit module paths shown below to make each mathematical interface clear.

Scope	Named exports covered here	Arithmetic and dependencies
`komorebi_math.quadratic_forms`	12: seven operations and five result types	Directed intervals; qualified `mpmath==1.3.0`
`komorebi_math.sequences`	15: twelve functions and three record types	Exact Python integers; offline local data
`komorebi_math.special_functions`	18 functions	Exact rational scalars or numerical mpmath scalars
`komorebi_math.signals.fourier_recovery`	5: three functions and two result types	NumPy numerical reconstruction
`komorebi_math.core.semilattice`	2 types	Exact finite table checks
`komorebi_math.quantum.dirac`	4 functions	NumPy finite matrices

The numerical examples and the directed backend need the scientific extra. From a source checkout, install and run them with:

SHELL

```
python -m pip install ".[scientific]" "mpmath==1.3.0"
python examples/worked_examples.py
```

This command installs the local source. The complete script `examples/worked_examples.py` contains the twelve example bodies printed below. Run without Python's `-O` flag so their assertions remain active. A successful run reports `12 worked cases passed`; any failed assertion stops the run with a traceback.

27.1 Positive quadratic-form tails

Use `komorebi_math.quadratic_forms` for the upper tail of

$$Q = \sum_{j=1}^{r} \lambda_j Z_j^2, \qquad \lambda_j > 0,$$

where the Z_j are independent standard Gaussian variables. The implemented COS remainder bound requires $r > 2$. Automatic tail planning additionally requires a threshold strictly above the exact sum of the supplied decimal coefficients. Signed coefficients and noncentral Gaussian laws are outside this API.

Export	Return and purpose
`certified_positive_logsf`	`AccuracyDrivenTailResult`: refine an above-mean query to a requested multiplicative width
`design_positive_tail_parameters`	Dictionary of admissible tilt and support design values
`interval_positive_logsf`	`IntervalLogTailResult`: enclose a specified finite transform and its analytic error bounds
`even_df_scaled_chi_square_logsf_interval`	Dictionary containing an independent finite Erlang-tail enclosure
`directed_relative_accuracy`	`DirectedAccuracy`: check a width request from outward literal endpoints
`certify_float64_factor_spectrum`	`VerifiedSpectrum`: enclose a stored real binary64 Gram spectrum
`certified_float64_factor_logsf`	`MatrixTailCertificate`: combine spectrum and tail enclosures

The five result types in the return column are also named exports. Importing the namespace leaves mpmath unloaded. Requesting a directed operation or one of these backend result types loads the qualified arithmetic implementation. A missing backend or a version other than `mpmath==1.3.0` raises `BackendUnavailableError` with an installation instruction.

Supply strings or `Decimal` objects to preserve exact decimal intent. A float contributes its `str(value)` decimal; this scalar-input convention differs from the stored-factor convention below. Spectrum iterables, including generators, are materialized once before refinement.

For a result with `log10_interval="[l,u]"`, the authoritative bounds are the outward decimal literals l, u. Under the arithmetic trust assumptions, `certified=True` proves

$$10^{u-l} - 1 = \frac{p_{\text{upper}}}{p_{\text{lower}}} - 1 \le \texttt{relative_accuracy}.$$

Midpoints and floating display summaries do not carry this enclosure and may round inward. Pass the unchanged literal interval to `directed_relative_accuracy`; it rejects interval

objects whose ordinary string conversion might collapse their width. When supplying your own interval literal, you are responsible for its outward enclosure of the intended value.

27.1.1 A tail with an independent Erlang control

Here the exact distribution is $Q = \frac{1}{4}\chi^2_{32}$. The example computes the finite Erlang survival formula separately and checks that its high-precision value lies between the returned literal endpoints. This numerical control is independent of the COS transform, while still depending on mpmath arithmetic.

PYTHON | Executable example

```
from decimal import Decimal
from fractions import Fraction
import mpmath as mp
from komorebi_math.quadratic_forms import certified_positive_logsf

tail = certified_positive_logsf(
    "30", ("0.25" for _ in range(32)), relative_accuracy="1e-9",
    initial_K=64, max_K=512, initial_L="10", max_L=14,
    initial_interval_dps=40, max_interval_dps=80, working_dps=120,
)
assert tail.certified, tail.failure_reason
assert Decimal(tail.achieved_relative_ratio_minus_one) <= Decimal("1e-9")
lower, upper = map(Decimal, tail.log10_interval[1:-1].split(","))

# Q = 0.25 * chi-square(32): its survival is exp(-60)*sum(60**k/k!).
term = total = Fraction(1)
for k in range(1, 16):
    term *= Fraction(60, k)
    total += term
ctx = mp.mp.clone()
ctx.dps = 110
reference = (-60 + ctx.log(ctx.mpf(total.numerator) / total.denominator))
reference = Decimal(ctx.nstr(reference / ctx.log(10), 100))
assert lower <= reference <= upper
```

27.1.2 A stored-factor spectrum and tail

The matrix bridge converts a real input factor A to binary64, then treats those stored entries as exact dyadic numbers. It encloses the eigenvalues of AA^{T}, using an eigensolver only to obtain candidates and directed residual bounds to validate them. Uncertainty in the matrix before conversion is outside this contract.

This example uses $A = I_8$, so every eigenvalue is exactly one and the tail is that of χ^2_8. Both controls are known independently of the candidate solver. `VerifiedSpectrum.rank` counts all coefficient intervals, including intervals for zero eigenvalues; it is not an algebraic matrix-rank report.

PYTHON | Executable example

```python
from decimal import Decimal
import mpmath as mp
import numpy as np
from komorebi_math.quadratic_forms import certified_float64_factor_logsf

result = certified_float64_factor_logsf(
    "20", np.eye(8), relative_accuracy="1e-3",
    initial_candidate_digits=25, max_candidate_digits=25, interval_dps=40,
    tail_accuracy_fraction="0.5",
    tail_kwargs=dict(initial_K=128, max_K=512, initial_L="10", max_L=12,
                     initial_interval_dps=40, max_interval_dps=60,
                     working_dps=100),
)
assert result.certified, result.failure_reason
spectrum = result.spectral_enclosure
assert spectrum.rank == 8
for lower, upper in zip(spectrum.lower, spectrum.upper, strict=True):
    assert Decimal(lower) <= 1 <= Decimal(upper)
# A = I gives chi-square(8): exp(-10)*(1 + 10 + 50 + 1000/6).
ctx = mp.mp.clone()
ctx.dps = 100
reference = (-10 + ctx.log(ctx.mpf(683) / 3)) / ctx.log(10)
lower, upper = map(Decimal, result.log10_interval[1:-1].split(","))
assert lower <= Decimal(ctx.nstr(reference, 90)) <= upper
assert Decimal(result.achieved_relative_ratio_minus_one) <= Decimal("1e-3")
```

The matrix-tail controller requires a threshold above the exact stored trace and at least three strictly positive lower spectral bounds. The spectrum bridge uses a dense full basis and is intended for modest matrices. Unresolved rank or planner-domain cases can raise an error. Valid accuracy budgets that are exhausted return `certified=False` and a `failure_reason`; always inspect those fields before using an enclosure as an accepted accuracy result.

Malformed domains and cap settings are rejected, usually with `ValidationError`; backend admissibility failures can also raise `ValueError`. When exact input domains are valid but planning or interval precision cannot resolve admissible margins within the supplied caps, the automatic controller returns `certified=False` with a failure reason. Matrix comparison-spectrum obstructions trigger bounded candidate refinement. Reported candidate precision is the actual last attempted precision. Direct domain and unresolved-rank errors remain explicit.

Caps restrict each attempted transform and precision, not elapsed time or total work. Public directed calls serialize their entire backend computation and restore point and interval

precision on success or failure. Concurrent external changes to mpmath global contexts remain outside that guarantee.

The enclosure claim is conditional on mpmath 1.3.0 interval arithmetic, exact binary endpoint storage, Python integer arithmetic, and directed Decimal serialization. It is a numerical enclosure claim, distinct from a Lean proof of the Python implementation. See Directed quadratic-form tails (§13) for the full hypotheses, parameterized evaluator, and source provenance.

27.2 Exact sequence families and offline OEIS comparisons

The `komorebi_math.sequences` namespace generates exact integer families and compares finite windows with data supplied explicitly by the caller. These operations require no additional dependency and make no network request.

Export group	Return and supported input
`universal_denominator_coefficients`	Dense integer EGF coefficients through an inclusive nonnegative degree
`reciprocal_sequence`, `reciprocal_coefficients`	Lazy `Sequence`, or exactly `count` compressed integer coefficients
`kbonacci_sequence`, `kbonacci_closed_form`	Lazy `Sequence`, or an integer term; order at least two
`fubini_sequence`, `derangement_sequence`	Lazy integer `Sequence` objects starting at index zero
`OEISRecord`, `LocalOEISCorpus`, `OEISPrefixMatch`	Local record, corpus, and finite-comparison types
`compare_oeis_prefix`, `find_local_oeis_matches`	One comparison, or a tuple of agreeing finite candidates
`parse_oeis_internal`, `parse_oeis_stripped`, `load_local_oeis`	`LocalOEISCorpus` parsed from text or an explicit file

27.2.1 Keep dense and compressed normalization separate

For spacing $N \geq 1$ and integer power $r \geq 0$, the normalization is

$$D_N(t) = \sum_{q \geq 0} \frac{(-1)^q t^{Nq}}{(Nq)!}, \qquad D_N(t)^{-r} = \sum_{n \geq 0} \frac{A_N^{[r]}(n) t^{Nn}}{(Nn)!}.$$

`universal_denominator_coefficients(N, degree)` returns coefficients multiplying $t^j/j!$, with zero entries outside the selected residue lane. Its result has `degree + 1` entries. `reciprocal_coefficients(N, count, power=r)` returns exactly `count` values of $A_N^{[r]}(n)$, starting at $n = 0$. These are compressed exponential generating coefficients, not ordinary Taylor coefficients. Power zero returns the identity sequence.

The k-bonacci convention starts with `k-1` zeros and then one. Fubini numbers count ordered set partitions; derangements start with $D_0 = 1, D_1 = 0$. The example checks familiar initial terms and compares the finite k-bonacci formula with recurrence execution at twelve indices.

PYTHON | Executable example

```
from komorebi_math.sequences import (
    derangement_sequence, fubini_sequence, kbonacci_closed_form,
    kbonacci_sequence, reciprocal_coefficients,
    universal_denominator_coefficients,
)

# These are compressed secant-number coefficients, not Taylor coefficients.
assert reciprocal_coefficients(2, 6) == (1, 1, 5, 61, 1385, 50521)
assert universal_denominator_coefficients(2, 6) == (1, 0, -1, 0, 1, 0, -1)
assert reciprocal_coefficients(3, 4, power=0) == (1, 0, 0, 0)
assert fubini_sequence().first(5) == (1, 1, 3, 13, 75)
assert derangement_sequence().first(6) == (1, 0, 1, 2, 9, 44)

# Check the finite binomial formula against direct recurrence execution.
recurrence = kbonacci_sequence(3).first(12)
assert recurrence[:7] == (0, 0, 1, 1, 2, 4, 7)
assert tuple(kbonacci_closed_form(3, n) for n in range(12)) == recurrence
```

27.2.2 Verify the whole finite indexed window

Local OEIS records retain an explicit starting offset when one is available. Supplying `query_offset` aligns sequence indices. Without it, comparison is positional and does not claim verified index alignment. An unknown reference offset cannot validate a query with a declared offset.

This example writes a small local JSON snapshot, loads it, and checks both a complete indexed match and an exhausted reference. The temporary file is removed after loading; the corpus retains its records and provenance.

PYTHON | Executable example

```
import json
from pathlib import Path
from tempfile import TemporaryDirectory
from komorebi_math.sequences import compare_oeis_prefix, load_local_oeis

# A user-supplied local snapshot; this example makes no network request.
payload = {
    "sequences": [{"id": "A000045", "terms": [1, 1, 2, 3, 5, 8],
                   "offset": 1, "name": "Fibonacci numbers"}],
    "provenance": {"purpose": "manual finite-window example"},
}
with TemporaryDirectory() as temporary:
```

```
    location = Path(temporary) / "snapshot.json"
    location.write_text(json.dumps(payload), encoding="utf-8")
    corpus = load_local_oeis(location, format="json")
reference = corpus.records[0]
matched = compare_oeis_prefix((2, 3, 5), reference,
                              query_offset=3, minimum_terms=3)
assert matched.passed and matched.index_alignment_verified
exhausted = compare_oeis_prefix((5, 8, 13), reference,
                                query_offset=5, minimum_terms=2)
assert exhausted.agrees_on_overlap and not exhausted.passed
assert exhausted.status == "reference-exhausted-prefix"
assert corpus.provenance["purpose"] == "manual finite-window example"
```

`agrees_on_overlap` checks all available compared terms against the requested minimum. `complete_query_match` and its `passed` alias additionally require the entire supplied query to have been compared. Candidate search can return an exhausted agreeing prefix, so inspect `passed` before treating a candidate as a check of the whole query. A known mismatch excludes the candidate even if an earlier prefix agreed.

The loaders accept internal-format, stripped-format, and archive-schema JSON text, including `.gz` files. Internal-format parsing assembles `%S/%T/%U` fields or their signed `%V/%W/%X` counterparts; `%O` supplies the offset. Stripped text has no offset field, so its offsets remain unknown. Internal-format parsing ignores lines that do not match its field syntax, including malformed identifiers; inspect the returned records before comparing them. Record construction, stripped rows, and JSON snapshots validate identifiers. Invalid terms in recognized fields, sizes, and indexing inputs raise `ValidationError`; file access errors retain the usual filesystem exceptions. Booleans and fractional integer coercions are rejected.

No reference corpus is bundled or downloaded. A finite match establishes only the supplied finite comparison. A failed local search cannot establish absence from the full OEIS, originality, or an infinite sequence identity. See Exact families and local OEIS comparisons (§ 10) for the loader schema, data provenance and recurrence conventions.

27.3 Bernoulli packets, Gamma shapes, and terminating scalars

`komorebi_math.special_functions` contains two arithmetic layers. Exact rational scalar functions return `Fraction` values without an optional backend. Numerical functions return mpmath scalars at the requested precision. Shape functions use isolated guard digits to resolve cancellation at large parameters, then round their results. More than 10,000 working decimal digits, or excessive positive window-exponent work, raises `ResourceLimitError`. These outputs retain ordinary numerical precision rather than a directed enclosure.

Importing the namespace does not load mpmath, SciPy, or SymPy.

Export group	Arithmetic and supported object
`bernoulli_packet_integer`	Exact rational Bernoulli polynomial residue packet
`bernoulli_hurwitz`, `bernoulli_hurwitz_packet`	Numerical complex-order interpolation and its residue packet, with real translation $a > 0$
`q_pochhammer`, `q_racah_lattice`, `q_racah_scalar`, `racah_scalar`	Exact finite rational products, lattice sites, and terminating scalar sums
`log_gamma_window`, `log_gamma_fourier`	Numerical positive-shape unit window and its unitary Fourier transform
`gamma_sech_ambiguity`, `shape_affinity`	Numerical centered cross-shape overlap and zero-shift affinity
`meixner_density`, `meixner_characteristic`, `beta_logit_density`	Numerical scalar densities and characteristic function
`meixner_pollaczek`	Numerical normalized shape polynomial
`beta_binomial_refinement`, `shape_refinement_coefficient`	Tuple of two-child amplitudes, or one many-child amplitude
`shape_transfer_singular_value`	Numerical $\sqrt{(s)_n/(s+t)_n}$, with $s, t > 0$

27.3.1 Exact residue reconstruction and a q-Racah negative control

The exact Bernoulli packet is

$$P_{n,M,r}(a) = M^{n-1} B_n\left(\frac{a+r}{M}\right), \qquad B_1(x) = x - \tfrac{1}{2}.$$

It accepts rational translations, including zero and negative values. Degree must be nonnegative, the modulus must be positive, and the residue must satisfy $0 \le r < M$. Integers, `Fraction` objects, and rational strings preserve exact input; floating inputs are rejected.

All exact scalar functions accept integers, `Fraction` objects, and rational strings and reject floating inputs. The q functions require rational $0 < q < 1$. Product lengths, scalar degrees `n`, and lattice indices `y` are nonnegative strict integers. Scalar q-Racah sums terminate at `min(n, y)`. A zero denominator in a retained term is rejected before any cancellation. Termination by itself does not establish positive orthogonality. The following control has two equal lattice sites and identical rows.

PYTHON | Executable example

```python
from fractions import Fraction as F
from komorebi_math.special_functions import (
    bernoulli_packet_integer, q_racah_lattice, q_racah_scalar,
)

a = F(3, 7)
packet_sum = sum(bernoulli_packet_integer(4, a, 3, r) for r in range(3))
# Independent polynomial control: B_4(x) = x^4 - 2*x^3 + x^2 - 1/30.
```

```
assert packet_sum == a**4 - 2 * a**3 + a**2 - F(1, 30)
assert bernoulli_packet_integer(1, 0, 1, 0) == F(-1, 2)

# A terminating q-Racah table can have repeated sites and no orthogonal rows.
q, alpha, beta, gamma, delta = F(1, 2), F(1, 2), F(1, 2), F(4), F(1)
assert gamma * q == q**-1
assert tuple(q_racah_lattice(y, gamma, delta, q) for y in range(2)) == (3, 3)
table = tuple(tuple(q_racah_scalar(n, y, alpha, beta, gamma, delta, q)
                    for y in range(2)) for n in range(2))
assert table == ((1, 1), (1, 1))
```

27.3.2 Check the numerical Gamma convention

The positive-shape window and Fourier convention are

$$g_s(y) = \Gamma(s)^{-1/2} \exp\left(\frac{sy - e^y}{2}\right), \qquad \hat{g}_s(\omega) = \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} g_s(y) e^{-i\omega y}\, dy.$$

`gamma_sech_ambiguity(alpha, gamma, h, eta)` instead uses the centered overlap with factor $e^{+i\eta y}$. The sign is part of that function's convention. `meixner_density(s, omega)` is the Fourier energy density, and `meixner_characteristic(s, h)` is $\operatorname{sech}(h/2)^s$.

This example compares the Fourier value with direct quadrature of the defining window after substituting $u = e^y$. It also checks a unit overlap, the shape-one density, and the squared refinement-amplitude normalization.

PYTHON | Executable example

```
import mpmath as mp
from komorebi_math.special_functions import (
    beta_binomial_refinement, gamma_sech_ambiguity,
    log_gamma_fourier, meixner_density,
)

ctx = mp.mp.clone()
ctx.dps = 45
shape, omega = ctx.mpf("1.75"), ctx.mpf("0.6")
# Direct Fourier integral after u = exp(y), independent of the Gamma formula.
def integrand(u):
    return ctx.exp((shape * ctx.log(u) - u) / 2) * u**(-ctx.j * omega) / u
integral = ctx.quad(integrand, [0, ctx.mpf(".25"), 1, 4, ctx.inf])
integral /= ctx.sqrt(2 * ctx.pi * ctx.gamma(shape))
assert abs(log_gamma_fourier(shape, omega, dps=45) - integral) < ctx.mpf("1e-32")
assert abs(gamma_sech_ambiguity(1, 1, 0, 0, dps=45) - 1) < ctx.mpf("1e-40")
density = meixner_density(1, omega, dps=45)
assert abs(density - 1 / ctx.cosh(ctx.pi * omega)) < ctx.mpf("1e-40")
amplitudes = beta_binomial_refinement(4, "1.25", "2.75", dps=45)
assert abs(sum(value**2 for value in amplitudes) - 1) < ctx.mpf("1e-40")
```

Numerical shapes must be positive finite real values; real coordinates must be finite. Integer degrees reject booleans and fractional values. The many-child shape and degree vectors must have the same positive length. Invalid domains raise `ValidationError`, dimension mismatches raise `DimensionError`, and a missing numerical backend raises `BackendUnavailableError`.

Each numerical call uses an isolated mpmath precision context. `dps` defaults to 50 and must be an integer at least 15. It requests working precision; it does not certify an error bound. The Bernoulli–Hurwitz functions accept finite complex orders, use the real translation domain $a > 0$, retain exact removable values at zero order, and resolve tiny nonzero orders with adaptive isolated precision capped at 10,000 digits. Their domain is narrower than the integer-order rational polynomial function's translation domain.

These APIs evaluate scalar formulas and finite refinements. They do not implement the full matrix recoupling engine, an infinite arithmetic transfer estimate, or a continuous-series spectral transform. See Special functions (§ 12) for the polynomial normalization, arithmetic conventions and integration scope.

27.4 Recover a finite periodic jump packet

Import from `komorebi_math.signals.fourier_recovery`. This explicit module is the entry point for the five exports below.

Export	Return and purpose
`PeriodicJumpPacket`	Canonical mean-zero periodic jump object
`FourierJumpRecovery`	Recovered packet, finite data arrays, and numerical certificate
`edge_moments_from_fourier`	NumPy vector of derivative moments
`hankel_moment_matrix`	NumPy matrix with entries $H_{ij} = m_{i+j}$
`recover_fourier_jumps`	`FourierJumpRecovery` from a consecutive high-mode block

The model is a real periodic signal $f = s + j$. The smooth background s has Fourier degree at most the declared `background_degree`; j has finitely many distinct active locations, real amplitudes summing to zero, and mean zero. For period $T > 0$, the normalization is

$$\hat{f}(n) = \frac{1}{T}\int_0^T f(x)e^{-2\pi i n x/T}\,dx, \qquad m_n = 2\pi i n \hat{f}(n) = \sum_\ell a_\ell e^{-2\pi i n \tau_\ell/T}$$

above the background degree. In this normalization, the multiplier is $2\pi i n$ for every period.

27.4.1 Recover three known edges on a nonstandard period

The recovery input must include `2 * maximum_jumps` consecutive positive modes starting at `background_degree + 1`. This example generates ten coefficients from three known jumps on period 3, then checks the recovered parameters against the original input. It also requires the returned numerical certificate.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.signals.fourier_recovery import recover_fourier_jumps

period = 3.0
locations = np.array([0.2, 1.1, 2.4])
amplitudes = np.array([2.0, -0.75, -1.25])
# Normalized coefficients use 2*pi*i*n for every period.
measurements = {
    n: np.sum(amplitudes * np.exp(-2j * np.pi * n * locations / period))
       / (2j * np.pi * n)
    for n in range(4, 14)
}
result = recover_fourier_jumps(measurements, background_degree=3,
                               maximum_jumps=5, period=period)
result.certificate.require()
assert result.order == 3
assert np.allclose(result.packet.locations, locations, rtol=0, atol=1e-10)
assert np.allclose(result.packet.amplitudes, amplitudes, rtol=0, atol=1e-10)
assert np.allclose(result.predicted_moments, result.measured_moments,
                   rtol=0, atol=1e-10)
```

27.4.2 Represent a signal with no active jumps

An all-zero high-mode block returns an empty jump packet. Its evaluation and all Fourier coefficients are zero. This is a valid model branch, rather than a request to invent an edge from a vanishing signal.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.signals.fourier_recovery import recover_fourier_jumps

result = recover_fourier_jumps({n: 0j for n in range(1, 5)},
                               background_degree=0, maximum_jumps=2)
result.certificate.require()
assert result.order == 0
assert np.array_equal(result.packet.evaluate([0, 0.5, 3]), np.zeros(3))
assert result.packet.fourier(0) == 0
```

The result contains `packet`, `annihilator_ascending`, `hankel_singular_values`, `moment_indices`, `measured_moments`, `predicted_moments`, and `certificate`. The packet wraps locations modulo

its period, sorts them, copies its arrays, and exposes read-only snapshots. `evaluate` uses the right-continuous representative with its mean removed. `fourier(0)` is therefore zero.

The rank gate compares singular values with `rank_tolerance * largest_singular_value`. A `known_order` hint must not exceed the observed numerical rank. The default tolerances are `rank_tolerance=1e-10` and `tolerance=1e-8`. Missing modes, invalid integer inputs, and out-of-range real scalar parameters raise `ValidationError`. Unsupported scalar types may raise Python conversion exceptions. Inconsistent real conjugacy, singular recovery geometry, and invalid active-packet assumptions can raise `AssumptionError`. Requiring a failed certificate raises `ValidationError`.

The certificate checks the represented model's finite moment fit, annihilating recurrence, node-circle residual, real amplitudes, and zero sum. It records Hankel and Vandermonde condition numbers. It does not verify the unknown background's bandlimit, the truth of the finite jump model, the chosen rank threshold's identification of the target order, or suitability of noise and conditioning. Decimated samples cannot replace the required consecutive block. A passing certificate establishes finite numerical consistency within these assumptions. See Fourier recovery of periodic jump packets (§ 15.14) for the algorithm and source boundaries.

27.5 Form exact finite join quotients

`komorebi_math.core.semilattice` exports `FiniteJoinSemilattice` and `SemilatticeQuotient`. A source object stores a nonempty square join table on labels `0, ..., n-1`. The constructor checks idempotence, commutativity, and associativity using exact integer comparisons, then keeps an immutable tuple snapshot. It accepts Python and NumPy integer labels and rejects booleans, floating labels, invalid ranges, and ragged tables.

27.5.1 Generate the least congruence containing a relation

For the Boolean semilattice of subsets of a two-element set, labels are bit masks and join is bitwise OR. Identifying zero with one also forces two to be identified with three. The certificate replays the finite merge derivation and checks the quotient homomorphism. The example checks every pair in the source table as a separate homomorphism control.

PYTHON | Executable example

```
from komorebi_math.core.semilattice import FiniteJoinSemilattice

# Bit masks label the four subsets of a two-element set; union is bitwise OR.
source = FiniteJoinSemilattice(
    tuple(tuple(left | right for right in range(4)) for left in range(4))
)
result = source.generated_congruence(((0, 1),))
result.certificate().require()
assert result.classes == ((0, 1), (2, 3))
assert result.projection == (0, 0, 1, 1)
assert result.quotient.join_table == ((0, 1), (1, 1))
for left in range(4):
    for right in range(4):
        assert result.projection[left | right] == result.quotient.join(
            result.projection[left], result.projection[right])
```

27.5.2 Validate a partition supplied by the caller

`quotient(partition)` accepts one arbitrary integer class label per source element, validates compatibility with join, and constructs the quotient. Canonical `partition` values are class minima; `projection` uses consecutive quotient labels. `classes` lists source members, and `quotient` is another validated `FiniteJoinSemilattice`.

PYTHON | Executable example

```
from komorebi_math.core.semilattice import FiniteJoinSemilattice

# The join of a finite chain is maximum; a convex partition gives a quotient.
chain = FiniteJoinSemilattice(
    tuple(tuple(max(left, right) for right in range(4)) for left in range(4))
)
result = chain.quotient((10, 10, -7, -7))
result.certificate().require()
assert result.partition == (0, 0, 2, 2)
assert result.projection == (0, 0, 1, 1)
assert result.classes == ((0, 1), (2, 3))
assert result.quotient.join_table == ((0, 1), (1, 1))
assert result.derivations is None  # This is not a leastness witness.
```

`generated_congruence(relations)` returns the least join congruence containing the supplied pairs. Its derivation establishes leastness for this finite input: every recorded merge follows

from an input pair or a join context of an earlier merge, and the output is itself a congruence containing the requested pairs. A manually supplied quotient has no such leastness witness.

Dimension failures raise `DimensionError`; invalid labels or join axioms raise `ValidationError`. A failed quotient certificate raises `ValidationError` when required. Storage is quadratic in source size, and source axiom validation is cubic. The exact decision concerns the supplied finite table. It does not imply a universal matroid, reconstruction, or doctrine theorem. See Finite join congruence and quotient (§ 14) for derivation fields, complexity, and independent enumeration coverage.

27.6 Evolve a fixed Dirac mode

Import from `komorebi_math.quantum.dirac` for finite four-spinor matrices in units $c = \hbar = 1$. Fourier labels belong to the unit torus $(\mathbb{R}/\mathbb{Z})^3$, with phase $e^{2\pi i n \cdot x}$ and continuum momentum $2\pi n$. Spinor coordinates use the order `e0, e1, e2, e3`.

Export	Return and supported input
`dirac_matrices`	Independent read-only `alpha_x, alpha_y, alpha_z, beta` NumPy snapshots
`central_mode_momentum`	Real vector for a strict integer mode triple and a finite real mesh
`dirac_symbol`	Complex 4×4 matrix for a finite real momentum triple and mass
`dirac_propagator`	Complex 4×4 evolution matrix for a finite real momentum, mass, and time

27.6.1 Compare the closed formula with an eigensystem exponential

For physical momentum q, the Hamiltonian and energy are

$$H(q) = \sum_{j=1}^{3} q_j \alpha_j + m\beta, \qquad E = \sqrt{m^2 + \|q\|^2}.$$

The Clifford identity gives $H^2 = E^2 I$, so for $E > 0$

$$U(t) = \cos(tE) I - i \sin(tE) \frac{H}{E}.$$

The implementation normalizes the real coefficients before constructing H/E, which avoids unstable division of complex entries at tiny positive energy. At zero energy it returns the identity. This example compares the closed formula with an independent Hermitian eigensystem exponential and checks unitarity and the scalar-square identity.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.quantum.dirac import dirac_propagator, dirac_symbol

momentum, mass, time = [0.3, -0.8, 1.2], -0.4, 0.7
hamiltonian = dirac_symbol(momentum, mass=mass)
evolution = dirac_propagator(momentum, time=time, mass=mass)
eigenvalues, eigenvectors = np.linalg.eigh(hamiltonian)
reference = (eigenvectors * np.exp(-1j * time * eigenvalues)) @
    ↪ eigenvectors.conj().T
assert np.allclose(evolution, reference, rtol=0, atol=2e-14)
assert np.allclose(evolution.conj().T @ evolution, np.eye(4), rtol=0, atol=2e-14)
energy_squared = mass**2 + sum(value**2 for value in momentum)
assert np.allclose(hamiltonian @ hamiltonian, energy_squared * np.eye(4),
                   rtol=0, atol=2e-14)
```

27.6.2 Check the Clifford and mesh conventions

`central_mode_momentum(mode, mesh)` computes $p\,\mathrm{sinc}(hp/\pi)$ coordinatewise, where $p = 2\pi n$, $h =$ `mesh`, and NumPy uses $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$. For nonzero mesh this equals $\sin(hp)/h$; at zero mesh the value is p. The example checks all pairwise Clifford anticommutators, continuum momentum, a central-difference value, and zero-energy evolution at a large finite time.

PYTHON | Executable example

```
import numpy as np
from komorebi_math.quantum.dirac import (
    central_mode_momentum, dirac_matrices, dirac_propagator,
)

matrices = dirac_matrices()
for i, left in enumerate(matrices):
    assert not left.flags.writeable
    for j, right in enumerate(matrices):
        expected = 2 * np.eye(4) if i == j else np.zeros((4, 4))
        assert np.array_equal(left @ right + right @ left, expected)
assert np.array_equal(central_mode_momentum([1, 0, -2]), 2 * np.pi * np.array([1,
    ↪ 0, -2]))
central = central_mode_momentum([1, 0, 0], mesh=1 / 12)
assert np.allclose(central, [12 * np.sin(np.pi / 6), 0, 0], rtol=0, atol=1e-14)
assert np.array_equal(dirac_propagator([0, 0, 0], time=1e100, mass=0), np.eye(4))
```

Real negative mass, mesh, and time are allowed. Nonfinite or complex scalar parameters and fractional or boolean mode labels raise `ValidationError`; wrong vector shapes raise `DimensionError`. Unrepresentable energy or phase is rejected. The central symbol can alias distinct Fourier labels: no Nyquist restriction or inverse recovery of those labels is provided.

These helpers return ordinary finite numerical matrices. They do not construct an unbounded Dirac operator, an infinite Fourier packet, or a sampled-grid Hilbert space, and they do not certify a uniform continuum limit. See Fixed-mode Dirac evolution (§ 22.6) for source coordinate actions and the larger finite comparison range.

27.7 What the worked examples check

The script executes twelve cases without skips. Its controls are the finite Erlang survival formula, known identity-factor spectrum, named integer prefixes, a direct recurrence comparison, local indexed window statuses, a Bernoulli polynomial, a degenerate q-Racah geometry, direct Fourier quadrature, shape normalization, known Fourier jump data, an empty-packet branch, finite quotient homomorphisms, Clifford coordinate identities, and an eigensystem exponential. Each example body above is copied from that script.

These examples check the stated finite inputs under each workflow's arithmetic and model assumptions. Broader mathematical conclusions require their own proofs. The release guide (§ E) describes package and platform qualification.

A Public API reference

The library has a curated 131-symbol stable surface and broader domain namespaces. The stable aliases below refer to the same objects; other domain APIs retain their documented contracts without automatically joining the 1.x compatibility promise.

A.1 Stable API

PYTHON | Executable example

```python
import komorebi_math.stable as stable
import komorebi_math.v1 as v1
import komorebi_math.api.v1 as api_v1

assert stable.__all__ == v1.__all__ == api_v1.__all__
```

The committed stable manifest checks exported names, implementation kinds, qualified identities, and callable signatures. A separate closed fixture checks mathematical behavior, while the portable codec contract checks record tags, represented Python types, monotone codec versions, and every required adjacent migration.

POWERSHELL

```powershell
komorebi api check --stable
komorebi verify stable
komorebi release check
```

The stable surface supports a complete finite workflow: construct operators and seeds, analyze coordinates, decompose into packets, refine polyphase addresses, certify results, render mathematical records, and persist portable objects. Research namespaces remain available but do not acquire the same 1.x compatibility promise merely by existing in `komorebi_math.__all__`.

See the stable API policy (§ 2) for the compatibility contract.

A.2 Typed finite expressions

Typed finite expressions are exposed through `komorebi_math.expression` and the broader research namespace. They are outside `komorebi_math.stable` and `komorebi_math.beta`.

PYTHON | Executable example

```python
from komorebi_math.expression import (
    PolynomialCoordinateSpace,
    apply,
    derivative,
    evaluate_expression,
    exact_value,
)

monomials = PolynomialCoordinateSpace(
    "x", 2, ((1,), (0, 1), (0, 0, 1)), "display label"
)
polynomial = exact_value(monomials, (1, 2, 3))
result = evaluate_expression(apply(derivative(monomials), polynomial))
assert result.value == (2, 6)
```

The semantic identity of a polynomial coordinate type includes its complete exact basis witness; labels do not participate. Closed terms normalize to exact rational matrices or vectors, equality is decidable in this finite language, and each reported rewrite is checked by exact normal-form replay. Expression JSON uses a dedicated strict schema and is not silently inserted into the frozen 51-codec persistence contract. Matrix-chain optimality is limited to binary parenthesizations under the stated classical dense scalar-operation model. See typed finite expressions (§ 20) for the assumptions and limits.

A.3 Finite measures and integration

Finite integration is exposed through `komorebi_math.integration` and the broad research namespace, not through `komorebi_math.stable` or `komorebi_math.beta`.

PYTHON | Executable example

```
from komorebi_math.integration import (
    CompositeSimpson,
    ExactPolynomial,
    IntervalMeasure,
    integrate,
    polynomial_derivative_bound,
)

polynomial = ExactPolynomial.univariate((0, 0, 0, 0, 1))
measure = IntervalMeasure(0, 1)
bound = polynomial_derivative_bound(polynomial, measure)
result = integrate(polynomial, measure, provider=CompositeSimpson(16, bound))
result.certificate.require()
```

This Simpson result is rigorous because the integrand is the same provenance-matched `Exact Polynomial`, the measure is the same exact rational interval, and nodes, weights, evaluation, summation, derivative bound, and error bound remain exact rational computations. Passing an arbitrary floating callable instead gives an estimated Richardson statement without a bound or a conditional statement with an explicitly assumed derivative bound. It does not give a rigorous result merely because the callable implements a similar formula.

Positive measures (`AtomicMeasure`, `IntervalMeasure`, `ProductMeasure`, and `PushforwardMeasure`) remain distinct from possibly signed quadrature functionals. Exact FTC, integration by parts, finite Fubini, affine pushforward, and integration-covector identities are available under their finite rational hypotheses. See Finite measures and integration (§ 19) for the full scope boundary.

A.4 General analysis–synthesis systems

PYTHON | Executable example

```
import numpy as np
from komorebi_math.stable import AnalysisSynthesisSystem

analysis = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
system = AnalysisSynthesisSystem.canonical_dual(analysis)
system.certificate().require()

components = system.components()
assert np.allclose(sum(components), np.eye(2))
```

For a square invertible atlas, the maps `S E_I A` are algebraic projectors. For a redundant frame, they are additive reconstruction components and need not be idempotent. The compatibility methods `projector()` and `projectors()` remain available, but new code should use `component()` and `components()` unless idempotence is independently established.

A.5 Operator and packet foundation

PYTHON | Executable example

```
from komorebi_math import (
    OperatorAlgebra,
    OperatorGrammar,
    OperatorRealization,
    SeedSystem,
    PacketSystem,
    PacketOperator,
    TransferGraph,
    UnifiedSystem,
)
```

These classes implement typed operator systems, finite generated hulls, packet coordinates, transfer blocks, support graphs, and compatible finite runtime models.

A.6 Refinement and obstruction

PYTHON | Executable example

```
from komorebi_math import (
    PacketRefinement,
    RefinementTower,
    WeightedTransferGraph,
    LatticePacketData,
    SmithCongruenceProblem,
    LinearRefinementDefect,
    NonlinearPacketMap,
    LinearObstructionProblem,
    LinearLiftProblem,
    PolynomialLiftProblem,
    TransportGraph,
)
```

The nonlinear namespace includes finite derivative providers, packet jets, higher composition, rooted trees, graded operations, correction and elimination, filtered spectral sequences, and shared smooth transport. Infinite analytic and projective-limit conclusions remain conditional on explicit hypotheses.

A.7 Advanced nonlinear calculus

PYTHON | Executable example

```
from komorebi_math import (
    CallableDerivativeProvider,
    PolynomialDerivativeProvider,
    PacketJetFamily,
    faa_di_bruno,
    PolynomialFixedPointProblem,
    NonlinearSchurElimination,
    LyapunovSchmidtReduction,
    FilteredVectorSpace,
    FiniteFilteredCochainComplex,
    KatoTransport,
)
```

Detailed result records and combinatorial utilities live under `komorebi_math.nonlinear`. See Nonlinear calculus (§ 16).

A.8 Structured computation

PYTHON | Executable example

```
from komorebi_math import (
    PacketLayout,
    PacketIR,
    DenseOperator,
    BlockDiagonalOperator,
    LowRankOperator,
    BlockDiagonalLowRankOperator,
    TreeNode,
    DTypeChoice,
    Task,
    recompress_factors,
    conjugate_gradient,
    compile_tree_lagrangian,
    compile_tree_budget,
    assign_discrete_precisions,
    schedule_unlimited,
)
```

The complete structured API is under `komorebi_math.algorithms`.

A.9 Signals

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.signals import (
    HaarPacketTree,
    PolyphaseSystem,
    SingularPacket1D,
    polyphase_analyze,
    recover_separated_jumps,
)

signal = np.arange(19, dtype=float)
coordinates = polyphase_analyze(signal, 4)
coordinates.certificate(signal).require()
assert np.array_equal(coordinates.reconstruct(), signal)

samples = np.concatenate((np.zeros(10), 2.0 * np.ones(10), -np.ones(10)))
recovery = recover_separated_jumps(
    samples,
    spacing=1.0,
    lipschitz_bound=0.0,
    noise_bound=0.0,
    minimum_jump=1.5,
)
recovery.certificate.require()
```

Full signal APIs include deterministic windows, finite framing, STFT and inverse STFT, spectrograms, periodogram and Welch estimators, cross spectra and coherence, FIR design and streaming, finite recursive filtering and stable biquads, FFT overlap-add convolution, rational resampling, LMS/NLMS/RLS adaptation, multilevel Haar denoising, polyphase coordinates, convolution blocks, filter banks, lifting, Haar packet trees, best basis selection, singular packets, scalar and vector jump recovery, finite difference break bands, nonlinear packet products, Fourier approximations, and Smith coordinates.

The finite convolution convention is signal first. `same` always returns the signal length, and `valid` returns zero samples when the kernel is longer than the signal.

A.10 Normalized generating functions

PYTHON | Executable example

```python
from komorebi_math.generating import (
    FiniteNormalizedSeries,
    FunctionRegistry,
    PrimePowerKernelCompiler,
    RationalDomain,
    compile_cartier_linear_representation,
    compile_finite_mask_carry_plan,
    factorial_power_basis,
)

basis = factorial_power_basis(2)
series = FiniteNormalizedSeries(basis, RationalDomain(), (1, 2, 3, 4))
series.certificate().require()

mask = compile_finite_mask_carry_plan((1, 1, 1), 2)
cartier = compile_cartier_linear_representation((1, 1), (1, -1), 5)
kernel = PrimePowerKernelCompiler(3, 5, 2)
```

The full namespace includes portable basis descriptors, exact domains, formal and domain-aware finite series, recurrence and algebraic sources, bounded closed-form recognition, residue sections, Mahler masks, finite carry matrices, rational Cartier automata, Bell digit reductions, Kummer layers, exact prime-power binomial residues, Gaussian q-Kummer support, Euler and cyclotomic collapse, portable frozen functions, and explicit FIR adapters. See Normalized generating functions (§ 11).

A.11 Dynamics

PYTHON | Executable example

```python
import numpy as np
from komorebi_math.dynamics import (
    BayesianUpdate,
    FeshbachReduction,
    MarkovLumping,
    MatrixSemigroup,
    PhaseTypeSurvival,
    fractional_power,
)

generator = np.array([[-1.0, 0.0], [0.0, -2.0]])
semigroup = MatrixSemigroup(generator)
semigroup.semigroup_certificate(0.3, 0.7).require()

positive = np.diag([1.0, 4.0, 9.0])
root = fractional_power(positive, 0.5)
assert np.allclose(root, np.diag([1.0, 2.0, 3.0]))
```

Complete dynamics APIs include semigroups, refinement defects, finite towers, continuous and

discrete Markov models, conditional expectations, martingales, Bayesian updates, exact sufficient partitions, approximate profile clustering, hidden Markov filtering, fractional operators, Green operators, Feynman Kac transfer, Feshbach reduction, memory kernels, and phase type survival.

The base wheel contains a NumPy matrix exponential fallback. It can be tested independently with

PYTHON | Executable example

```
import numpy as np
from komorebi_math.dynamics import matrix_exponential

generator = np.diag([-1.0, -2.0])
exponential = matrix_exponential(generator, method="taylor")
assert np.allclose(exponential, np.diag(np.exp([-1.0, -2.0])))
```

A.12 Certificates

Validation APIs return a `Certificate` directly or a result with a `certificate` attribute. For an existing certificate, the following attributes describe its evidence and `require()` checks its live validity:

PYTHON | Contextual fragment

```
certificate.valid
certificate.exactness
certificate.residuals
certificate.tolerance
certificate.assumptions_checked
certificate.assumptions_unchecked
certificate.diagnostics
certificate.require()
```

Execution classes are `exact`, `numerical`, `conditional`, `heuristic`, and `planned`.

A.13 Configuration

PYTHON | Executable example

```
from komorebi_math import configure, get_settings, settings_context

configure(tolerance=1e-12)
with settings_context(tolerance=1e-14):
    assert get_settings().tolerance == 1e-14
current = get_settings()
assert current.tolerance == 1e-12
```

A.14 Provenance and feature status

PYTHON | Executable example

```python
from komorebi_math import describe, feature, features

record = describe("komorebi_math.signals.recover_separated_jumps")
status = feature("signals")
all_statuses = features()
```

A.15 Command line interface

POWERSHELL

```
komorebi info
komorebi doctor
komorebi features
komorebi verify core
komorebi verify refinement
komorebi verify nonlinear
komorebi verify algorithms
komorebi verify signals
komorebi verify dynamics
komorebi verify hardening
komorebi verify dsp
komorebi verify generating
komorebi verify coordinates
komorebi api check --strict-signatures
```

With the `exact`, `scientific` and `yaml` extras installed, use `komorebi verify all` for the complete installed functional suite. Source qualification and reproducible distribution checks are described in the release guide (§ E).

A.16 Extension discovery

Separately installed packages can register with the entry point group

OUTPUT

```
komorebi_math.extensions
```

See Extensions and companion C integration (§ 26) for the extension contract.

A.17 Domain namespaces

Domain namespaces provide finite APIs with the scope stated in each guide. Individual global or completion objects remain partial or conditional when they require external mathematical theorems. Feature status is available through `feature(...)`.

A.18 Analytic and quantum models

A.19 `komorebi_math.analytic`

The analytic namespace implements finite coefficient calculus and numerical diagnostics for the cyclic entire function families.

A.19.1 Cyclic coefficient coordinates

PYTHON | Contextual fragment

```
cyclic_project_coefficients(coefficients, order, residue)
cyclic_packets(coefficients, order)
cyclic_reconstruct(packets)
cyclic_project_value(function, z, order, residue)
cyclic_product_packets(left_packets, right_packets)
cyclic_projection_certificate(coefficients, order)
```

A.19.2 Transfer and signed sections

PYTHON | Contextual fragment

```
transfer_section(order, residue, z, scale=1)
signed_section(order, residue, z)
transfer_sections(order, z, scale=1)
signed_sections(order, z)
denominator(order, z)
denominator_derivative(order, z, derivative_order=1)
transfer_shift_matrix(order, scale=1)
signed_shift_matrix(order)
cyclic_addition(...)
section_certificate(...)
CyclicSectionFamily
```

A.19.3 Reciprocal, Appell, and theta families

PYTHON | Contextual fragment

```
logarithmic_coefficients(order, degree)
reciprocal_coefficients(order, parameter, degree)
reciprocal_polynomial(order, degree)
reciprocal_value(order, z, parameter=1)
transfer_tangent(order, z)
transfer_tangent_coefficients(order, degree)
congruential_inverse(order, residue, degree)
congruential_reciprocal_coefficients(order, residue, parameter, degree)
appell_polynomial(order, residue, parameter, degree, x)
appell_coefficients(order, residue, parameter, degree)
theta_polynomial(order, degree, parameter)
```

Symbolic polynomial constructors require the optional exact dependency.

A.19.4 Zero diagnostics

PYTHON | Contextual fragment

```
positive_zero_string(order, count, start=0)
asymptotic_positive_zero(order, index)
zero_count_asymptotic(order, radius, reduced=False)
hadamard_partial_product(order, z, positive_zeros)
reciprocal_coefficient_from_zeros(order, degree, positive_zeros)
reduced_zero_zeta(order, exponent, terms=64)
CyclicDenominator
```

A `ZeroStringResult` contains the computed representatives, asymptotic seeds, numerical residuals, and a certificate. The certificate does not assert exhaustion of the global zero set.

A.20 `komorebi_math.quantum`

The quantum namespace implements finite dimensional matrix models only.

A.20.1 Seeds and symmetry

PYTHON | Contextual fragment

```
QuantumSeedSystem
schmidt_rank(state, left_dimension, right_dimension)
local_orbit_dimension(state, left_dimension, right_dimension)
observable_seed_rank(block_dimensions, multiplicities)
computational_basis_generation(qubits)
finite_group_projector(representation, character, irrep_dimension)
cyclic_symmetry_projectors(symmetry, order)
charge_certificate(operator, symmetry, order, charge)
charge_selection_certificate(packets, operator, charge)
total_charge_packets(left_packets, right_packets)
```

A.20.2 Circuits and moving packets

PYTHON | Contextual fragment

```
CircuitPacketExpansion
fixed_packet_leakage(hamiltonian, projector, time)
MovingPacketSystem
KatoTransport
direct_packet_rotation(source_packets, target_packets)
moving_geometric_coupling(hamiltonian, system, time, packet)
moving_packet_leakage_bound(...)
```

A.20.3 Channels and error correction

PYTHON | Contextual fragment

```
QuantumChannel
amplitude_damping(gamma)
operator_packets(operator, packets)
reconstruct_operator_packets(blocks)
dephase(operator, packets)
LindbladGenerator
QuantumCode
```

A.20.4 Effective and spectral diagnostics

PYTHON | Contextual fragment

```
QuantumFeshbachReduction
EuclideanTransfer
periodic_laplacian(shape, spacing=None)
periodic_lattice_eigenvalues(shape, spacing=None)
heat_trace(operator, time)
spectral_dimension(operator, time)
low_mode_dispersion_bound(shape, mode, spacing=None)
constraint_closure(constraints)
transported_constraint_defect(...)
BoundedHamiltonianTower
```

The quantum API returns finite matrix certificates. It does not report continuum quantum field theory, anomaly freedom, Lorentzian reconstruction, or gravitational infrared conclusions.

A.21 Arithmetic

The public arithmetic namespace includes `ModularRing`, `PrimeField`, `TruncatedPadic`, `HurwitzSeries`, `StandardWindow`, `EulerLocalCoordinates`, `MahlerSeries`, `UnitModeMahler`, `LinearRecurrence`, and `RecurrencePacket`.

Exact local computations use Python integers and fractions. Modular recurrence execution avoids constructing the full integer term when only a residue is required.

A.22 Sequences

The public sequence namespace includes `Sequence`, `CompiledSequence`, `AutomaticSequence`, and `HolonomicRecurrence`, together with the catalog and exact transforms. Compilation prefers a certified declared model, then a conditional declared model, then a finite detected recurrence, and finally the original evaluator.

A.23 `komorebi_math.deformation`

PYTHON | Contextual fragment

```
FiniteLiftForest
LiftBranch
LiftForestReport
VisibilityTree
RationalCountingSeries
FiniteCochainComplex
CohomologyData
PresentedModule
FittingGenerators
FiberSupportResult
FrobeniusSpine
CompanionForest
CompatibleCarrier
finite_frobenius_separator
nested_subspace_occupancy
```

Exact presentation and Fitting calculations require SymPy. Forests, visibility, counting, finite field cohomology, finite occupancy, and Frobenius separation run with the base NumPy installation.

A.24 `komorebi_math.langlands.local`

PYTHON | Contextual fragment

```
FiniteField
FiniteFieldElement
FrobeniusOrbit
TameWeilPacket
SpecialMonodromyBlock
GaussSumResult
HeisenbergModel
AlternatingForm
QuadraticRefinement
ProjectiveIntertwinerEnvelope
SymplecticBridge
LocalPacketBridge
```

These classes compute finite local packet data. They do not return an unrestricted local correspondence.

A.25 `komorebi_math.langlands.finite_image`

PYTHON | Contextual fragment

```
FiniteGroup
FiniteSubgroup
LinearCharacter
InducedRepresentation
MonomialAction
MackeyDecomposition
CliffordTower
AutomorphicInductionPlan
VirtualCharacterResolution
```

Finite-group and orbit combinatorics use exact checks. Character and intertwiner certificates computed with floating complex arithmetic remain numerical. Automorphic induction plans and unresolved Clifford extension steps are conditional.

A.26 Ramification, global support, and completion

The first three types below are exported by `komorebi_math.langlands.ramification`, the next three by `komorebi_math.langlands.global_`, and the final two by `komorebi_math.langlands.completion`.

PYTHON | Contextual fragment

```
ProjectiveMultiplicityPacket
RamificationStage
RamificationCliffordTower
FiniteHeckePackets
SupportComparison
PatchingSupportReport
AutomorphicCompletionReport
CompatiblePropagationReport
```

The completion API reports which external gates are supplied. It never treats a successful finite calculation as a proof of automorphy.

A.27 `komorebi_math.export`

PYTHON | Contextual fragment

```
DocumentMetadata
MathDocument
Heading
Paragraph
Equation
Table
CodeBlock
ListBlock
Figure
RawLatex
ExportFormat
ExportOptions
ExportResult
ExportBundleResult
LatexRenderOptions
HtmlRenderOptions
LatexEngine
LatexCompileOptions
LatexCompilation
to_serializable
table_rows
document_from_data
sequence_document
render_text
render_markdown
render_html
render_latex
export_data
export_document
export_many
export_bundle
available_export_formats
available_latex_engines
compile_latex
```

The base installation supports every format except YAML and PDF when their external backends are absent. YAML requires PyYAML. PDF requires a local supported LaTeX engine.

A.28 Scalable nonlinear execution and compatibility

A.28.1 Sparse tensors and providers

- `SparseTensorEntry`
- `SparseSymmetricTensor`
- `SparsePolynomialDerivativeProvider`

A.28.2 Automatic differentiation

- DirectionalJVP
- AutomaticDifferentiationProvider
- JaxDerivativeProvider
- TorchDerivativeProvider
- autodiff_provider
- available_ad_frameworks

A.28.3 Packet-jet execution and snapshots

- PacketJetInputGroup
- PacketJetExecutionPlan
- PacketJetEvaluation
- PacketJetBatchResult
- compile_packet_jet_plan
- execute_packet_jet_plan
- execute_packet_jets
- PacketJetCoefficient
- PacketJetSnapshot
- snapshot_packet_jet

A.28.4 Optional CuPy boundary

- CuPyDeviceDerivativeProvider
- CuPyPolynomialDerivativeProvider
- CuPyPacketJetExecutor
- CuPySnapshotExecutor
- CuPyPacketJetSnapshotExecutor
- CuPyRuntimeStatus
- CuPyValidationResult
- cupy_runtime_status
- validate_cupy_snapshot
- nonlinear_cupy_available
- packet_jet_benchmark_contract

A.28.5 Portable persistence

- PortableCodec
- register_portable_codec
- portable_codecs
- to_portable_data
- from_portable_data
- portable_digest
- dumps_portable
- loads_portable
- save_portable
- load_portable
- from_serializable

A.28.6 API compatibility

- `PublicApiSymbol`
- `PublicApiManifest`
- `ApiCompatibilityReport`
- `create_public_api_manifest`
- `load_public_api_manifest`
- `save_public_api_manifest`
- `compare_public_api`
- `check_public_api`
- `deprecated`
- `warn_deprecated`
- `KomorebiDeprecationWarning`

Structural API checks compare explicit exports and reflected signatures. Mathematical semantics and portable persistence are checked separately.

A.29 Streams, migrations and behavioral checks

A.29.1 Streamed packet jets

- `PacketJetStreamChunkRecord`
- `PacketJetStreamManifest`
- `PacketJetStreamWriteResult`
- `PacketJetStreamSelection`
- `StreamedPacketJetEvaluation`
- `PacketJetSnapshotStreamReader`
- `capture_packet_jet_stream`
- `save_packet_jet_snapshot_stream`
- `open_packet_jet_snapshot_stream`
- `load_packet_jet_snapshot_stream`

A.29.2 Portable migrations

- `PortableMigration`
- `register_portable_migration`
- `portable_migrations`

A.29.3 Golden semantics and profiling

- `BehavioralGoldenCaseResult`
- `BehavioralGoldenReport`
- `load_behavioral_golden_fixture`
- `run_behavioral_golden_fixture`
- `PacketJetComplexityEstimate`
- `RuntimeProfileSample`
- `RuntimeStageProfile`
- `PacketJetProfileReport`
- `estimate_packet_jet_complexity`
- `profile_operation`

- `profile_packet_jet_runtime`

A.29.4 Curated beta surface

`komorebi_math.beta` re-exports 83 reviewed names from the finite certificate, packet, nonlinear execution, streaming, persistence, profiling, and compatibility layers. Existing beta signatures are checked strictly while additions remain permitted. The 131-symbol stable namespace has the separate compatibility promise described above.

- `BETA_PUBLIC_NAMESPACES`
- `BETA_API_STABILITY_LEVEL`
- `DEFAULT_BETA_API_MANIFEST`
- `create_beta_api_manifest`
- `load_beta_api_manifest`
- `check_beta_api`

See streamed packet jets (§ 18), behavioral fixtures and profiling (§ 25), and the stable API contract (§ 2).

A.30 General coordinate atlas

The complete coordinate API is available from `komorebi_math.coordinates`. It is organized around arbitrary coordinate systems rather than one named sequence conversion.

The namespace includes:

- exact finite bases, subspaces, atlases, path composition, and commuting diagrams
- normalized generating-function basis changes
- Stirling, Lah, binomial, forward-difference, divisor-zeta, and subset-zeta transforms
- classical, interpolation, and custom polynomial families
- arbitrary constant-coefficient recurrence models and state-space coordinates
- affine, barycentric, and formal changes of variable
- vector, covector, operator, bilinear-form, and higher tensor transport
- exact prime-field maps
- Fourier, cosine, Hadamard, Haar, and modal numerical transforms
- residue, digit-reversal, Chinese-remainder, tensor-axis, and custom packet permutations
- trusted portable exact and numerical coordinate records

PYTHON | Executable example

```python
from komorebi_math.coordinates import (
    CoordinateAtlas,
    basis_from_matrix,
    connection_transform,
    get_polynomial_family,
    get_recurrence_model,
    standard_basis,
)

standard = standard_basis(3, name="standard")
shear = basis_from_matrix("shear", ((1, 1, 0), (0, 1, 1), (0, 0, 1)))
```

```
standard.connection_to(shear).certificate().require()

legendre = get_polynomial_family("legendre")
chebyshev = get_polynomial_family("chebyshev-t")
connection_transform(legendre, chebyshev, 8).certificate().require()

pell = get_recurrence_model("pell")
tribonacci = get_recurrence_model("tribonacci")
pell.coordinate_transform_to(tribonacci, 20).certificate().require()
```

POWERSHELL

```
komorebi verify coordinates
komorebi coordinates catalog
komorebi coordinates polynomial --source legendre --target chebyshev-t --degree 8
komorebi coordinates model --source pell --target tribonacci --count 20
```

Euler-to-Fibonacci helpers provide one worked bridge within the general coordinate namespace.

B Thematic API inventory

This inventory groups public interfaces by mathematical task. The curated compatibility surface is available through `komorebi_math.stable`, `komorebi_math.v1` and `komorebi_math.api.v1`. The committed stable API manifest and the source provide the authoritative signature references. Broader domain interfaces retain the scope described in their individual guides.

B.1 `komorebi_math.core`

B.1.1 Grammars and realizations

- `Arrow`
- `OperatorGrammar`
- `OperatorRelation`
- `OperatorWord`
- `OperatorRealization`

B.1.2 Algebras and seed systems

- `OperatorAlgebra`
- `AlgebraBasisResult`
- `SeedSystem`
- `cyclic_span`
- `matrix_units`
- `semisimple_seed_rank`
- `single_operator_seed_rank`
- `shared_direct_sum_seed_bounds`
- `independent_direct_sum_seed_rank`
- `tensor_seed_rank_upper_bound`

B.1.3 Packets and transfers

- `PacketSystem`
- `PacketCoordinates`
- `PacketOperator`
- `TransferGraph`
- `SupportPreorder`
- `block_from_paths`
- `path_contributions`

B.1.4 Morphisms

- `OperatorMorphism`
- `PacketMorphism`
- `SeedMorphism`

B.2 `komorebi_math.refinement`

B.2.1 Towers and packet addresses

- `PacketRefinement`
- `RefinementTower`
- `common_refinement`
- `ResidueCoordinates`
- `residue_analyze`
- `residue_packet_system`
- `crt_pair`
- `crt_many`

B.2.2 Weighted transfer

- `WeightedTransferEdge`
- `WeightedTransferGraph`

B.2.3 Lattices and Smith congruences

- `LatticePacketData`
- `SmithCongruenceProblem`

B.2.4 Linear defects

- `LinearRefinementDefect`

B.3 `komorebi_math.nonlinear`

B.3.1 Coordinates, defects, obstruction, lifting, and holonomy

- `NonlinearPacketMap`
- `NonlinearRefinementDefect`
- `LinearObstructionProblem`
- `LinearLiftProblem`
- `PolynomialLiftProblem`
- `FiniteLiftTree`
- `TransportEdge`
- `TransportGraph`
- `build_lift_tree`
- `solve_mod_prime`

B.3.2 Derivative providers and composition

- `DerivativeProvider`
- `CallableDerivativeProvider`

- PolynomialDerivativeProvider
- FiniteDifferenceDerivativeProvider
- SympyDerivativeProvider
- ComposedDerivativeProvider
- faa_di_bruno
- packet_faa_di_bruno
- set_partitions
- stirling_second

B.3.3 Packet jets and trees

- PacketJet
- PacketJetAddress
- PacketJetFamily
- PacketJetSupport
- packet_jet_support
- packet_taylor
- OrderedRootedTree
- PolynomialFixedPointProblem
- majorant_convergence
- packet_tree_reconstruction
- ordered_rooted_trees
- ordered_trees_with_leaves
- catalan_number

B.3.4 Graded nonlinear operations

- FiniteAbelianGroup
- GradedPacketOperation
- maurer_cartan_packets
- higher_maurer_cartan_packets
- divided_square_packets

B.3.5 Correction and elimination

- HilbertObstructionProjector
- RightInverseCorrectionProblem
- LyapunovSchmidtReduction
- NonlinearSchurElimination

B.3.6 Filtered lifting and spectral sequences

- FilteredVectorSpace
- FilteredCorrectionProblem
- FiniteFilteredCochainComplex
- verify_finite_filtered_branch
- conditional_projective_limit_completion

B.3.7 Shared smooth transport

- MovingPacketSystem
- KatoTransport
- direct_packet_rotation

The transport objects are re-exported from `komorebi_math.quantum` for backward compatibility.

B.4 `komorebi_math.algorithms`

B.4.1 Layout and packet IR

- PacketLayout
- PacketBlock
- PacketIR

B.4.2 Structured operators

- StructuredOperator
- DenseOperator
- IdentityOperator
- DiagonalOperator
- BlockDiagonalOperator
- CirculantOperator
- ToeplitzOperator
- KroneckerOperator
- WalshHadamardOperator
- as_structured_operator

B.4.3 Low rank algebra

- LowRankOperator
- RecompressionResult
- BlockDiagonalLowRankOperator
- recompress_core
- recompress_factors
- product_chain_error_bound
- accumulated_recompression_bound

B.4.4 Support, Krylov, precision, compilation, and scheduling

The namespace includes structural support propagation, Krylov and conjugate gradient routines, block Jacobi certificates, continuous and discrete precision allocation, exact finite tree compilers, Pareto frontiers, task scheduling, complexity estimates, restricted structure discovery, NumPy execution, optional CuPy execution, and matched benchmark contracts.

B.5 `komorebi_math.signals`

B.5.1 Production finite DSP

- `WindowMetrics`, `get_window`, `window_metrics`, `overlap_profile`, `overlap_add_certificate`, `canonical_dual_window`
- `FrameBatch`, `STFTResult`, `frame_signal`, `stft`, `istft`, `spectrogram`, `overlap_add`
- `PowerSpectrum`, `CrossSpectrum`, `periodogram`, `welch_psd`, `cross_spectrum`, `parseval_certificate`
- `FIRFilter`, `FrequencyResponse`, `StreamingFIR`, `design_fir`, `design_lowpass`, `design_highpass`, `design_bandpass`, `design_bandstop`, `fir_filter`, `stream_fir_filter`
- `RationalResamplingPlan`, `ResamplingResult`, `upsample`, `downsample`, `interpolate`, `decimate`, `design_resample_filter`, `resample_poly`, `resample_poly_result`
- `AdaptiveFilterResult`, `lms_filter`, `nlms_filter`, `rls_filter`
- `HaarDecomposition`, `WaveletDenoiseResult`, `haar_decompose`, `wavelet_denoise`, `threshold_coefficients`, `denoising_risk_certificate`
- `StreamingSTFT`, `StreamingSTFTBatch`, `stream_stft`
- `LinearFilter`, `FilterOutput`, `StreamingLinearFilter`, `lfilter`, `stream_lfilter`, `design_biquad`
- `OverlapAddConvolver`, `fft_convolve`, `overlap_add_convolve`, `certified_overlap_add_convolve`

B.5.2 Polyphase coordinates

- `PolyphaseCoordinates`
- `LatticePolyphaseCoordinates`
- `PolyphaseKernel`
- `PolyphaseSystem`
- `polyphase_analyze`
- `polyphase_synthesize`
- `polyphase_roundtrip_certificate`
- `lattice_polyphase_analyze`
- `lattice_polyphase_synthesize`
- `polyphase_filter_kernels`
- `mixed_radix_address`
- `split_mixed_radix_address`
- `refinement_children`
- `residue_project`

B.5.3 Convolution

- `ConvolutionMode`
- `PolyphaseConvolution`
- `PolyphaseBlockMatrix`
- `FinitePolyphaseKernel`
- `convolve_signal`
- `convolution_matrix`
- `convolution_output_length`
- `polyphase_convolution`
- `finite_polyphase_blocks`
- `finite_polyphase_filter_kernels`

B.5.4 Filter banks and wavelets

- `PeriodicFilterBank`
- `FiniteFilterBank`
- `HaarFilterBank`
- `PeriodicLiftingScheme`
- `FilterBankReport`
- `HaarPacketTree`
- `PrunedPacketCoefficients`
- `BestBasisResult`
- `haar_analyze`
- `haar_synthesize`
- `haar_matrix`
- `haar_filter_bank`
- `periodic_five_three_lifting`
- `best_basis`
- `entropy_cost`
- `lp_cost`
- `packet_energy`

B.5.5 Singular packets and recovery

- `TruncatedPowerTerm`
- `TruncatedPowerPacket`
- `SingularPacket1D`
- `RetainedPacketReconstruction`
- `RetainedPacketResult`
- `JumpDetection`
- `JumpRecoveryResult`
- `VectorJumpRecoveryResult`
- `BreakpointBandResult`
- `retained_packet_reconstruct`
- `retained_packet_reconstruction`
- `estimated_packet_reconstruct`
- `recover_separated_jumps`
- `recover_jumps_subgaussian`
- `recover_vector_jumps`
- `recover_vector_jumps_subgaussian`
- `detect_piecewise_polynomial_breaks`
- `finite_difference`
- `scalar_region_error`
- `scalar_region_lp_error`
- `overlap_matrix_error`
- `overlap_partition_lp_error`
- `step_location_lp_error`

B.5.6 Secondary signal tools

- cyclic coefficient packet products
- shifted nonlinear filters
- Fourier low pass approximation
- Otsu thresholding
- the positive contrast packet estimator under its declared restricted model
- Smith and Cayley phase coordinates

B.6 `komorebi_math.dynamics`

B.6.1 Semigroups and refinement

- `MatrixSemigroup`
- `PacketSemigroup`
- `SemigroupRefinement`
- `FiniteSemigroupTower`
- `ProjectiveSemigroupTower`
- `matrix_exponential`
- `semigroup_refinement_defect`

B.6.2 Markov and martingale systems

- `FiniteMarkovChain`
- `ContinuousTimeMarkovChain`
- `AggregationMap`
- `MarkovLumping`
- `ConditionalExpectation`
- `ConditionalExpectationTower`
- `MartingaleTower`
- `aggregation_matrix`
- `coordinate_lumpability_residual`
- `normalize_partition`
- `partition_refines`

B.6.3 Bayesian models

- `BayesianUpdate`
- `BayesianPartition`
- `BayesianFlow`
- `HiddenMarkovFilter`
- `coarsest_sufficient_partition`
- `approximate_likelihood_partition`
- `relative_entropy`
- `relative_entropy_decomposition`

B.6.4 Fractional and Green operators

- `FractionalDefectResult`
- `fractional_power`
- `fractional_semigroup`
- `fractional_defect`
- `fractional_naturality_certificate`
- `DirichletGreen1D`

B.6.5 Feynman Kac, Feshbach, memory, and survival

- `FiniteFeynmanKac`
- `FeshbachReduction`
- `MemoryKernel`
- `RationalMemoryKernel`
- `RationalMemoryRealization`
- `memory_kernel_error_bound`
- `kernel_l1_error`
- `volterra_trajectory_bound`
- `PhaseTypeSurvival`

B.7 Infrastructure

B.7.1 Certificates and configuration

- `Certificate`
- `ComputationResult`
- `ConditionalResult`
- `Exactness`
- `Settings`
- `configure`
- `get_settings`
- `settings_context`

B.7.2 Registries

- `FeatureSpec`
- `FeatureStatus`
- `feature`
- `features`
- `ClaimStatus`
- `ProvenanceRecord`
- `describe`

B.7.3 Unified runtime

- `UnifiedStage`
- `StageSpec`
- `UnifiedSystem`
- `STAGES`

B.8 `komorebi_math.analytic`

B.8.1 Cyclic coordinates and sections

- `CyclicSectionFamily`
- `cyclic_project_coefficients`
- `cyclic_packets`

- cyclic_reconstruct
- cyclic_project_value
- cyclic_product_packets
- cyclic_projection_certificate
- transfer_section
- signed_section
- transfer_sections
- signed_sections
- denominator
- denominator_root_sum
- denominator_derivative
- transfer_shift_matrix
- signed_shift_matrix
- cyclic_addition
- section_certificate

B.8.2 Reciprocal and polynomial families

- logarithmic_coefficients
- reciprocal_coefficients
- reciprocal_polynomial
- reciprocal_value
- reciprocal_semigroup_certificate
- transfer_tangent
- transfer_tangent_coefficients
- congruential_inverse
- congruential_reciprocal_coefficients
- appell_polynomial
- appell_coefficients
- appell_polynomial_symbolic
- theta_polynomial
- theta_polynomial_symbolic

B.8.3 Zero diagnostics

- CyclicDenominator
- ZeroStringResult
- asymptotic_positive_zero
- positive_zero_string
- zero_count_asymptotic
- hadamard_partial_product
- reciprocal_coefficient_from_zeros
- reduced_zero_zeta

B.9 `komorebi_math.quantum`

B.9.1 Seeds, symmetry, and circuits

- QuantumSeedSystem
- schmidt_rank
- local_orbit_dimension
- observable_seed_rank
- computational_basis_generation
- finite_group_projector
- cyclic_symmetry_projectors
- charge_certificate
- charge_selection_certificate
- total_charge_packets
- symmetry_projector_certificate
- CircuitPacketExpansion
- CircuitPathAmplitude
- InterferenceReport

B.9.2 Moving packets and leakage

- LeakageResult
- fixed_packet_leakage
- MovingPacketSystem
- KatoTransport
- DirectRotationResult
- direct_packet_rotation
- moving_geometric_coupling
- moving_packet_leakage_bound

B.9.3 Channels and error correction

- ChannelPacketBlock
- QuantumChannel
- amplitude_damping
- operator_packets
- reconstruct_operator_packets
- dephase
- LindbladGenerator
- QuantumCode
- KnillLaflammeReport

B.9.4 Effective dynamics and finite geometry

- QuantumFeshbachReduction
- EuclideanTransfer
- heat_trace
- spectral_dimension
- periodic_lattice_eigenvalues
- periodic_laplacian
- low_mode_dispersion_bound
- ConstraintClosureReport
- constraint_closure
- transported_constraint_defect
- BoundedHamiltonianTower
- HamiltonianTowerReport

B.10 Arithmetic and sequences

Arithmetic and sequence interfaces cover exact local domains, valuations, Hurwitz convolution, standard windows, Euler coordinates, Mahler transfer, recurrence packets, lazy sequences, automatic models, holonomic recurrences, transforms, and compilation.

B.11 Deformation

The deformation namespace covers lift forests, visibility, counting charts, finite cochain complexes, packet cohomology, Fitting fibers, nested occupancy, Frobenius spines, companion forests, and compatible carriers.

B.12 Local Langlands finite models

Finite local models cover finite fields, tame Frobenius packets, monodromy chains, Gauss sums, Heisenberg models, alternating forms, quadratic refinements, projective envelopes, and finite bridges.

B.13 Finite image, ramification, and completion

The finite image namespace exports explicit groups, subgroups, linear characters, induced representations, orbit packets, Mackey packets, Clifford towers, conditional induction plans, and virtual character resolutions. The ramification namespace exports finite ramification tower diagnostics. The global and completion namespaces export finite Hecke packets, support comparisons, patching reports, dyadic sign diagnostics, five gate completion reports, and compatible propagation reports.

B.14 Portable exports and LaTeX

The export namespace provides document blocks, serialization adapters, JSON and tabular writers, text and web renderers, LaTeX renderers, LaTeX compiler discovery, PDF compilation, manifested export bundles with normalized ZIP metadata, and artifact certificates. See Export and LaTeX (§ 24) for complete examples.

B.15 Scalable nonlinear execution

Scalable execution includes sparse derivative tensors and providers, lazy directional automatic-differentiation adapters, packet-jet plans, provider-independent snapshots, CuPy runtime diagnostics, versioned portable persistence and structural API compatibility tools.

Preferred public CuPy names use `CuPy`. Existing implementation spellings beginning with `Cupy` remain aliases inside the nonlinear modules and are not the preferred documented surface.

B.16 Streams and persistence

Streams support directory-backed packet-jet snapshots, resumable live capture, bounded chunking, adjacent codec migrations, behavioral fixtures, finite complexity counts and runtime profiles.

B.17 `komorebi_math.generating`

B.17.1 Bases and coefficient domains

- `WeightedMonomialBasis`, `BasisDescriptor`, `FiniteNormalizedSeries`, `FormalSeries`
- `IntegerDomain`, `RationalDomain`, `ResidueRingDomain`, `PrimeFieldDomain`, `TruncatedPadicDomain`
- `ordinary_basis`, `exponential_basis`, `factorial_power_basis`, `q_factorial_basis`, `fibonacci_factorial_basis`, `pochhammer_basis`

B.17.2 Custom sources and recognition

- `CustomFunctionSpec`, `FunctionRegistry`
- recurrence, rational, hypergeometric, algebraic, Euler-product, composition, and factorial-power Bell constructors
- bounded recurrence, P-recursive, hypergeometric, algebraic, and quasipolynomial recognizers

B.17.3 Arithmetic compilers

- `FiniteMaskCarryPlan`, `CartierLinearRepresentation`, `CartierAutomaton`
- `mahler_mask_product`, `mahler_mask_inverse`, `compile_finite_mask_carry_plan`, `compile_digit_cauchy_product`

- `compile_cartier_linear_representation`, `compile_rational_mask_automaton`
- `KummerLayerDistribution`, `PrimePowerUnitCompiler`, `PrimePowerKernelCompiler`
- `BellParityAutomaton`, `BellResidueAutomaton`
- `q_binomial_valuation_decomposition`, `q_binomial_support`
- cyclotomic collapse, trusted snapshots, and finite signal adapters

B.18 `komorebi_math.coordinates`

The coordinate namespace provides a general atlas with arbitrary finite bases and subspaces, path composition and commuting diagrams, normalized-series changes, combinatorial transforms, classical and custom polynomial families, recurrence and state-space coordinates, affine and formal charts, tensor and operator transport, prime-field maps, spectral and modal transforms, packet permutations, resource budgets, and trusted portable records. Euler-to-Fibonacci conversion remains one named bridge inside the general system.

C Platforms, backends and PowerShell

C.1 Base package

The supported CPU targets are Linux and Windows on Python 3.11, 3.12 and 3.13. The base package requires NumPy 1.26 or newer. JAX, PyTorch, CuPy, SciPy, SymPy and LaTeX are optional.

The release matrix checks both source execution and clean installations of one common wheel and source distribution on all six combinations of operating system and Python version. Each installed job rebuilds a wheel from the source distribution and executes the same runtime profile against both wheels. The release guide (§ E) defines those checks and their records.

C.2 Optional dependencies

Extra	Requirement	Use
`exact`	SymPy 1.14 or newer	Symbolic algorithms and Smith transformation matrices
`scientific`	SciPy 1.11 or newer, mpmath 1.3 or newer	Scientific and high precision routines
`plot`	Matplotlib 3.8 or newer	Plotting examples
`yaml`	PyYAML 6.0 or newer	YAML input and output
`jax`	JAX 0.4 or newer	Directional automatic differentiation
`torch`	PyTorch 2.1 or newer	Directional automatic differentiation
`autodiff`	JAX and PyTorch	Both derivative adapters
`all`	Exact, scientific, plotting and YAML extras	Combined mathematical dependencies

Directed quadratic-form enclosures specifically require `mpmath==1.3.0`. That arithmetic trust requirement is narrower than the general scientific extra's version range. The `all` extra does not install autodiff, CUDA or development tools.

The minimum CPU dependency job uses Python 3.11 with NumPy 1.26.4, SymPy 1.14.0, SciPy 1.11.4 and mpmath 1.3.0. It tests that combination, rather than every combination allowed by the metadata. Exact environments used for a release are recorded with its qualification results.

C.3 Native backends

JAX and PyTorch are loaded lazily. Availability requires a successful child-interpreter import and the directional JVP capability used by the adapter. Finding an installed package is insufficient. See backend isolation (§ C.5).

CuPy is installed separately because its distribution must match the local CUDA line. The CUDA installation guide (§ C.6) describes the environment boundary, and GPU validation (§ C.7) describes the checked computations.

C.4 Additional environments

macOS, Windows ARM64, Linux ARM64, native Windows JAX GPU and other unrecorded platforms are outside the declared release matrix. They require their own execution records before a support claim is made. A constraints file records one reproduced environment. It does not make every listed dependency mandatory for the base package.

C.5 Optional derivative backends

JAX and PyTorch adapters are loaded lazily. A package being discoverable does not prove that its native runtime can import or provide the required derivative operation. DLL conflicts and native initialization failures can terminate an interpreter before Python raises an exception.

C.5.1 Availability contract

For a backend B, availability means

$$A(B) = D(B) \wedge I(B) \wedge C(B),$$

where D is package discovery, I is successful import in a bounded child interpreter, and C is the required directional Jacobian-vector product (JVP) capability. The adapters use `jax.jvp` and `torch.func.jvp`, respectively.

`ad_framework_diagnostics()` reports discovery, availability, version, return code and a bounded diagnostic. `available_ad_frameworks()` returns the resulting Boolean states. An isolated probe must succeed before a provider attempts to import the backend in the calling process. Python exceptions become `BackendUnavailableError`.

C.5.2 Diagnose the active environment

SHELL

```
python verification/optional_ad_backend_status.py
python -m komorebi_math doctor
```

Use the same interpreter that will execute the calculation. Install the `jax`, `torch` or `autodiff` extra only when that adapter is needed. CPU backend availability does not imply CUDA availability.

C.5.3 Reproduce the integration checks

With working CPU JAX and PyTorch runtimes:

SHELL

```
python verification/stress_release_candidate.py --campaign optional_backends
    ↪  --skip-resources --output-directory qualification/optional-backends
```

All seven selected cases must execute without skips in this strict campaign. The integration checks compare derivatives through order three or four with analytic controls, exercise real and complex dtypes, reject nonfinite evidence and check CPU execution when a PyTorch build advertises CUDA support. That last control alters the reported build metadata to test adapter selection. It does not execute a GPU.

The ordinary source suite can skip unavailable optional backends. The strict campaign instead reports a missing or unusable backend as failed qualification. Both successful and failed records belong with the tested source identity and dependency versions. See the release guide (§ E).

C.6 CUDA installation

The base Komorebi Math wheel needs no GPU runtime. CUDA execution uses optional PyTorch and CuPy installations that must be compatible with the NVIDIA driver and the selected Python interpreter.

Use the official PyTorch installer to choose a CUDA-enabled build for the host. Install one CuPy distribution for the matching CUDA line using the official CuPy instructions. For a configured CUDA 12.x environment, the CuPy package is `cupy-cuda12x`. Do not install it alongside `cupy` or a second CUDA-specific CuPy package.

After configuring the runtimes, install the local Komorebi wheel with the same interpreter:

POWERSHELL

```
python -m pip install .\dist\komorebi_math-1.0.0-py3-none-any.whl
python -m pip check
python -m komorebi_math doctor
python -m komorebi_math verify gpu
```

If using an existing environment, invoke its full `python.exe` path rather than relying on an unrelated `python` on `PATH`. Package detection alone does not establish that the driver, device and native runtimes work together.

The GPU validation guide (§ C.7) describes the actual computations. Maintainers binding a result to exact release files should follow the release workflow (§ E.5).

C.7 GPU validation

The CUDA checks execute PyTorch and CuPy in separate child interpreters. If a worker aborts, times out, fails to import or exits with a nonzero status, the parent interpreter remains alive and records the failure.

For the package installed in the selected environment:

SHELL

```
python -m komorebi_math verify gpu
```

C.7.1 Checked computations

The PyTorch worker checks CUDA availability and evaluates a cubic directional JVP at `2`, with value `8` and tangent `12`. It also checks a device matrix-vector product with expected result `[14, 38, 62, 86]`.

The CuPy worker checks device availability, executes a device-resident `PacketIR`, and compares the result with `[0, 1, 4, 6]` using Euclidean residual norm at most `1e-12`.

These controls check finite computations in the recorded environment. They do not establish GPU performance, complete stress coverage or compatibility with every driver and framework build.

C.7.2 Release records

The hardware producer verifies the installed wheel inventory before and after execution, clears source-path overrides, records interpreter/device/runtime versions and retains the nested worker results. Its temporary installation uses the exact candidate wheel and inherits the selected host's CUDA runtimes.

Follow the release guide (§ E.5) for that artifact-bound command. Keep the complete report, including failed runs. The installed-package diagnostic checks the selected environment. The release record additionally binds that execution to exact distribution files.

C.8 PowerShell workflows

Run these commands from the source checkout or the unpacked source bundle. The wheel path is relative to that directory. Use the same environment for installation, verification and export.

C.8.1 Install the wheel

POWERSHELL

```
$Wheel = ".\dist\komorebi_math-1.0.0-py3-none-any.whl"
python -m venv .venv
$Py = ".\.venv\Scripts\python.exe"
$Komorebi = ".\.venv\Scripts\komorebi.exe"
& $Py -m pip install $Wheel
```

For a fully offline installation, supply a compatible wheelhouse containing NumPy and the other requested dependencies:

POWERSHELL

```
& $Py -m pip install --no-index --find-links .\wheelhouse $Wheel
```

C.8.2 Verify the package

POWERSHELL

```
& $Komorebi info
& $Komorebi doctor
& $Komorebi verify stable
& $Komorebi release check
& $Komorebi api check --stable
& $Komorebi export formats
& $Komorebi latex engines
```

The complete installed suite also needs the `exact`, `scientific` and `yaml` extras. The following command installs the checkout with those extras before running the suite. Use the release guide when the exact wheel must be qualified without replacement:

POWERSHELL

```
& $Py -m pip install ".[exact,scientific,yaml]"
& $Komorebi verify all
```

C.8.3 Export a sequence

POWERSHELL

```
New-Item -ItemType Directory -Force .\out | Out-Null
& $Komorebi export sequence fibonacci --terms 30 --compile --output
   ↪ .\out\fibonacci.json --force
& $Komorebi export sequence fibonacci --terms 30 --compile --output
   ↪ .\out\fibonacci.csv --force
& $Komorebi export sequence fibonacci --terms 30 --compile --output
   ↪ .\out\fibonacci.md --force
& $Komorebi export sequence fibonacci --terms 30 --compile --output
   ↪ .\out\fibonacci.html --force
& $Komorebi export sequence fibonacci --terms 30 --compile --output
   ↪ .\out\fibonacci.tex --force
& $Komorebi export sequence fibonacci --terms 30 --compile --output
   ↪ .\out\fibonacci.pdf --force
```

PDF export requires a local LaTeX engine. By default, the generated `.tex` source is preserved beside the PDF.

C.8.4 Recompile the generated LaTeX

POWERSHELL

```
& $Komorebi latex compile .\out\fibonacci.tex --output
    ↪  .\out\fibonacci_recompiled.pdf
```

C.8.5 Export local arithmetic

POWERSHELL

```
& $Komorebi export standard-window --prime 7 --compression 2 --address 3 --count 24
    ↪  --output .\out\standard_window.tex --title "Standard window" --force
& $Komorebi export euler --prime 2 --index 20 --depth 2 --output .\out\euler20.pdf
    ↪  --title "Dyadic Euler coordinate" --force
```

C.8.6 Convert JSON to LaTeX

POWERSHELL

```
@'
{
  "name": "example",
  "indices": [0, 1, 2, 3],
  "values": [1, 1, 2, 3]
}
'@ | Set-Content -Encoding UTF8 .\out\data.json

& $Komorebi export json --input .\out\data.json --output .\out\data.tex --title
    ↪  "JSON report" --force
& $Komorebi export json --input .\out\data.json --output .\out\data.pdf --title
    ↪  "JSON report" --force
```

C.8.7 Build a multiformat ZIP

POWERSHELL

```
& $Komorebi export bundle --input .\out\data.json --output .\out\data_bundle.zip
    ↪  --formats "json,markdown,html,latex,pdf" --force
```

C.8.8 Test the Python API from PowerShell

POWERSHELL

```
@'
from pathlib import Path
from fractions import Fraction
from komorebi_math.export import DocumentMetadata, ExportOptions, MathDocument,
    ↪ export_data

out = Path("out")
out.mkdir(exist_ok=True)

document = MathDocument(DocumentMetadata(title="Python API report",
    ↪ author="Komorebi P."))
document.paragraph("This report was generated from the installed Python package.")
document.equation(r"\sum_{k=1}^{n} k = \frac{n(n+1)}{2}", numbered=True)
document.table(("n", "value"), ((0, 1), (1, Fraction(1, 2)), (2, Fraction(1, 3))))

for suffix in ("json", "md", "html", "tex", "pdf"):
    result = export_data(document, out / f"python_report.{suffix}",
    ↪ options=ExportOptions(overwrite=True))
    result.certificate.require()
    print(result.path, result.bytes_written)
'@ | & $Py -
```

C.8.9 Source qualification

The release guide (§ E) describes full source tests, strict documentation, offline installed checks and the CPU matrix. Use its neutral Python entry points through `$Py` so that all commands use the same environment.

C.8.10 Run the bundled Python example

POWERSHELL

```
& $Py .\examples\export_reports.py --output .\out\full_report --engine pdflatex
```

Replace `pdflatex` with `xelatex`, `lualatex`, `latexmk`, or `auto`.

D Upgrading applications

D.1 Select an API tier

Applications that need the 1.x compatibility contract should import the stable surface through one of its equivalent aliases:

PYTHON | Executable example

```
from komorebi_math import stable
from komorebi_math import v1
from komorebi_math.api import v1
```

Use one style consistently. Broad domain imports remain available under their documented scope. See the stable API contract (§ 2).

D.2 Polyphase names

The canonical functions are `refine_polyphase` and `coarsen_polyphase`. `polyphase_refine` and `polyphase_coarsen` are identity aliases with the same implementation.

PYTHON | Executable example

```
from komorebi_math.stable import refine_polyphase, coarsen_polyphase
```

D.3 Analysis and synthesis

For a square invertible system, `projector()` describes an algebraic projector. A redundant frame instead has additive reconstruction components, which need not be idempotent. Prefer `component()` and `components()` in new code unless idempotence is separately established.

D.4 Portable records

Retain original portable payloads when upgrading. The predecessor compatibility contract covers the supported `0.15.0a2` and `0.16.0a2` records and checks their producer versions, file/payload digests, reconstructed types and available certificate execution. Codec changes require complete adjacent migrations. Do not edit `__komorebi_type__` or `__version__` manually.

These checks cover declared portable schemas. Pickles, undocumented internal objects and arbitrary third-party codecs are outside the upgrade contract.

D.5 Verify an upgrade

SHELL

```
python -m komorebi_math api check --stable
python -m komorebi_math release contract
python -m komorebi_math release upgrade
python -m komorebi_math verify all
python -m komorebi_math release check
```

A failing stable or persistence contract requires investigation before adopting the new package. Compare the exception and certificate diagnostics, interpreter version and dependency versions with those of the previous working environment.

D.6 Deprecation policy

A stable removal or incompatible change requires a documented transition. Ordinary removals follow an announced deprecation interval and normally take effect in the next major release. A correctness or security defect that cannot safely remain requires an explicit compatibility decision and explanation.

A deprecated callable remains usable during the announced period. It preserves its behavior or delegates transparently, and emits `KomorebiDeprecationWarning`. Its record identifies the first warning version, a replacement when available, the planned removal release and the reason.

PYTHON | Executable example

```
from komorebi_math import deprecated


def replacement(value: int) -> int:
    return value + 1


@deprecated(since="1.1", replacement="replacement", remove_in="2.0")
def legacy(value: int) -> int:
    return replacement(value)
```

The decorator exposes `__komorebi_deprecation__` for tooling. Spelling aliases may remain without warnings when they have unambiguous identical behavior. Preferred device names use `CuPy`; existing `Cupy` implementation spellings remain compatibility aliases.

Persistence versions have a separate obligation: supported older records must remain decodable or have an explicit migration. An API alias does not permit silent reinterpretation of stored data.

Before a removal, review the affected symbols, replacement, warning and removal versions, persistence and CLI surfaces, migration tests and API manifest differences. No existing stable symbol is scheduled for removal in 1.x.

E Reproducibility and qualification

Release qualification binds execution evidence to a specific source tree and set of distribution files. Use one unchanged checkout and one set of distribution files throughout qualification. Keep reports outside the installed package so that recording a wheel's hash does not alter the wheel being identified.

E.1 Prepare the environment

From the checkout root, create an environment for a supported Python minor. Install the development dependencies and the pinned build tools:

SHELL

```
python -m pip install -e ".[dev,exact,scientific,yaml]" "setuptools==82.0.1" wheel
python -m pip check
python verification/build_distributions.py --root . --output-directory dist
    ↪  --report qualification/distributions.json
```

The builder creates `komorebi_math-1.0.0-py3-none-any.whl` and `komorebi_math-1.0.0.tar.gz`. Generated files belong in `dist/` and `qualification/`. They are excluded from the published source payload. Rebuild after changing any bound source or documentation file.

Qualification uses `pytest==8.4.2`. Its JUnit output records concrete parent cases consistently while executing their unittest subtest assertions. Failed subtests fail their parent. Do not run qualification with assertions disabled.

E.2 Source and documentation

SHELL

```
python verification/check_release.py --root . --full-tests --require-mkdocs
    ↪  --output qualification/source.json --markdown qualification/source.md
python verification/check_release_manual.py --profile
    ↪  verification/release_manual_profile.json --output qualification/examples.json
```

The full source report retains the actual run's JUnit XML, digest, counts and failure diagnostics. Strict documentation requires a successful MkDocs build and valid mathematical rendering. A quick or skipped check is recorded with its smaller scope and cannot replace this command.

The installed functional check is separate:

SHELL

```
python -m komorebi_math api check --stable
python -m komorebi_math verify all
python -m komorebi_math release check
```

These commands inspect the installed implementation. They do not establish source cleanliness, a platform matrix or external hardware qualification.

E.3 Clean installed distributions

Prepare a wheelhouse for the operating system, architecture and Python minor that will run the check. The download step can use the network. The subsequent qualification runs offline:

SHELL

```
python -m pip download --only-binary=:all: --dest qualification/wheelhouse
    ↪ "setuptools==82.0.1" wheel numpy "pytest==8.4.2" sympy scipy mpmath PyYAML
python verification/release_artifact_gate.py --root . --runtime-tests
    ↪ --dependency-mode isolated --wheelhouse qualification/wheelhouse --output
    ↪ qualification/installed.json
```

The build interpreter and wheelhouse must use the same setuptools version. The gate creates clean environments without inherited host packages, checks the original wheel, rebuilds a wheel from the source distribution, compares exact bytes and executes the installed profile against both wheels. Missing local dependencies cause failure rather than an online fallback. Installed runtime qualification rejects skipped cases.

E.4 One release, six CPU targets

The hosted workflow builds one common wheel and source distribution for Linux and Windows on Python 3.11, 3.12 and 3.13. The same distribution files are tested on all six targets. Separate jobs check the source matrix, minimum dependencies, documentation, bounded stress and optional CPU backends.

A candidate manifest records the clean Git commit and tree, release identity, source/tool binding and exact distribution hashes. Replace `COMMIT_SHA` with the actual checked-out commit:

SHELL

```
python verification/candidate_evidence.py create --root . --revision COMMIT_SHA
    ↪ --output qualification/candidate.json
python verification/candidate_evidence.py cpu --root . --candidate
    ↪ qualification/candidate.json --artifact-report qualification/installed.json
    ↪ --output qualification/cpu-local.json
```

The producer observes the actual host. A Linux record cannot fill a Windows cell by changing its filename. Aggregate six separate paths for the same candidate in place of `SIX_CPU_RECORDS`:

SHELL

```
python verification/candidate_evidence.py aggregate --root . --candidate
    ↪ qualification/candidate.json --records SIX_CPU_RECORDS --output
    ↪ qualification/cpu.json
```

The reviewer rejects missing or duplicate cells, changed bytes, mismatched identities, incomplete installed profiles and inherited dependencies. Preserve the original records and hosted run provenance. Hashes bind content. They do not identify who executed a program.

E.5 Stress and hardware qualification

All requested stress campaigns must execute. For a promotion review, include both base campaigns and the optional JAX/PyTorch CPU campaign:

SHELL

```
python verification/stress_release_candidate.py --root .
    ↪ --include-optional-backends --revision COMMIT_SHA --output-directory
    ↪ qualification/stress
```

The output directory must be inside the checkout. Reports retain bounded resource controls, selected source hashes, raw outputs and concrete test cases. A missing optional backend fails this strict campaign.

On the NVIDIA host, select the existing interpreter with compatible CUDA-enabled PyTorch and CuPy. The hardware producer installs the exact local wheel in a temporary environment, clears source-path overrides and verifies the installed bytes before executing both backend checks:

POWERSHELL

```
.\Run_Hardware_Qualification.ps1 -Python "C:\path\to\python.exe" -Candidate
    ↪ qualification\candidate.json -Output qualification\hardware.json
```

Dependencies are inherited from that selected hardware environment and the report says so. This is a different isolation claim from the clean CPU gate. Retain successful and failed reports. The checked computations are described in GPU validation (§ C.7).

E.6 Publication review

Package metadata identifies a release stage. Publication qualification requires current records for all six CPU targets, the full source suite, strict documentation, all stress campaigns, positive NVIDIA hardware results, a clean release tag and a successful qualified release candidate (RC) predecessor. Keep these obligations separate from the installed package's self-check.

`verification/release_promotion.py` revalidates the original nested records rather than trusting outer passed flags. Supply the six paths, exact tag and successful RC review in place of the uppercase fields:

SHELL

```
python verification/release_promotion.py --root . --candidate
    ↪ qualification/candidate.json --records SIX_CPU_RECORDS --hardware
    ↪ qualification/hardware.json --publication qualification/source.json --stress
    ↪ qualification/stress/manifest.json --tag RELEASE_TAG --predecessor
    ↪ QUALIFIED_RC_REVIEW --output qualification/promotion.json
```

The final tracked inventory and Git modes must match the RC after the permitted normalization of version and stage metadata. Other source changes require another qualified RC. The historical embedded registry remains a record of its own runs. Version equality alone does not qualify a changed source tree.

`release readiness --final --require-gpu` is a separate diagnostic of that registry and the current hardware host. The external promotion review supplies the exact source/distribution binding.

E.7 Build the source handoff

After the full same-source gate passes:

SHELL

```
python verification/build_source_release.py --root . --output-directory dist
    ↪ --artifact-directory dist --gate qualification/source.json --report
    ↪ qualification/source_release.json
```

The deterministic source ZIP contains clean source files, the distributions and a source manifest. It excludes local environments, cached bytecode, historical reports and generated qualification output. A clean extraction is replayed to check the handoff. Preserve its checksum with the reviewed records. Creating these files does not itself merge a pull request, create a release tag or publish to a package index.

F Glossary

The terms below describe how the library represents and checks finite mathematical objects. Their precise hypotheses and arithmetic rules are given in the relevant chapters.

Term	Meaning in this manual
Analysis	A map from a represented object into a chosen coordinate space.
Arithmetic domain	The scalar rules used by a computation, such as integers, rational numbers, a finite field or floating point.
Certificate	A record of validity, evidence class, residuals and assumptions produced by a specified operation.
Codec	A declared representation and reconstruction rule for a portable object, with an identity and schema version.
Conditional result	A conclusion that retains assumptions not established by the computation.
Coordinate transform	A map between finite descriptions of data. Invertibility requires its own hypotheses or diagnostics.
Defect	A measured or exact failure of a stated identity, recurrence, compatibility condition or lifting equation.
Directed enclosure	Bounds computed with outward arithmetic under an explicitly stated arithmetic backend and trust model.
Exact result	A finite statement established using the producer's declared exact arithmetic and hypotheses.
Finite carrier	The explicitly represented finite space or data structure on which an operation is defined.
Grammar	A specification of legal objects, arrows and operator paths, together with the relations required by a realization.
Independent control	A comparison based on a separate formula or construction; its own arithmetic assumptions remain relevant.
Normal form	A canonical finite representation used to compare terms within a specified expression language.
Numerical result	A floating computation assessed by a stated residual, tolerance or error rule.

Term	Meaning in this manual
Obstruction	A condition that prevents a specified lift, compatibility relation or completion from being achieved.
Packet	A component selected by the projections or decomposition rules of a particular construction.
Polyphase coordinates	A decomposition of a finite signal into residue classes of its sample indices.
Provenance	Information identifying the objects, assumptions and operation to which evidence belongs.
Realization	Concrete spaces and operators that implement a specified finite grammar and its relations.
Refinement	A change to a more detailed representation with specified compatibility maps and identities.
Residual	A producer-defined quantity used to assess a stated identity or approximation. A residual has meaning only with its comparison rule.
Stable API	The reviewed interface with the declared compatibility promise for application code.
Synthesis	A map from coordinates back to the represented object. In a redundant system, coordinate roundtrip need not be the identity.
Transfer	An operator or compatible map that transports information between represented components.
Typed expression	A term whose source and target coordinate spaces, operations and evaluation rules belong to a specified finite language.

F.1 License

Original text and software are distributed under the MIT License. Third-party components and separately supplied datasets retain their own notices and terms. The manual uses the library's documented finite mathematical contracts.